

GameVerse: Can Vision-Language Models Learn from Video-based Reflection?

Kuan Zhang^{*1} Dongchen Liu^{*1} Qiyue Zhao¹ Jinkun Hou¹
Xinran Zhang¹ Qinlei Xie¹ Miao Liu^{†1} Yiming Li^{†1}

<https://gameverse-bench.github.io/>

Abstract

Human gameplay is a visually grounded interaction loop in which players act, reflect on failures, and watch tutorials to refine strategies. Can Vision-Language Models (VLMs) also learn from video-based reflection? We present **GameVerse**, a comprehensive video game benchmark that enables a *reflective visual interaction loop*. Moving beyond traditional *fire-and-forget* evaluations, it uses a novel *reflect-and-retry* paradigm to assess how VLMs internalize visual experience and improve policies. To facilitate systematic and scalable evaluation, we also introduce a *cognitive hierarchical taxonomy* spanning 15 globally popular games, *dual action space* for both semantic and GUI control, and *milestone evaluation* using advanced VLMs to quantify progress. Our experiments show that VLMs benefit from video-based reflection in varied settings, and perform best by combining failure trajectories and expert tutorials—a *training-free* analogue to reinforcement learning (RL) plus supervised fine-tuning (SFT).

1. Introduction

Video games have been closely linked to artificial intelligence (AI) since its inception (Turing, 1953). These virtual worlds provide diverse environments with varying dynamics and objectives. Humans adapt to these worlds via a visually grounded interaction loop (see Figure 1): they observe, act, reflect on failures, and consult external resources to refine their strategies. This ability to perceive, act, and learn from mistakes is fundamental to human intelligence, enabling continual adaptation through experience.

Motivated by the richness and diversity of video games,

^{*}Equal contribution ¹College of AI, Tsinghua University, China. Correspondence to: Yiming Li <yimingli9702@gmail.com>, Miao Liu <lmptx4869@gmail.com>.

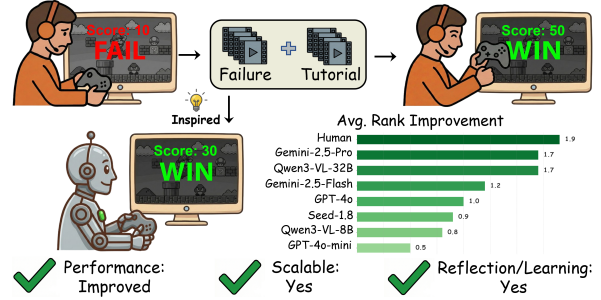


Figure 1. Humans improve gameplay by reflecting on failures and consulting expert tutorials. Our benchmark mimics this process, enabling agents to learn from video-based reflection. Humans show the largest gains, while all models benefit to varying degrees.

recent work has adopted them as embodied AI benchmarks for Vision-Language Models (VLMs). Unlike static datasets (e.g., VQA (Antol et al., 2015)), which risk saturation, data quality (Zhang et al., 2025b) and data contamination (Li et al., 2024), video games provide high quality, dynamic, visually rich, and long-horizon environments that demand a synthesis of perception, planning, reasoning, and motor control. However, building a comprehensive benchmark capturing human-like gameplay remains non-trivial.

In this work, we address this gap by establishing the desiderata for a comprehensive video game benchmark for VLM agents. **(1) Systematic Taxonomy.** Unlike the physical world, video games come in a wide variety of genres and settings. A fine-grained categorization is essential to rigorously evaluate agent performance across various scenarios. **(2) Vision-Centric Input.** Human gameplay relies solely on visual input. A good benchmark should therefore mimic this setup, allowing agents to learn from pixels without privileged information. **(3) Failure Reflection.** Most importantly, agents must be able to reflect on past failures and leverage external resources (e.g., tutorials) to improve.

Unfortunately, a comprehensive benchmark unifying all three aspects remains lacking (see Table 1). Existing benchmarks often omit a systematic game taxonomy, e.g., relying on coarse classifications such as commercial genres (e.g., RPG, FPS) (Zheng et al., 2025; Hu et al., 2025a; Park et al., 2025). Moreover, many depend on scaffolding mechanisms

Table 1. **GameVerse** (GV) distinguishes itself through a **Cognitive Taxonomy** rather than commercial genres (*Comm.Genre*), **Dual Action Space**, and a **Visually-Grounded Interaction Loop** allowing agents to reflect and retry. *Mixed* features selecting games spanning various eras and genres. Tag † denotes using scaffolds/APIs to extract ground-truth state text, with vision as an auxiliary role.

Benchmarks	Game Env.	Taxonomy	# Games	Diff. Tiers	Major Modality	Privilege-Free	Paradigm	Action	Scalability
GameArena (Hu et al., 2025b)	1D Text World	✗	3	✗	Text-Centric	✓	Fire-and-Forget	Semantic	✓
BALROG (Paglieri et al., 2025)	1D Text + 2D Grid	✗	6	✓	Text-Centric†	Scaffold	Fire-and-Forget	Semantic	✗
LVLM-Play (Wang et al., 2025b)	2D Grid	✗	6	✓	Vision-Centric	✓	Fire-and-Forget	Semantic	✓
VideoGameBench (Zhang et al., 2025a)	2D Retro Arcade	✗	10	✗	Vision-Centric	✓	Fire-and-Forget	GUI	✓
FlashAdventure (Ahn et al., 2025)	2D Web Flash	✗	34	✗	Vision-Centric	✓	Fire-and-Forget	GUI	✗
V-MAGE (Zheng et al., 2025)	2D Retro Arcade	Comm. Genre	5	✓	Vision-Centric	✓	Fire-and-Forget	Semantic	✓
LMGAME-BENCH (Hu et al., 2025a)	2D Mixed	Comm. Genre	6	✗	Text-Centric†	API + Scaffold	Fire-and-Forget	Semantic	✗
Orak (Park et al., 2025)	2D Mixed + 3D Mixed	Comm. Genre	12	✗	Text-Centric†	API + Scaffold	Fire-and-Forget	Semantic	✗
GameVerse (Ours)	2D Mixed + 3D Mixed	Cognitive	15	✓	Vision-Centric	✓	Reflect-and-Retry	Dual	✓

that extract ground-truth state information as text (Paglieri et al., 2025; Park et al., 2025; Hu et al., 2025a), rather than requiring agents to reason directly from pixels. This setup departs from human-like real-world interaction and limits scalability to closed-source games. Most importantly, all existing benchmarks employ a *fire-and-forget* paradigm, measuring instantaneous performance without accounting for failure reflection and tutorial-driven improvement.

To this end, we develop **GameVerse** (GV), a comprehensive benchmark to evaluate VLM agents in a human-like manner. GameVerse distinguishes itself through a *diverse environment matrix* spanning 15 globally popular games, *dual action space* supporting both semantic and GUI control, and a *visually grounded interaction loop* allowing failure reflection and tutorial learning (see Table 1 and Figure 2). Our major contributions are summarized as follows.

- **Cognitive Hierarchical Taxonomy.** We build a brand-new fine-grained taxonomy based on three cognitive axes: *image structure*, *temporal dynamics*, and *causal linearity*. By classifying games into five distinct categories and three difficulty tiers, we can precisely map the capability boundaries of different models.
- **Video-Based Reflection.** We propose a novel *reflect-and-retry* paradigm to enable visual experience internalization and policy refinement. Unlike conventional *fire-and-forget* evaluation, we allow agents to diagnose their failure trajectories against expert video tutorials and leverage this feedback in subsequent attempts.
- **Scalable Evaluation Protocol.** We introduce a milestone scoring pipeline that leverages advanced VLMs to quantify agent progress purely from pixels, without relying on internal APIs or manual annotation. This enables scalable evaluation of closed-source games.

We conduct comprehensive experiments and reveal that while VLMs succeed in simple tasks, they struggle to generalize in complex games. Besides, reflection yields improvements, but VLMs are incapable of human-level adaptation from experience. Notably, the largest gains emerge from jointly leveraging failures and tutorials—a *training-free paradigm* that combines *in-context exploration* (failure-as-RL) with *in-context imitation* (tutorial-as-SFT). This syn-

ergy outperforms either approach alone, mirroring the complementary benefits of reinforcement plus supervised learning in foundation model post-training (Chu et al., 2025).

2. Related Work

Game Environments as AI Benchmarks. Early game agents relied on reinforcement learning (RL) to solve simple environments (Bellemare et al., 2013; Towers et al., 2024). Recent attention has shifted toward tackling complex video games with general-purpose Vision-Language Models (VLMs). Balrog (Paglieri et al., 2025) and LVLM (Wang et al., 2025b) are primarily based on 2D-grid worlds. V-MAGE (Zheng et al., 2025), VGB (Zhang et al., 2025a) and GameStore (Ying et al., 2026) assess VLMs with strict visual input in classic video games but suffer from poor performance and struggle to differentiate between models. Recently, Orak (Park et al., 2025) and Lmgame-bench (Hu et al., 2025a) have focused on popular video games but rely heavily on internal game states via APIs for scaffolding; however, such privileged access is unrealistic for a human-like play-learning loop and does not scale to closed-source commercial titles.

VLM Agents on Games. Video game agents typically fall into modular or learning-based categories. Modular frameworks leverage frozen models with scaffolding: Voyager (Wang et al., 2023) uses a code policy for exploration; Cradle (Tan et al., 2025b) employs a six-module cognitive architecture; FlashAdventure (Ahn et al., 2025) integrates clue-seeking for puzzles; and Gemini Plays Pokemon utilizes a memory module (Joel, 2025). These systems exhibit the power of prompt engineering but often struggle with real-time precision. Learning-based approaches bridge the execution gap via training: CombatVLA (Chen et al., 2025) and JARVIS-VLA (Li et al., 2025b) improve performance via multi-stage training. UI-TARS, Game-TARS, and Nitro-Gen (Qin et al., 2025; Wang et al., 2025a;c; Magne et al., 2026) validate model capability to control games via atomic GUI actions, while Lumine (Tan et al., 2025a) achieves real-time interaction, showcasing live gameplay. While these agents perform well, they require extensive training on massive datasets of image-(language)-action pairs.

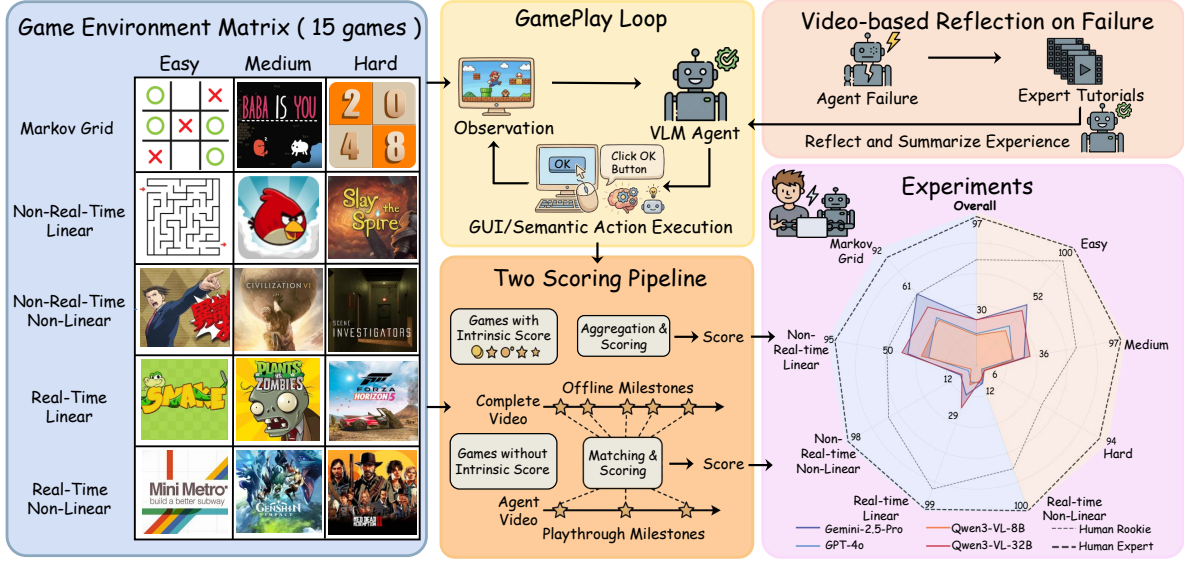


Figure 2. Overview of GameVerse, which effectively probes the capability boundaries of VLMs in diverse video game worlds. GameVerse supports dual action space, enables human-like reflection by integrating failure and tutorial videos, and delivers process score.

Self-Reflection in Agents. The ability to reflect on past actions is a hallmark of advanced intelligence. Early work in LLMs pioneered text-based self-reflection (Shinn et al., 2023; Madaan et al., 2023). Reflection mechanisms evolved as agents moved into embodied environments: Voyager (Wang et al., 2023) used environment feedback within a Minecraft prompt mechanism to refine its skill library; GROOT (Cai et al., 2024) employed videos as direct instructions for open-world tasks, training agents to guide their behavior; R3V (Cheng et al., 2025) reflects on Chain-of-Thought rationales to enhance multimodal reasoning; and ROE (Xu et al., 2025) utilized predefined expert text experience, updated from past episodes in Starcraft II. Despite these advances, existing methods either learn offline from videos (MineDojo (Fan et al., 2022), GROOT (Cai et al., 2024)) or reflect internally with complex workflows (R3V (Cheng et al., 2025), ROE (Xu et al., 2025)). They do not replicate the direct, purposeful human gameplay loop of watching a tutorial to diagnose a specific past failure.

3. GameVerse

In this section, we will detail the proposed hierarchical game taxonomy, the video-based reflection paradigm, and the novel evaluation metrics (see Figure 2).

3.1. Cognitive Hierarchical Game Taxonomy

Recent benchmarks (Zhang et al., 2025a; Hu et al., 2025a; Park et al., 2025) usually lack of a systematic taxonomy. They typically categorize games by commercial genres (e.g., RPG). While useful for consumers, they are too coarse for analyzing the bottleneck of VLM agents. For instance, a

Strategy game can range from a simple grid, turn-based task (e.g., Tic-Tac-Toe) to a complex, real-time, partially observable task (e.g., StarCraft), but they vary significantly in difficulty for the evaluation of VLM agents.

Motivated by traditional game design (Zimmerman & Salen, 2003; Juul, 2011), to evaluate the boundaries of VLM capabilities, we move beyond commercial genres taxonomy. We construct a hierarchical taxonomy based on three cognitive axes: *Image Structure* (Grid/2D/3D), *Temporal Dynamics* (Real-time/Non-Real-time) and *Causal Linearity* (Linear/Non-linear), resulting in five distinct categories: **Markov Grid**: Turn-based discrete state transitions with full observability. **Non-Real-time Linear**: Turn-based progression with a fixed narrative path. **Non-Real-time Non-linear**: Turn-based mechanics with open-ended goals. **Real-time Linear**: Continuous time constraints with singular objectives. **Real-time Non-linear**: Continuous dynamics with complex, branching objectives. For each category, we curate games across three difficulty tiers: *easy*, *medium*, and *hard* by factors like environmental complexity, action space, and etc (see Appendix C). It probes the capability boundaries of VLM agents. In total, GV selects 15 globally popular video games, distributed across these categories: (*Tic-Tac-Toe*, *Baba Is You*, 2048), (*Maze*, *Angry Birds*, *Slay the Spire*), (*Ace Attorney*, *Civilization VI*, *Scene Investigator*), (*Snake*, *Plants vs. Zombies*, *FORZA HORIZON 5*), and (*Mini Metro*, *Genshin Impact*, *Red Dead Redemption 2*).

Dual Action Space. To assess both high-level reasoning and low-level control, we define two action mode. Semantic Actions A_S : High-level semantic actions (e.g., "Position(1,3)"), testing the agent’s perception and reasoning. GUI Actions A_G : Low-level GUI operations (e.g., KeyPress(A)), testing

Table 2. Overview of the 15 games in GameVerse. We detail the number of trials (N), average time per round (T), supported evaluation modes (Semantic A_S and GUI A_G), and a concise summary of the game content and the specific cognitive capabilities evaluated.

Game	Trials (N) / Time (T)	Eval. Mode	Feature & Capabilities Evaluated
<i>Markov Grid</i>			
Tic-Tac-Toe	10 / 40 sec	A_S, A_G	Classic zero-sum grid game; tests basic adversarial logical reasoning and turn-taking planning .
Baba Is You	5 / 1.5 min	A_S, A_G	Puzzle with manipulable rules; tests out-of-the-box thinking and flexible logical inference .
2048	5 / 10 min	A_S, A_G	Stochastic number puzzle; tests probabilistic planning and spatial foresight to manage grid constraints.
<i>Non-Real-Time Linear</i>			
Maze	10 / 45 sec	A_S, A_G	Navigational puzzle; tests virtual perception and spatial pathfinding algorithms.
Angry Birds	5 / 2 min	A_S, A_G	Physics-based puzzle; tests spatial intelligence and understanding of causal physical interactions .
Slay the Spire	3 / 20 min	A_S, A_G	Roguelike deck-builder; tests long-term strategic resource management and combinatorial optimization .
<i>Non-Real-Time Non-Linear</i>			
Ace Attorney	5 / 15 min	A_G	Narrative-driven adventure; tests high-level reading comprehension and contradiction detection .
Civilization VI	3 / 30 min	A_G	Grand strategy; tests complex multi-step planning , economic management , and long-horizon reasoning .
Scene Investigators	3 / 50 min	A_G	Deductive investigation; tests visual observation , information synthesis , and causal inference .
<i>Real-Time Linear</i>			
Snake	10 / 1 min	A_S, A_G	Real-time arcade; tests reflex-based control and spatial path planning under growing constraints.
Plants vs. Zombies	5 / 3 min	A_S, A_G	Tower defense; tests real-time tactical deployment and efficient resource allocation .
Forza Horizon 5	3 / 3 min	A_G	High-fidelity racing simulation; tests fine-grained continuous control and visual-motor coordination .
<i>Real-Time Non-Linear</i>			
Mini Metro	5 / 5 min	A_G	Subway simulation; tests graph optimization and efficient flow management in evolving networks.
Genshin Impact	3 / 30 min	A_G	Cartoon open-world RPG; tests embodied exploration , 3D navigation , and real-time combat mechanics .
Red Dead Redemption 2	3 / 2 hr	A_G	Real open-world RPG; tests comprehensive general intelligence , realistic physics , and social interaction .

end-to-end precise visual control.

Game Description. As shown in Table 2, for each game, we provide a brief description of the game state, number of trials (N) and average estimated time for each person per round (T), and supported evaluation mode. Due to cost and time constraints, we limited the evaluation to 3 $\sim$ 20 trials, consistent with prior work, 1 in VideoGameBench (Zhang et al., 2025a), 3 in LMGAME-Bench (Hu et al., 2025a), 3 $\sim$ 20 in Orak (Park et al., 2025). More descriptions of environments and tasks are elaborated in Appendix C and D.

3.2. Video-based Reflection and Learning

Existing benchmarks predominantly use a *fire-and-forget* workflow, maximizing immediate performance while ignoring reflection—a key dimension of human intelligence. We address this with a video-based reflection paradigm, enabling agents to refine gameplay by observing failures and tutorials. The paradigm consists of four steps. **(1) Trial and Failure.** The agent first attempts the game task. If it fails, the system records the sequence of visual observations leading to the negative outcome. **(2) Expert Demonstration Retrieval.** The system retrieves an expert walkthrough from online gameplay videos. **(3) Visual Reflection.** The VLM acts as a reflector. It takes both its failure trajectory and the expert demonstration as visual input. The model is prompted to contrast the two, analyzing the divergence in strategy and execution, and generating condensed empirical reflections (e.g., "I failed because I targeted the wrong support beam; the expert targeted the central pillar"). **(4) Policy Update.** These reflections are injected into the agent’s system prompt, enabling it to re-attempt the task with the new context.

3.3. Scoring Metrics

We use a hybrid evaluation strategy tailored to each game’s feedback mechanism across the 15-game suite.

Intrinsic Scoring. For games that provide fine-grained, numerical feedback or clear competitive metrics (8 games, e.g., 2048 and etc.), we utilize the game’s native scoring systems as the primary performance metrics.

Milestone Scoring. Evaluation in complex games (7 games, e.g., Genshin and etc.) is challenging due to sparse rewards and the absence of quantifiable scores. We propose a scalable pipeline that utilizes advanced VLMs (*Gemini-3-pro* in this paper) to quantify progress purely from pixels.

Phase 1: Offline Milestone Detection. We employ an advanced VLM to watch expert walkthrough videos of the target task. The model extracts a structured JSON reference, denoted as $M_{ref} = \{M_1, M_2, \dots, M_N\}$. Each milestone M_i contains three attributes: (1) Timestamp: The time of occurrence in the video. (2) Title: A concise label for the event. (3) Description: A detailed description of the scene.

Phase 2: Online Scoring. Following gameplay, an advanced VLM analyzes the playthrough video, comparing the agent’s trajectory against the reference milestones M_{ref} . By matching visual states to these milestones, the system calculates a process-oriented score $S = \frac{|M_{finished}|}{|M_{ref}|} \in [0, 1]$. This method scales easily to closed-source commercial games where internal state information is inaccessible.

Notably, all milestones in GameVerse are manually verified to ensure accurate evaluation. We find them to be highly representative and suitable for direct use. Despite occasional hallucinations, they require minimal human verification (see Appendix A.4). This significantly reduces

labor intensity compared to prior works (Chen et al., 2024; Tan et al., 2025b; Ahn et al., 2025) and ensures scalability.

4. Experiment

4.1. Experimental Setup

Models. We evaluate the performance of 7 VLMs to assess the current landscape of capabilities. The models include 2 open-source VLMs: Qwen3-VL-8B/32B (Bai et al., 2025), and 5 proprietary VLMs: GPT-4o/4o-mini (OpenAI et al., 2024), Seed-1.8 (Bytedance Seed Team, 2025) and Gemini-2.5-Pro/Flash (Comanici et al., 2025).

Setup. We conduct experiments in two modes to verify model’s *reflect-and-retry* ability in 3.2. W/o video-based reflection ($VR = No$): The agent plays the game directly. W/ video-based reflection ($VR = Yes$): The agent utilizes failure and expert videos to refine its policy in next attempt. We retrieve the top-ranked videos on public platforms. Prompts, FPS, and video details are in Appendix A.3.

Baselines. To contextualize model performance, we compare them against three baselines. (1) **Random.** Selects valid actions uniformly at random. (2) **Human Rookie.** Users who are computer literate but unfamiliar with the target games. (3) **Human Expert.** Users who are computer literate and possess significant prior experience with the target games. We recruited 37 participants, conducting a total of 458 gameplay sessions. Further details are in Appendix B.

Evaluation Metrics. For each game, we report the average normalized scores on a **100-point scale** with standard deviation. All performance metrics were manually verified. For 10 games, we use a zero-shot agent, where only the current screenshot and the vision-action pair from the last step are provided as context. For 5 games, we employ a memory-aware agent, which utilizes a memory module to store long-term and short-term game history. The comparison between zero-shot and memory agents is detailed in Appendix D. More detailed configurations are in Appendix A.

4.2. Performance Overview

Table 3 presents the performance of 7 VLMs across different game categories in GameVerse. Specifically, **Gemini-2.5-Pro** achieves the highest average ranking (7.5/9.2) among all VLMs. It is closely followed by **Gemini-2.5-Flash** (8.1/9.3), **Seed-1.8** (8.4/9.3) and the open-weights model **Qwen3-VL-32B** (7.8/9.5), establishing a clear performance hierarchy.

Surpassing the Random Baseline. Our results establish that current VLMs have acquired basic gameplay capabilities under the vision-centric condition. Across almost all games, models consistently outperform the random baseline, indicating that they are not merely acting stochastically but engaging in meaningful visual reasoning and execution.

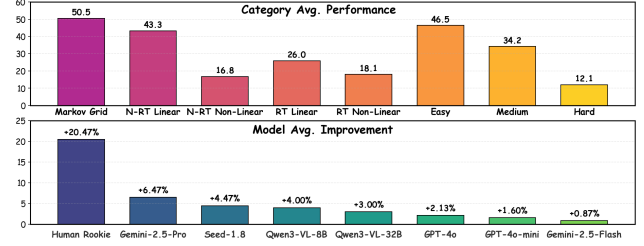


Figure 3. **Top:** Average performance across 5 cognitive and 3 difficulty categories. **Bottom:** Average improvement across models.

The Rich-Get-Richer Effect in Video-Based Reflection.

We observe that reflection generally yields improvements, but its efficacy is non-uniform. Data reveals two opposing trends. As shown in Figure 3 (Bottom), gains scale positively with model capability. Gemini-2.5-Pro improves 6.47% significantly more compared to GPT-4o-mini with 1.60%. As shown in Figure 4 (Left), gains scale negatively with cognitive complexity and difficulty, dropping from 4.4% in Non-Real-time to 1.7% in Real-time. This mirrors a **Rich-Get-Richer** phenomenon: models require a basic threshold of reasoning capital to effectively convert reflection into policy updates. Strong models possess sufficient capacity to internalize reflection, whereas weak models or even strong ones facing overwhelming complexity, fail to ground these insights into perception, reasoning and execution, resulting in diminishing returns.

The Generalization Gap: Robust Humans vs. Brittle Agents.

Human players demonstrate remarkable generalization: they maintain high competitiveness and adaptivity regardless of whether the environment is a simple grid game or a complex, open-world simulation. In contrast, VLM agents exhibit severe degradation as complexity increases, lacking generalization. In *Easy* games like *Tic-Tac-Toe*, Gemini-2.5-Pro achieve perfect scores (100), effectively matching human expert baselines (98.9). In *Hard* games that demand long-horizon reasoning or real-time control, such as *Scene* or *RDR 2*, model performance collapses to 0, falling short of human rookie (58.4 ~ 70.4). While models have mastered distinct, simple tasks, they lack the generalization capability of humans to transfer knowledge and skills to more challenging and dynamic environments.

The Knowing-Doing Gap: Semantic vs. GUI Control.

Although these models have demonstrated impressive performance on static GUI benchmarks (Xie et al., 2024; Li et al., 2025a), the precise visual grounding still remains a bottleneck in challenging video games. As shown in Table 4, model averages 50.5 in semantic mode, consistently outperforming the GUI mode average of 33.5. While current VLMs possess strong reasoning capabilities for high-level planning, they still struggle with the visual grounding required to translate these plans into precise pixel coordinates.

Table 3. Performance of 7 VLMs on 15 GameVerse games in GUI mode, with and without Video-based Reflection (VR). **Green** / **Red** cells indicate performance gain/drop after applying VR. **Bold** represents the top performance across all models and human baselines.

Model	VR	Markov Grid			Non-Real-time Linear			Non-Real-time Non-Linear			Real-time Linear			Real-time Non-Linear			Avg
		TicTacToe	Baba	2048	Maze	AngryBird	Slay	Attorney	Civilization	Scene	Snake	PvZ	Horizon	Metro	Genshin	RDR 2	
Three Baselines																	
Random	—	26.3±19.3	10.8±7.6	1.2±0.9	21.3±2.7	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	7.0±9.1	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	16.5
Human Expert	—	98.9±2.6	100.0±0.0	77.1±16.4	100.0±0.0	85.3±7.2	99.4±1.2	100.0±0.0	100.0±0.0	94.2±6.3	100.0±0.0	100.0±0.0	98.1±4.4	100.0±0.0	100.0±0.0	100.0±0.0	1.6
Human Rookie	No	85.1±19.3	83.4±15.2	15.9±4.1	99.3±1.2	46.4±10.3	34.2±7.1	77.4±27.2	54.1±17.2	70.4±5.2	93.2±10.4	89.3±10.8	73.9±9.1	91.2±15.4	63.3±18.2	58.4±21.3	4.1
	Yes	99.4±1.9	100.0±0.0	47.3±18.2	100.0±0.0	74.2±5.4	81.3±14.1	100.0±0.0	96.4±8.9	90.1±9.3	100.0±0.0	97.9±4.4	85.3±3.1	98.4±4.2	88.2±18.4	83.1±21.2	2.2
Seven Vision-Language Models																	
Qwen3-VL-8B	No	53.3±15.4	60.0±0.0	4.4±2.1	80.3±45.4	19.6±8.3	11.7±0.0	7.1±6.2	0.0±0.0	6.3±4.4	2.2±6.8	33.4±4.1	3.2±0.0	7.8±5.2	14.3±0.0	7.2±5.4	12.4
	Yes	51.8±14.6	70.4±8.3	5.1±1.2	89.4±31.1	31.2±10.8	7.4±8.4	26.1±19.2	0.0±0.0	6.3±4.4	0.0±0.0	51.3±21.4	3.2±0.0	8.3±4.2	14.3±0.0	7.2±5.4	11.6
Qwen3-VL-32B	No	70.6±17.4	73.3±9.1	6.4±3.2	100.0±0.0	41.6±7.4	8.2±4.1	17.3±6.3	8.3±0.0	8.3±0.0	42.6±26.2	41.2±12.6	3.2±0.0	5.4±4.2	14.3±0.0	7.2±5.4	9.5
	Yes	63.1±7.4	80.0±0.0	6.1±3.7	100.0±0.0	59.4±10.9	11.3±1.4	37.2±6.4	8.3±0.0	8.3±0.0	33.4±16.9	48.3±19.0	3.2±0.0	11.2±6.3	14.3±0.0	10.0±0.0	7.8
GPT-4o-mini	No	34.4±17.2	60.0±0.0	1.3±0.4	77.1±43.4	16.7±11.3	2.0±0.0	0.0±0.0	0.0±0.0	3.2±4.1	0.0±0.0	27.6±14.0	3.2±0.0	4.2±3.4	14.3±0.0	10.0±0.0	14.1
	Yes	43.2±12.1	68.3±14.9	1.4±1.2	91.3±26.2	13.3±10.2	2.0±0.0	0.0±0.0	0.0±0.0	3.2±4.1	0.0±0.0	22.4±10.4	3.2±0.0	7.1±2.4	14.3±0.0	10.0±0.0	13.6
GPT-4o	No	58.6±4.2	60.9±1.4	3.8±0.9	80.2±44.1	14.3±9.0	2.0±0.0	18.2±6.1	0.0±0.0	0.0±0.0	0.0±0.0	32.2±12.4	2.4±2.2	11.2±2.1	14.3±0.0	10.0±0.0	13.3
	Yes	64.2±9.8	60.0±0.0	3.4±1.1	85.1±39.4	15.4±12.3	2.0±0.0	33.3±11.1	0.0±0.0	0.0±0.0	0.0±0.0	36.4±15.2	3.2±0.0	14.3±3.4	14.3±0.0	10.0±0.0	12.3
Seed-1.8	No	92.3±3.4	80.0±0.0	9.1±5.2	87.8±20.9	29.4±7.1	39.6±21.2	33.3±0.0	22.2±10.4	3.2±4.1	7.2±10.1	26.1±8.2	5.4±2.1	1.3±2.4	14.3±0.0	10.0±0.0	9.3
	Yes	100.0±0.0	80.0±0.0	18.2±8.4	100.0±0.0	57.2±12.7	28.4±8.1	59.3±6.2	19.5±7.8	8.3±0.0	10.4±10.8	22.1±10.3	2.4±2.2	1.4±1.2	14.3±0.0	7.2±5.4	8.4
Gemini-2.5-Flash	No	88.2±16.4	80.0±0.0	7.8±2.1	100.0±0.0	9.4±8.2	2.0±0.0	33.3±11.1	0.0±0.0	3.2±4.1	20.4±17.9	51.1±13.2	5.4±2.1	14.6±2.7	14.3±0.0	10.0±0.0	9.3
	Yes	95.1±12.4	80.0±0.0	11.4±3.8	100.0±0.0	10.2±6.4	2.0±0.0	29.3±25.1	0.0±0.0	8.3±0.0	10.4±8.1	55.3±10.4	5.4±2.1	19.1±9.6	14.3±0.0	13.4±4.6	8.1
Gemini-2.5-Pro	No	90.0±9.0	80.0±0.0	13.2±3.1	100.0±0.0	32.7±12.8	4.4±3.4	37.2±6.4	0.0±0.0	0.0±0.0	24.2±30.1	33.4±17.2	3.2±0.0	11.3±3.1	14.3±0.0	10.0±0.0	9.2
	Yes	100.0±0.0	80.0±0.0	26.4±6.8	100.0±0.0	42.2±11.3	7.9±0.0	48.3±6.1	0.0±0.0	0.0±0.0	68.4±32.2	30.3±16.4	5.4±2.1	18.2±8.1	14.3±0.0	10.0±0.0	7.5

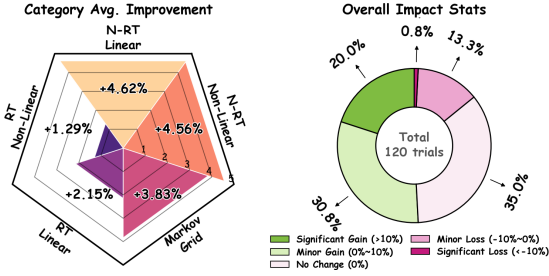


Figure 4. **Left:** Average improvement across 5 cognitive category tasks. **Right:** Overall improvement statistics of 120 trials.

Table 4. Performance of 4 VLMs on 4 games of GameVerse, evaluated with and without Video-base Reflection (VR) in two modes.

Model	VR	TicTacToe		AngryBird		Slay the Spire		Plants vs. Zombies	
		GUI	Semantic	GUI	Semantic	GUI	Semantic	GUI	Semantic
Random	—	26.3±19.3	30.4±24.1	0.0±0.0	7.4±6.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0
Qwen3-VL-8B	No	53.3±15.4	51.7±13.6	19.6±8.3	37.2±11.7	11.6±0.0	11.7±0.0	33.4±4.1	54.2±5.8
	Yes	51.8±14.6	67.0±12.9	31.2±10.8	63.2±15.1	7.4±8.4	5.4±5.2	51.3±14.4	65.2±11.3
GPT-4o	No	58.6±4.2	66.8±10.2	14.3±9.0	49.3±13.0	2.0±0.0	25.3±10.2	32.2±12.4	42.7±16.0
	Yes	64.2±9.8	64.3±8.0	15.4±12.3	61.5±17.6	2.0±0.0	29.4±8.1	36.4±15.2	53.4±7.8
Gemini-2.5-Flash	No	88.2±16.4	93.8±9.1	9.4±8.2	45.8±9.2	2.0±0.0	11.6±0.0	51.1±13.2	70.2±8.5
	Yes	95.1±12.4	100.0±0.0	10.2±6.4	68.4±15.4	2.0±0.0	16.4±0.0	55.3±10.4	81.8±17.0
Gemini-2.5-Pro	No	90.0±9.0	100.0±0.0	32.7±12.8	64.2±11.3	4.4±3.4	37.6±23.2	33.4±17.2	45.4±3.3
	Yes	100.0±0.0	100.0±0.0	42.2±11.3	81.0±15.8	7.9±0.0	28.5±8.0	30.3±16.4	52.7±15.2
Human Rookie	No	85.1±19.3	85.1±19.3	46.4±10.3	46.2±9.8	34.2±7.1	34.2±7.1	89.3±10.8	89.3±10.8
	Yes	99.4±1.9	99.4±1.9	74.2±5.4	74.0±5.2	81.3±14.1	81.3±14.1	97.9±4.4	97.9±4.4
Human Expert	—	98.9±2.6	98.9±2.6	85.3±7.2	85.4±7.3	99.4±1.2	99.4±1.2	100.0±0.0	100.0±0.0

Besides, this gap suppresses the gains from learning. The gains $\sim 8.7\%$ in semantic mode are consistently higher than $\sim 3.75\%$ in GUI mode. While agents successfully learn the strategies, the strategies fail to translate into effective gameplay, highlighting a disconnect between high-level reasoning and low-level execution.

The Latency-aware Evaluation. Figure 5 reveals a performance divergence between reasoning and reactive models under three latency constraints. Reasoning models, Gemini-2.5-Pro and Seed-1.8, demonstrate high sensitivity to time constraints. In PvZ, their performance peaks (~ 85) in

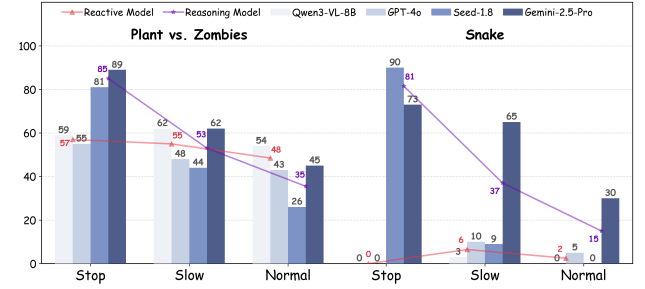


Figure 5. Performance of 4 VLMs on 2 games of GameVerse in semantic action mode with three latency settings. Reactive model is average performance of GPT-4o and Qwen3-VL-8B. Reasoning model is average performance of Seed-1.8 and Gemini-2.5-Pro.

the *Stop* setting but degrades sharply (~ 35) in the *Normal* real-time setting. This indicates that inference delays desynchronize the agent’s planning from the evolving game state, nullifying their cognitive advantage. Conversely, reactive models, GPT-4o and Qwen3-VL-8B, exhibit stability across settings (57, 55, 49 in PvZ), suggesting their performance is bounded by reasoning capacity rather than response speed. Consequently, reducing latency is critical for deploying reasoning models in real-time environments, whereas it offers limited gains for reactive architectures.

Due to space limitations, Appendix D presents a comprehensive analysis of game prompt, per-game performance, detailed error attribution, and reflection improvements.

4.3. Unpacking Video-based Reflection

On the Roles of Failures and Tutorials. To understand how models learn, we decompose the reflection process in Table 5. It shows that the integration of failure and tutorials (*Self*) consistently outperforms relying on either source by at

Table 5. Performance comparison of 2 VLMs on 3 games in semantic mode. The VR settings are categorized into: **No**, **Self-F**, **Self-T**, **Self**, and **Other** (the alternative VLM).

Model	VR	TicTacToe	AngryBird	Plants vs. Zombies	Avg
GPT-4o-mini <i>weaker model</i>	No	29.2±15.1	54.6±6.2	53.2±25.6	45.6
	Self-F	43.1±4.3	57.4±13.4	52.1±18.0	50.8
	Self-T	49.2±13.7	73.2±8.2	63.3±13.1	61.9
	Self	52.3±16.0	75.8±10.6	68.6±14.6	65.5
	Other	45.0±7.2	77.1±7.0	55.4±7.2	59.1
Qwen3-VL-32B <i>stronger model</i>	No	67.0±15.3	48.5±11.3	67.6±26.0	61.0
	Self-F	84.5±16.8	54.4±12.9	77.3±13.2	72.0
	Self-T	67.1±15.6	48.3±12.1	73.5±17.3	62.9
	Self	84.5±13.2	66.8±15.8	78.8±9.1	76.7
	Other	69.2±16.4	74.5±10.2	74.3±1.6	72.6

least 3.6% for GPT-4o-mini and 4.7% for Qwen3-VL-32B. Conceptually, this mirrors the paradigm of foundation model post-training (Chu et al., 2025). Reflecting on failure (*Self-F*) acts as a negative constraint mechanism, analogous to **Reinforcement Learning (RL)**, better with strong Qwen3-VL-32B +11.0%. By identifying erroneous branches, the agent learns "what not to do," effectively refining its policy space. While learning from tutorials (*Self-T*) provides positive behaviors, parallel to **Supervised Fine-Tuning (SFT)**, better with weak GPT-4o-mini +16.4%. This supplies the agent with optimal trajectories that are difficult to discover through sparse-reward exploration alone. The integration of two mechanisms yields the most robust improvement, validating that a *reflect-and-retry* loop can serve as a training-free proxy for combining SFT and RL in post-training.

On the Roles of Reflection Models. Regarding the transferability of reflection, we employ a stronger Qwen3-VL-32B and a weaker GPT-4o-mini to exchange reflections. We observe that reflections generated by Qwen3-VL-32B do not enhance GPT-4o-mini (*Other* 59.1 < *Self* 65.5). This suggests that the primary bottleneck is internalization rather than extraction; while extracting insights from video is relatively accessible, the capability to ground these insights into precise actions represents a higher intelligence barrier that cannot be bridged solely by better reflections.

4.4. Qualitative Evaluation of Video-based Reflection

To vividly illustrate how reflection alters agent behavior, we visualize the trajectories before and after the VR protocol.

Case Study 1: Strategic Shift in 2048. In 2048, the agent initially often made greedy moves. After reflection, it synthesized high-level strategies. In the reflection experiences, the agent explicitly formulated rules: (1) "*Anchor the Largest Tile in a Corner*" to prevent board clutter; (2) "*Strictly Limit Swipes to Three Directions*" to lock the anchor row; (3) "*Build a Descending Snake Chain*" to facilitate merging. This strategic shift resulted in a score increase from 1920 to 5960, shown in Figure 6a and 6b.

Case Study 2: Physics Adaptation in Angry Birds. For

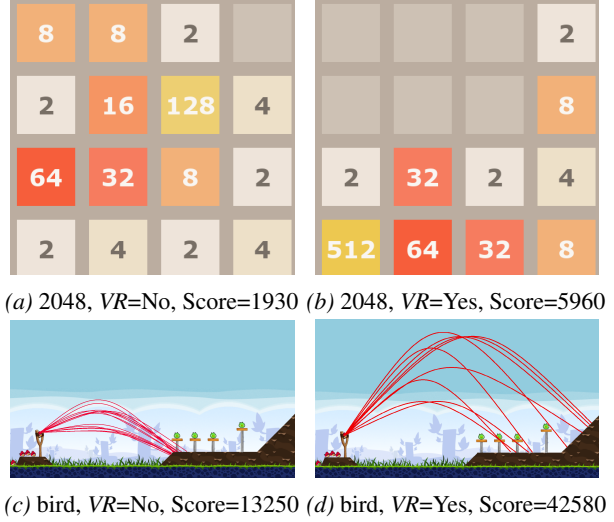


Figure 6. Screenshots of Gemini-2.5-Pro playing 2048 and trajectories of Qwen3-VL-8B playing angry birds level 2.

physics-based puzzles, the VR protocol helped correct causal misunderstandings. In Level 2, the agent initially attempted to hit the top plank directly. After reflection, it refined its plan: "*Aim for the central support pillar... Shoot the bird at the right-hand platform... ensure the falling blocks roll down the slope for a second wave of splash damage.*" This physics adaptation led to a score improvement from 13,250 to 42,580, shown in Figure 6c and 6d.

4.5. Additional Insights of Video-based Reflection

Unlike humans who derive stable and substantial gains, model improvements are sometimes inconsistent and constrained. Why does reflection work, and where does it fail? We identify one core advantage countered by two barriers:

Advantage: Dual-Stream Capability Acquisition. The improvements stem from two distinct cognitive gains extracted from failure and tutorials: (1) **Abstract Principle Comprehension**, where the agent distills high-level logic (e.g., "Build a snake chain" in 2048) to enhance global planning; and (2) **Specific Behavioral Mimicry**, where the agent learns concrete, context-specific maneuvers (e.g., "Aim at the central pillar" in Angry Birds). This indicates that current VLMs can effectively parse multimodal demonstrations into both strategic and tactical knowledge.

Barrier I: The Cognitive Bottleneck. We identify a *Cognitive Bottleneck* that fundamentally restricts the agent performance. Embodied environments demand robust spatial perception and long-horizon reasoning. We observe that even when agents explicitly retrieve the correct insight, they often fail to *internalize* it into the next attempt. This is due to inherent limitations in reasoning and planning capabilities. Unlike humans who seamlessly integrate new lessons into

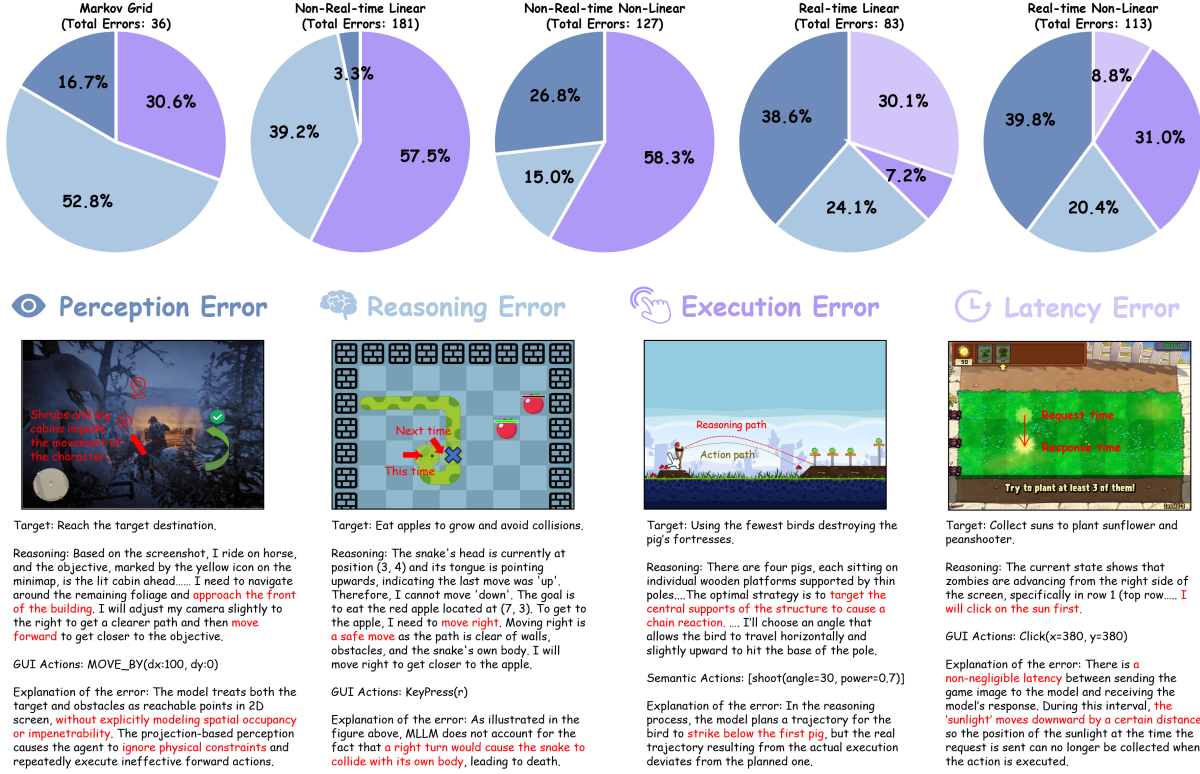


Figure 7. **Top:** GameVerse results breakdown of Qwen3-VL-32B. **Bot:** Failure cases about 4 types of errors in GameVerse.

their mental models, models struggle to "compile" textual reflections into a robust, updated policy, leading to repeated strategic errors despite correct theoretical knowledge.

Barrier II: The Physical Disconnect. Complementary to the cognitive constraint, we identify a *Physical Disconnect* that hinders the agent's interaction with the environment. Embodied environments demand real time reaction and precise control. We observed that even when the strategy is sound, the agent is hindered by inference latency and knowing-doing gap. The high latency desynchronizes the agent's reasoning from the rapidly evolving world, while the knowing-doing gap severs the link between high-level semantic planning and precise low-level execution. Consequently, the agent remains too slow and clumsy, preventing the success of reasoning models in *GameVerse*.

4.6. Error Analysis

To diagnose the capability boundaries of VLMs, we classify gameplay failures into four error types based on the agent's capability: **Perception Error:** Fail to interpret visual information correctly. **Reasoning Error:** Fail in logical deduction or future state prediction, where the agent misinterprets the causal consequences of an action. **Execution Error:** There is a misalignment between a correct high-level semantic plan and its low-level motor implementation. **La-**

tency Error: This is from the temporal desynchronization between observation and action due to inference time.

We give the breakdown of Qwen3-VL-32B and typical failure cases in Figure 7. The total error count escalates as tasks shift from static grids to dynamic, non-linear worlds. As visual fidelity increases from abstract grids to high-fidelity 3D environments, the proportion of *perception errors* rises sharply (16.7% → 39.8%), indicating that processing complex visual semantics remains a bottleneck. Regardless of the category, *reasoning and execution errors* consistently maintain a substantial proportion (> 15%), suggesting that the reasoning barrier and physical disconnect is a systemic issue. In Real-time settings, *latency errors* emerge as a critical failure mode, confirming that for dynamic agents, inference speed is as vital as cognitive accuracy.

5. Discussion

Conclusion. In this work, we introduced **GameVerse**, benchmarking VLM agents through *reflect-and-retry* loop. Through a cognitive hierarchical taxonomy across 15 games, we revealed that while current VLMs show promise in simple environments, they struggle with generalization in challenging settings. Reflection yields improvements, but models still lack the human-like robustness to consistently inter-

nalize experience. Moreover, the integration of failures and tutorials combines negative error-pruning with imitating behaviors, mirroring the complementary benefits of combining reinforcement and supervised learning in post-training (Chu et al., 2025). Beyond a benchmark, *GameVerse* serves as an arena where agents mimic human reflection and generalize across virtual worlds toward general intelligence.

Limitations and Future Work. Currently, our agent passively receives failures and tutorials in a single turn. Future systems could employ active, multi-turn interactions, allowing agents to dynamically query specific video segments or diagnose failures through iterative dialogue. While our scalable milestone evaluation pipeline reduces the labor intensity in previous work (Tan et al., 2025b; Zhang et al., 2025a; Ahn et al., 2025), it relies on a single VLM. Future iterations could enhance robustness and accuracy by incorporating multi-model voting or hybrid visual-text retrieval.

Impact Statement

This paper presents work whose goal is to advance the field of Vision-Language Models. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Ahn, J., Kim, J., Yun, H., Son, J., Park, D., Cho, J., and Kim, G. Flashadventure: A benchmark for gui agents solving full story arcs in diverse adventure games. In *EMNLP*, 2025.
- Antol, S., Agrawal, A., Lu, J., Mitchell, M., Batra, D., Zitnick, C. L., and Parikh, D. Vqa: Visual question answering. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, December 2015.
- Bai, S., Cai, Y., Chen, R., and et al., K. C. Qwen3-vl technical report. *arXiv:2511.21631*, 2025.
- Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, June 2013. ISSN 1076-9757.
- Bytedance Seed Team. Seed1.8 model card: Towards generalized real-world agency. https://seed.bytedance.com/zh/seed1_8, dec 2025. Accessed: 2026-01-10.
- Cai, S., Zhang, B., Wang, Z., Ma, X., Liu, A., and Liang, Y. GROOT: Learning to follow instructions by watching gameplay videos. In *The Twelfth International Conference on Learning Representations*, 2024.
- CAPCOM. Phoenix wright: Ace attorney trilogy, 2019. URL https://store.steampowered.com/app/787480/Phoenix_Wright_Ace_Attorney_TriLOGY/. Accessed: 2026-01-05.
- Chen, P., Bu, P., Song, J., Gao, Y., and Zheng, B. Can vlms play action role-playing games? take black myth wukong as a study case. *arXiv:2409.12889*, 2024.
- Chen, P., Bu, P., Wang, Y., and Wang, X. e. a. Combatvla: An efficient vision-language-action model for combat tasks in 3d action role-playing games. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 10919–10928, October 2025.
- Cheng, K., YanTao, L., Xu, F., Zhang, J., Zhou, H., and Liu, Y. Vision-language models can self-improve reasoning via reflection. In *ACL*, pp. 8876–8892, 2025.
- Chu, T., Zhai, Y., Yang, J., Tong, S., Xie, S., Schuurmans, D., Le, Q. V., Levine, S., and Ma, Y. SFT memorizes, RL generalizes: A comparative study of foundation model post-training. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 10818–10838. PMLR, 2025.
- Comanici, G., Bieber, E., Schaeckermann, M., Pasupat, I., and et al., N. S. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv:2507.06261*, 2025.
- Dubisayr Polo Club. Mini metro, 2015. URL https://store.steampowered.com/app/287980/Mini_Metro/. Accessed: 2026-01-05.
- EQ Studios. Scene investigators, 2023. URL https://store.steampowered.com/app/1159830/Scene_Investigators/. Accessed: 2026-01-05.
- Fan, L., Wang, G., Jiang, Y., Mandlekar, A., Yang, Y., and etc. Minedojo: Building open-ended embodied agents with internet-scale knowledge. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2022.
- Firaxis Games. Sid meier’s civilization@ vi, 2016. URL https://store.steampowered.com/app/289070/Sid_Meiers_Civilization_VI/. Accessed: 2026-01-05.
- Hempuli Oy. Baba is you, 2019. URL https://store.steampowered.com/app/736260/Baba_Is_You/. Accessed: 2026-01-05.
- Hu, L., Huo, M., Zhang, Y., Yu, H., Xing, E. P., Stoica, I., Rosing, T., Jin, H., and Zhang, H. Imgame-bench:

- How good are llms at playing games? *arXiv preprint arXiv:2505.15146*, 2025a.
- Hu, L., Li, Q., Xie, A., Jiang, N., Stoica, I., Jin, H., and Zhang, H. Gamearena: Evaluating LLM reasoning through live computer games. In *The Thirteenth International Conference on Learning Representations*, 2025b.
- Joel, Z. Gemini plays pokemon twitch stream, 2025. URL https://www.twitch.tv/gemini_plays_pokemon. Accessed: 2025-12-09.
- Juul, J. *Half-Real: Video Games between Real Rules and Fictional Worlds*. MIT Press, 2011.
- Li, K., Ziyang, M., Lin, H., Luo, Z., and etc. Screenspot-pro: GUI grounding for professional high-resolution computer use. In *Workshop on Reasoning and Planning for Large Language Models*, 2025a.
- Li, M., Wang, Z., He, K., Ma, X., and Liang, Y. JARVIS-VLA: Post-training large-scale vision language models to play visual games with keyboards and mouse. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 17878–17899, 2025b.
- Li, Y., Guo, Y., Guerin, F., and Lin, C. An open-source data contamination report for large language models. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 528–541, 2024.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., and etc. Self-refine: Iterative refinement with self-feedback. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 46534–46594, 2023.
- Magne, L., Awadalla, A., Wang, G., Xu, Y., and etc. Nitrogen: An open foundation model for generalist gaming agents. *arXiv:2601.02427*, 2026.
- Mega Crit. Slay the spire, 2019. URL https://store.steampowered.com/app/646570/Slay_the_Spire/. Accessed: 2026-01-05.
- Mihoyo. Genshin impact, 2020. URL <https://www.yuanshen.com/>. Accessed: 2026-01-05.
- OpenAI, :, Hurst, A., Lerer, A., Goucher, A. P., and et al., A. P. Gpt-4o system card. *arXiv:2410.21276*, 2024.
- Paglieri, D., Cupiał, B., Coward, S., and et al., U. P. BAL-ROG: Benchmarking agentic LLM and VLM reasoning on games. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Park, D., Kim, M., Choi, B., Kim, J., and etc. Orak: A foundational benchmark for training and evaluating llm agents on diverse video games. *arXiv:2506.03610*, 2025.
- Playground Games. Forza horizon vi, 2021. URL https://store.steampowered.com/app/1551360/_5/. Accessed: 2026-01-05.
- PopCap Games, Inc. Plants vs. zombies, 2009. URL https://store.steampowered.com/app/3590/Plants_vs_Zombies_GOTY_Edition/. Accessed: 2026-01-05.
- Qin, Y., Ye, Y., Fang, J., and et al., H. W. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv:2501.12326*, 2025.
- Rockstar Games. Red dead redemption 2, 2019. URL https://store.steampowered.com/app/1174180/Red_Dead_Redemption_2/. Accessed: 2026-01-05.
- Rovio. Angry birds, 2009. URL <https://www.angrybirds.com/>. Accessed: 2026-01-05.
- Russell, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: language agents with verbal reinforcement learning. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 8634–8652, 2023.
- Tan, W., Li, X., Fang, Y., Yao, H., and et al., S. Y. Lumine: An open recipe for building generalist agents in 3d open worlds. *arXiv:2511.08892*, 2025a.
- Tan, W., Zhang, W., Xu, X., Xia, H., Ding, Z., and etc. Cradle: Empowering foundation agents towards general computer control. In *Forty-second International Conference on Machine Learning*, 2025b.
- Towers, M., Kwiatkowski, A., Terry, J., and et al., J. U. B. Gymnasium: A standard interface for reinforcement learning environments. *arXiv:2407.17032*, 2024.
- Turing, A. M. Digital computers applied to games. In *Faster Than Thought*. Pitman, London, 1953.
- Wang, G., Xie, Y., Jiang, Y., Mandlkar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Wang, H., Zou, H., Song, H., and et al., J. F. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv:2509.02544*, 2025a.

- Wang, X., Zhuang, B., and Wu, Q. Are large vision language models good game players? In *The Thirteenth International Conference on Learning Representations*, 2025b.
- Wang, Z., Li, X., Ye, Y., Fang, J., and et al., H. W. Gametars: Pretrained foundation models for scalable generalist multimodal game agents. *arXiv:2510.23691*, 2025c.
- Wen, Y., Ye, Y., Zhang, Y., Yang, D., and Zhu, H. Real-time reasoning agents in evolving environments. *arXiv:2511.04898*, 2025.
- Xie, T., Zhang, D., Chen, J., Li, X., and etc. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv:2404.07972*, 2024.
- Xu, X., Li, Z., Lu, C., Qi, R., and etc. Reflection of episodes: Learning to play game from expert and self experiences. *arXiv:2502.13388*, 2025.
- Ying, L., Truong, R., Sharma, P., Zhao, K. I., Cloos, N., Allen, K. R., Griffiths, T. L., Collins, K. M., Hernández-Orallo, J., Isola, P., Gershman, S. J., and Tenenbaum, J. B. Ai gamestore: Scalable, open-ended evaluation of machine general intelligence with human games. *arXiv:2602.17594*, 2026.
- Zhang, A. L., Griffiths, T. L., Narasimhan, K. R., and Press, O. Videogamebench: Can vision-language models complete popular video games? *arXiv:2505.18134*, 2025a.
- Zhang, K., Chai, C., Xu, J., Zhang, C., Han, H., Yuan, Y., Wang, G., and Cao, L. Handling label noise via instance-level difficulty modeling and dynamic optimization. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b.
- Zheng, X., Li, L., Yang, Z., Yu, P., Wang, A. J., Yan, R., Yao, Y., and Wang, L. V-mage: A game evaluation framework for assessing vision-centric capabilities in multimodal large language models. *arXiv:2504.06148*, 2025.
- Zimmerman, E. and Salen, K. *Rules of Play: Game Design Fundamentals*. MIT Press, 2003.

GameVerse: Can Vision-Language Models Learn from Video-based Reflection? (Supplementary materials)

This appendix provides supplementary materials to support the findings presented in the main paper, ensuring providing detailed information and offering deeper insights into the GameVerse.

First of all, for the readers’ better understanding, we describe the main content of the appendix.

Appendix A: Implementation Details. This section detailed the architecture of zeroshot and memory agent, the pipeline of milestone detection and scoring and the universal GUI action space.

Appendix B: Human Baselines. This section detailed the recruitment methodology, and testing protocols used to establish the Human Rookie and Human Expert baselines.

Appendix C: Game Selection Criteria. This section details the rigorous hierarchical taxonomy, ensuring a diverse coverage of cognitive demands and difficulty tiers.

Appendix D: Game Environment Details and Extended Analysis. This section provides detailed descriptions of the task objectives and mechanics for each of the 15 games. Furthermore, it presents an extended qualitative analysis of the Video-Based Reflection efficacy and a comprehensive case study of agent failures.

Contents

A	Implementation Details	13
A.1	Agent Architecture	13
A.2	GUI Action Space	14
A.3	Video-based Reflection Pipeline	14
A.4	Milestone Scoring Pipeline	15
A.5	Latency Setting	16
B	Human Baselines	18
C	Game Selection Criteria	19
C.1	GameVerse Surpasses Traditional Static Benchmarks	19
C.2	Detailed Game Selection Criteria	19
C.3	Detailed Game Introduction	20
D	Game Environment Details and Extended Analysis.	21
D.1	Tic-Tac-Toe	21
D.2	Baba Is You	24
D.3	2048	28
D.4	Maze	31
D.5	Angry Birds	34
D.6	Slay the Spire	36
D.7	Ace Attorney	40

D.8 Civilization VI	41
D.9 Scene Investigators	47
D.10 Snake	52
D.11 Plants vs. Zombies	56
D.12 Forza Horizon 5	60
D.13 Mini Metro	63
D.14 Genshin Impact	66
D.15 Red Dead Redemption 2	70

A. Implementation Details

A.1. Agent Architecture

In GameVerse, we initially use two kinds of agents, zeroshot agent and memory agent. For *Tic-Tac-Toe*, *Baba Is You*, *2048*, *Maze*, *Angry Birds*, *Slay the Spire*, *Snake*, *Plant vs. Zombie*, *Forza Horizon 5* and *mini metro*, we use zeroshot agent because the current visual input includes most of the information and the goal of the game remains the same. For *Ace Attorney*, *Civilization VI*, *Scene Investigators Demo*, *Genshin Impact* and *Red Dead Redemption 2*, we use memory agent because these games include long story arcs and the memory of key information is needed. We conduct the comparison experiments of zero-shot and memory agent for these games in Appendix D.

For zeroshot agent, only the current screenshot and the screenshot-action pair from the last one step are provided as context for VLM agents. For memory agent, the screenshot-action pair from the last five step are provided as context for VLM agents to update a short-term memory. The agent can choose to store the key information as long-term memory into vector database and retrieve the long-term memory based on relevance to the current game state.

Zero-shot Agent Details. Zeroshot agent is designed to handle the situation where the screenshot covers most information of the current game state, and the game goal is obvious and stable. The agent operates through only one module:

1. *Action Inference Module.* This module performs direct action inference from the current game state without maintaining any form of historical context.

Memory Agent Details. Memory agent is designed to enable long-term memory capabilities for game-playing agents, addressing the challenge of maintaining contextual awareness across extended gameplay sessions. The agent operates through a sequential pipeline of four interconnected modules:

1. *History Review and Reasoning Module.* This module processes short-term history by reviewing the most recent 5 image-action pairs alongside the previous history summary. It employs an importance-based replacement strategy with four priority levels: CRITICAL (confirmed contradictions, successful strategies), HIGH (potential contradictions, important testimonies), MEDIUM (dialogue progression, routine actions), and LOW (outdated information, minor observations). The module maintains a fixed-length summary (500-600 words) through selective compression and merging of similar information, ensuring that the most valuable context is preserved while staying within token limits.

2. *Long-term Memory Retrieval Module.* This module implements semantic retrieval from a persistent vector database. We use ChromaDB as the vector store, supporting both OpenAI and Qwen embedding models for flexibility across different deployment scenarios. The retrieval query is constructed from three components: (1) the current observation state, (2) the compressed history summary, and (3) the reasoning from the previous module. Retrieved memories are filtered using a similarity threshold (default 0.4) to ensure relevance, with top-k (default k=3) most similar memories returned.

3. *Action Inference with Memory Module.* This module performs action selection by integrating multiple information sources: the current game screenshot, the compressed history summary, reasoning context, and retrieved long-term memories. The module also implements an optional memory storage mechanism where the agent decides whether to save new information to long-term memory based on predefined importance criteria (e.g., strategic decisions, puzzle solutions, important character

information). A deduplication mechanism prevents storing highly similar memories using a similarity threshold (default 0.8).

4. *Short-term History Update Module*. This module maintains the temporal consistency of the agent’s memory by recording the current step’s state, image, action, and reasoning. It ensures that image-action pairs are properly indexed for subsequent history review iterations, implementing a sliding window approach over the agent’s trajectory.

GameVerse is designed with a modular architecture that enables easy integration of different components and facilitates ablation studies. Different modules such as self-reflection, subtask planning and etc., can be easily added for future studies.

A.2. GUI Action Space

Following the GUI action settings in UI-TARS (Qin et al., 2025) and FlashAdventure (Ahn et al., 2025), we design our GUI action space as shown in Table 6.

Table 6. GUI Action Types Summary

Action Type	Description	Parameters
Mouse Movement		
MOVE_TO	Move to position	x, y (int, req): Target coordinates (window relative)
MOVE_BY	Move relatively	dx, dy (int, req): Offset; duration (float, opt): Seconds
Mouse Click		
CLICK	Click mouse	x, y (opt); button (left/right/middle); num_clicks
RIGHT_CLICK	Right click	x, y (int, opt): Click coordinates
DOUBLE_CLICK	Double click	x, y (int, opt): Click coordinates
Mouse Drag		
MOUSE_DOWN	Press button	button (str, opt): "left", "right", or "middle"; duration (float, opt)
MOUSE_UP	Release button	button (str, opt): "left", "right", or "middle";
DRAG_TO	Drag to target	x, y (int, req): Target coordinates
Mouse Scroll		
SCROLL	Scroll wheel	dx, dy (int, req): Scroll amount (+/-); duration (float, opt)
Keyboard Input		
TYPING	Type text	text (str, req); interval (float, opt)
PRESS	Press key	key (str, req); duration (float, opt)
KEY_DOWN	Key down	key (str, req): Key name; duration (float, opt)
KEY_UP	Key up	key (str, req): Key name;
HOTKEY	Key combo	keys (list, req): e.g., ['ctrl', 'c']; duration (float, opt)
Control Flow		
WAIT	Wait time	duration (float, req): Seconds to wait
DONE	Task success	No parameters
FAIL	Task failure	No parameters

A.3. Video-based Reflection Pipeline

Given the limited context window of current VLMs and the complexity of processing multiple videos concurrently, we adopt a sequential analysis strategy. We first query the model to diagnose the failure video—identifying the specific scenario and root cause. This diagnostic context then guides the analysis of expert demonstrations, facilitating the extraction of core insights that are injected into the subsequent prompt. The prompt for extracting experience of failure and tutorials is in Figure 8. These extracted experience will be injected into the next attempt, taking place of `<|learned_experience>` in the

system prompt.

For Qwen3-VL-8B/32B, Gemini-2.5-Pro/flash and seed-1.8, whose API facilitates direct video ingestion, GPT-4o and GPT-4o-mini accept only image-based inputs. To address this, we implement uniform sampling at 1 fps. When the extracted frames exceed the context window due to video length, we iteratively downsample the sequence until it accommodates the model’s context limits.

Reflect Failure Prompt	Reflect Tutorial Prompt
System Prompt You are a game strategy analysis expert. Your task is to analyze failure scenarios and reasons from failed game videos. Requirements: 1. Carefully observe the game process in the video 2. Identify key errors and issues that led to failure 3. Summarize failure scenarios and reasons, be specific and clear 4. Output format: Plain text, list failure scenarios and reasons in bullet points User Prompt <failure video> or <failure frames> Game name: <game name> Please analyze this failure video and summarize the failure scenarios and reasons.	System Prompt You are a game strategy analysis expert. You have analyzed the failure video and understood the failure scenarios and reasons. Now please watch the expert video, learn successful strategies, and summarize experiences that can help avoid the previous failures. Requirements: 1. Combine the previously analyzed failure scenarios and reasons 2. Observe successful strategies in the expert video 3. Summarize experiences that can avoid failures and improve game performance (no more than <max length> words or tokens) 4. Experiences should be specific and actionable, avoid vague suggestions 5. Sort by importance, put the most important experiences first 6. **IMPORTANT: Keep your response within <max length> words or tokens. Be concise and focus on the most critical insights.** Output format: Plain text, one experience per line, described in concise language. User Prompt <tutorial video> or <tutorial frames> Game name: <game name> ### Previously analyzed failure scenarios and reasons: <failure analysis> ### Expert video: Please watch the following expert video, learn successful strategies, and summarize experiences that can help avoid the previous failures.

Figure 8. Prompt of reflecting experience from failure and tutorial

We provide links in Table 7 to all walkthroughs used for reflection and milestone extraction, as well as estimating game lengths for human expert. Each of these walkthroughs assumes full knowledge of the game and does not consider time spent exploring. The videos were not subjectively selected; rather, they were identified as the top-ranked search results on mainstream video platforms corresponding to the target game tasks. The similar videos are abundant online, ensuring our scalability to more video games.

Table 7. List of longplay walkthrough links for games in GameVerse.

Game	Video Type	Link	Selected Time
Tic-Tac-Toe	Skill Tutorial	https://www.youtube.com/shorts/ZBZcnImNmhk	00:00:59
Baba Is You	Walkthrough	https://www.bilibili.com/video/BV1wb41lp7ja/	00:00:38
2048	Skill Tutorial	https://www.bilibili.com/video/BV1VH4y1K7Gr/	00:03:02
Maze	Skill Tutorial	https://www.bilibili.com/video/BV1pp4y197fY/	00:03:37
Angry Birds	Walkthrough	https://www.bilibili.com/video/BV1b54y1s7pe/	00:01:31
Slay the Spire	Skill Tutorial	https://www.bilibili.com/video/BV1RVdLYyEFf/	00:09:36
Ace Attorney	Walkthrough	https://www.bilibili.com/video/BV1oUpEeaEyU/	00:21:50
Civilization VI	Walkthrough	https://www.bilibili.com/video/BV1Ax41197bu/	00:19:23
Scene Investigators	Walkthrough	https://www.bilibili.com/video/BV1NA411U7qj/	00:27:26
Snake	Walkthrough	https://www.youtube.com/shorts/CHhnAhoO_ak	00:00:44
Plants vs. Zombies	Walkthrough	https://www.bilibili.com/video/BV12b41167b7/	00:05:51
Forza Horizon 5	Skill Tutorial	https://www.bilibili.com/video/BV1Pi4we3EdF/	00:04:58
Mini Metro	Skill Tutorial	https://www.bilibili.com/video/BV1cY4y1d7vS/	00:04:41
Genshin Impact	Walkthrough	https://www.bilibili.com/video/BV1H64y1e7YJ/	00:25:51
Red Dead Redemption 2	Walkthrough	https://www.bilibili.com/video/BV1hb411N7Rk/	00:44:07

A.4. Milestone Scoring Pipeline

There are 7 games lacking intrinsic scoring system in *GameVerse*, *Baba is you*, *Ace Attorney*, *Civilization VI*, *Scene Investigator*, *Plants vs. Zombies*, *Genshin Impact*, *Red Dead Redemption 2*. Facing the absence of intrinsic scoring systems in long-horizon games (e.g. *Genshin Impact*, *RDR 2*, previous methods necessitated either brittle hash-matching (Zhang

et al., 2025a) or labor-intensive manual milestone annotation (Chen et al., 2024; Tan et al., 2025b; Zhang et al., 2025a; Ahn et al., 2025). We introduce an milestone scoring pipeline utilizing advanced VLMs (Gemini-3-pro used in this paper), which offer extensive knowledge (knowing these famous games), long-context video comprehension (understanding the gameplay video), and web-search capabilities. This system extracts standardized milestone JSON profiles M_{ref} from expert walkthrough videos and employs the model’s reasoning capabilities to align these milestones with agent playthrough videos, thereby generating procedural scores without human intervention. The milestone detection and matching prompts are shown in Figure 9

A.4.1 MILESTONE DETECTION PIPELINE

We take expert walkthrough videos as input to extract representative milestones, guided by specific prompts. The videos of these games are the walkthrough videos mentioned in Table 7. An example of extracting milestone is shown Figure 10. We provide a detailed breakdown of the milestones extracted from the seven games in Table 8, covering metrics such as count, hallucination rate, and representativeness as evaluated by 10 human experts. Experimental results demonstrate high expert consensus regarding the validity of the extracted milestones, confirming them as significant, stage-defining nodes within the gameplay. Furthermore, the hallucination rate remains within a controllable range. We check the detailed hallucination and find that inaccuracies are primarily confined to fine-grained details—such as entity descriptions and specific perceptions—while macro-level scene depictions remain accurate. Consequently, the proposed milestone extraction pipeline exhibits fundamental robustness, scalability, and reusability.

Table 8. The breakdown of the extracted milestones. L means level. Hallucination rate is defined as $\frac{\#inaccurate\ milestones}{\#total\ milestones}$. Representation score is defined as how representative the milestones are to significant game nodes, ranging in $[0, 1]$. The score is evaluated by 10 human video game experts

Game	Video Length	# Milestone	Hallucination Rate	Representation Score
Baba Is You	00:00:38	3 ($L0$), 5 ($L1$)	12.5%	0.92
Ace Attorney	00:21:50	9	0%	1.00
Civilization VI	00:19:23	12	17%	0.81
Scene Investigators	00:27:26	17	11%	0.95
Plants vs. Zombies	00:05:51	7 ($L1$), 6($L2$), 6($L4$)	5%	0.78
Genshin Impact	00:25:51	7	0%	0.95
Red Dead Redemption 2	00:44:07	10	10%	0.87

A.4.2 MILESTONE MATCHING PIPELINE

Following extracting the offline milestone JSON files, we establish a reusable framework for evaluating the agent’s gameplay process score via video matching. Specifically, the ‘gameplay video’ consists of the *discrete frame* sequences received by the agent at each step. We adopt this discrete approach because recording the continuous gameplay process would yield excessively long videos that exceed context windows, given the inference latency which ranges from seconds (GPT-4o, GPT-4o-mini) to minutes (Gemini-2.5-Pro, Seed-1.8). Finally, we employ Gemini-3-Pro, guided by specific prompts, to compute matching scores between the milestones and the agent’s gameplay. The results will include whether the milestone is completed and the evidence to prove for further human verified. In Table 9, we present the mean VLM-evaluated and human-evaluated matching scores on agent play video and human play video with percentage difference across seven games. The comparison distinguishes between human gameplay videos (continuous frame sequences) and agent gameplay frames (discrete input frames). The results demonstrate that milestone matching via advanced VLMs exhibits robustness and stability. However, continuous video matching yields higher accuracy compared to discrete environmental frames. We observe that matching scores for agent videos tend to be lower; this is attributed to the discrete sampling nature, where specific milestones occurring in the temporal gaps between frames may fail to be visually captured. Consequently, enhancing the continuity of agent gameplay videos without exceeding context window limits remains a meaningful research problem.

A.5. Latency Setting

We conducted latency ablation studies on two representative real-time games: Plants vs. Zombies and Snake. Given the inherent variability and uncontrollability of API inference and network transmission latencies, we instead modulated the environmental latency (i.e., game simulation speed) to systematically assess its impact on agent performance. Specifically, we established three latency levels, **Stop**, **Slow**, and **Normal**.

Milestone Detection Prompt

System Prompt

You are a game milestone identifier. Please carefully watch this {game_name} playthrough video (including frame sequences and possibly game audio and commentary audio), identify and list in chronological order all milestone events where player actions are crucial to game progress. Please base your analysis on the video content itself.

Requirements:

1. Milestones should be meaningful game progress nodes, including: first time completing an operation, making important decisions, completing levels, reaching save points, obtaining important items, unlocking important abilities, meeting new characters, defeating important enemies, entering new major areas, completing key steps.
2. Each milestone should include: timestamp range (min.sec - min.sec), milestone name, detailed description, category, importance
3. Arrange in chronological order
4. Descriptions should be specific and detailed, including content summary and specific context
5. Ensure each milestone represents a single, distinct event.

Please return in JSON format as follows:

```
{
  "milestones": [
    {
      "timestamp": "123.45 - 127.46",
      "title": "Complete Level 1",
      "description": "Complete Level 1: After defeating the boss, the player obtains a new weapon, character health is full, and unlocks new skills",
      "category": "Level Completion",
      "importance": "High"
    },
    {
      "timestamp": "234.56 - 241.57",
      "title": "Defeat First Boss",
      "description": "Defeat Boss: Defeat the first Boss, gain experience points and gold rewards",
      "category": "Boss Defeat",
      "importance": "High"
    },
    {
      "timestamp": "345.67 - 412.57",
      "title": "Obtain Rare Item X",
      "description": "Obtain Rare Item: Discover hidden treasure chest and obtain rare item",
      "category": "Item Acquisition",
      "importance": "Medium"
    }
  ]
}
```

Please ensure the JSON format is correct and timestamps are accurate to seconds.

User Prompt

<walkthrough video> or <walkthrough frames>

Game name: {game_name}

Please carefully watch this walkthrough video and identify and list all milestone events in chronological order.

Milestone Matching Prompt

System Prompt

You are a game milestone checker. Your task is to watch a gameplay video and check whether each milestone from the reference milestones list has been completed in this video.

Requirements: 1. Carefully observe the gameplay video

2. For each milestone, determine if it has been completed in the video by comparing the milestone's TITLE and DESCRIPTION with what you observe in the video

3. ****IMPORTANT:** Focus primarily on matching the milestone's TITLE and DESCRIPTION. Do NOT require the timestamp to match. The milestone may occur at any time in the video, as long as the event described in the title and description is observed.**

4. Provide clear evidence for your judgment (what you see in the video that indicates completion or non-completion)

5. Be accurate and objective in your assessment

Return your results in JSON format as follows:

```
{
  "milestone_matches": [
    {
      "milestone_index": 0,
      "completed": true,
      "evidence": "I can see in the video that the player established the first city 'Ra-Kedet', which matches the milestone title 'Found First City' and description about establishing the capital city."
    },
    {
      "milestone_index": 1,
      "completed": false,
      "evidence": "I did not observe the player selecting 'Mining' technology in the technology tree during the video, which is required by the milestone title 'Select First Technology' and description."
    }
  ]
}
```

Important:

- Return results for ALL milestones in the same order as provided
- milestone_index should match the index in the milestones list (starting from 0)
- ****Focus on matching the TITLE and DESCRIPTION content, NOT the timestamp****
- The milestone can occur at ANY time in the video - timestamp matching is NOT required
- Be specific about what you see or don't see in the video that relates to the milestone's title and description
- If a milestone is partially completed or unclear, mark it as false and explain why""

User Prompt

{tutorial video} or {tutorial frames}

Game name: {game_name}

{milestones_text}

Please watch the gameplay video and check which milestones have been completed.

Figure 9. Prompt of milestone extraction and milestone matching

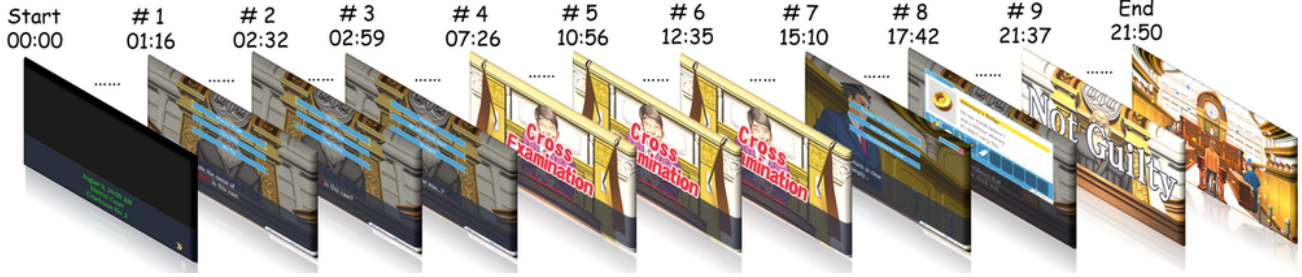


Figure 10. 9 milestones are extracted by Gemini-3-pro in the walkthrough video of Ace Attorney.

Table 9. The average VLM-evaluated and human-evaluated matching scores on agent play video and human play video with percentage difference across seven games with VL.

Game	Agent gameplay video (discrete input frames)			Human gameplay video (continuous frame sequences)		
	VLM eval.	human eval.	Percentage Diff. ↓	VLM eval.	human eval.	Percentage Diff. ↓
Baba Is You	0.68±0.15	0.74±0.10	8.1%	0.98±0.06	1.00±0.00	2.0%
Ace Attorney	0.29±0.17	0.33±0.19	12.1%	1.00±0.00	1.00±0.00	0%
Civilization VI	0.04±0.08	0.04±0.08	0.0%	0.92±0.11	0.96±0.09	4.2%
Scene Investigators	0.05±0.04	0.05±0.04	0.0%	0.90±0.09	0.90±0.09	0%
Plants vs. Zombies	0.34±0.15	0.38±0.13	10.5%	1.00±0.00	0.98±0.04	2.0%
Genshin Impact	0.12±0.05	0.14±0.00	14.2%	0.83±0.15	0.88±0.18	5.6%
Red Dead Redemption 2	0.10±0.02	0.10±0.02	0.0%	0.86±0.15	0.83±0.21	3.4%

In *Plants vs. Zombies*, the **Normal** setting follows standard game dynamics without any artificial delays. The **Slow** setting introduces a 5-second pause after input ingestion to buffer the model’s inference latency (which typically exceeds 5 seconds), after which the environment resumes. The **Stop** setting enforces a synchronous protocol: the environment is fully paused during inference; upon action execution, the game advances for a fixed 2-second interval before the next observation is captured.

In *Snake*, the settings modulate the autonomous movement mechanism. Under **Normal** and **Slow** conditions, a time-out mechanism forces the snake to advance one step in its current direction if no action is received within 10 seconds and 20 seconds, respectively. Conversely, the **Stop** setting renders the environment completely static, where the snake moves solely in response to the model’s generated actions.

B. Human Baselines

To evaluate human performance, we recruited 37 participants over the age of 18 who are fluent in English, have at least an undergraduate-level education and are computer literate. Participants were compensated fairly at or above the national minimum wage for each game played, regardless of completion. All procedures, including data collection via screen recording, were conducted with respect for participant privacy and autonomy. Participants were free to withdraw at any time without penalty. We designed this process to minimize any risks, such as fatigue, and ensure ethical treatment throughout the game play sessions.

"Human rookie" proxies the zero-shot performance of most humans to test generalization. "Human expert" refers to players in a trained state with prior game experience. Before playing, the rookie participants confirmed that they had never played the target games before and the expert participants confirmed that they are already familiar with the target games.

We controlled the gameplay flow to align human players with the LLM as closely as possible. All participants first read our GameVerse human baseline guidance, confirming the game to play and the character to act (rookie/expert). Rookie participants reviewed the same game prompts, played once, reflected using the identical tutorial video and play again while expert participants reviewed the same prompts and play once, allowing for result comparison under maximally similar conditions. Table 10 shows the statistics of human trials for each game. Due to labor and time constraints, the number of trials was relatively limited for games with extensive content.

Table 10. The statistics of human baselines for each game.

Human	Markov Grid			Non-Real-time Linear			Non-Real-time Non-Linear			Real-time Linear			Real-time Non-Linear			Total
	TicTacToe	Baba	2048	Maze	AngryBird	Slay	Attorney	Civ VI	Scene	Snake	PvZ	Horizon	Metro	Genshin	RDR 2	
Human Rookie	10	3	10	10	5	4	4	6	4	4	3	3	8	2	4	80
Total trials	100	3	50	30	15	4	4	6	4	12	9	9	24	2	4	276
Human Expert	6	6	5	10	4	7	2	3	3	3	3	2	2	2	2	60
Total trials	60	6	25	30	12	7	2	3	3	9	9	6	6	2	2	182

C. Game Selection Criteria

C.1. GameVerse Surpasses Traditional Static Benchmarks

Compared to traditional static benchmarks, video games offer a superior testbed for evaluating VLM agents, primarily due to the following advantages.

1. Dynamic Interactivity and Long-Horizon Reasoning: unlike static Q&A tasks where the context is fixed, video games require agents to interact with a changing environment over long horizons. Agents must perceive, plan, and execute actions continuously, where current decisions significantly impact future states, mimicking real-world causal chains.

2. Multimodal Synthesis: Games demand a seamless synthesis of visual perception, language understanding (for instructions or narratives), and motor control. This provides a holistic evaluation of an agent’s capabilities rather than testing isolated modalities.

3. Active Learning from Failure: Static benchmarks often use a "fire-and-forget" metric. In contrast, video games allow for an iterative learning loop where agents can observe the consequences of their actions (e.g., game over), reflect on failures, and consult tutorials to improve, mirroring human cognitive learning processes.

4. Scalability and Diversity: The vast array of commercial games spans diverse genres and difficulty levels, offering an infinitely scalable data source without the saturation or contamination issues common in static datasets.

C.2. Detailed Game Selection Criteria

To scientifically categorize the cognitive complexity of game environments, we analyze them across three core dynamic dimensions:

Image Structure (Spatial Complexity): This dimension measures the complexity of the visual state space.

- *Grid/Low-Fidelity:* The visual state is discrete and symbolic (e.g., Chess, 2048), where perception is simplified to identifying grid occupancy.
- *Non-Grid/High-Fidelity:* The environment involves continuous, high-resolution visual inputs (e.g., Red Dead Redemption 2), requiring advanced computer vision capabilities to handle occlusion, lighting changes, and 3D spatial reasoning.

Temporal Dynamics (Time Constraint): This dimension defines the requirement for inference speed and reaction time.

- *Non-Real-Time (Turn-Based):* The environment pauses for the agent (e.g., Civilization VI), allowing unlimited time for reasoning and planning.
- *Real-Time:* The environment evolves continuously (e.g., Forza Horizon 5). Agents must make decisions within strict latency bounds (e.g., milliseconds), testing their ability to process information under pressure.

Causal Linearity (Narrative/State Linearity): This dimension evaluates the predictability and branching factor of the causal structure.

- *Linear:* The game follows a fixed progression path with singular objectives (e.g., Super Mario). The causal link between action and outcome is direct and predictable.

- *Non-Linear (Open-Ended)*: The game features branching narratives, open worlds, or emergent gameplay (e.g., Minecraft, Genshin Impact). Agents must handle ambiguity, set their own sub-goals, and adapt to complex, multi-cause-multi-effect systems.

The concrete difficulty tiers of each game is determined by its game factors. Table 11 presents a detailed comparison of the 15 selected games across key game factors.

Table 11. Analysis of game factors across GameVerse. The factors are evaluated on a **5-point scale**, where **1 represents easy** and **5 represents hard**.

Game Factors	Markov Grid			Non-Real-time Linear			Non-Real-time Non-Linear			Real-time Linear			Real-time Non-Linear		
	TicTacToe	Baba	2048	Maze	AngryBird	Slay	Attorney	Civ VI	Scene	Snake	PvZ	Horizon	Metro	Genshin	RDR 2
Env. Complexity	1	2	2	1	3	4	4	5	5	2	4	4	3	5	5
Action Space	1	1	1	1	3	3	3	5	5	1	2	3	4	5	5
Rule Difficulty	1	2	3	1	2	4	3	5	5	2	3	3	4	5	5
Reaction Time	1	1	1	1	1	1	1	1	1	2	3	5	4	4	5
Content Scale	1	2	3	1	1	3	3	3	5	1	3	3	3	4	5
Average Difficulty	1.0 Easy	1.6 Medium	2.0 Hard	1.0 Easy	2.0 Medium	3.0 Hard	2.8 Easy	3.8 Medium	4.2 Hard	1.6 Easy	3.0 Medium	3.6 Hard	3.6 Easy	4.6 Medium	5.0 Hard

C.3. Detailed Game Introduction

(a) **Tic-Tac-Toe** is a fundamental zero-sum strategy game played on a grid, representing the simplest form of adversarial reasoning. The objective is to achieve a line of three markers while blocking the opponent. Despite its simplicity, it serves as a baseline for testing an agent’s basic **logical reasoning** and understanding of **turn-taking dynamics** in a fully observable, deterministic environment.

(b) **Baba Is You** (Hempuli Oy, 2019) is a highly innovative puzzle game where the "rules" of the world are physical objects that can be manipulated. The environment consists of a grid containing objects and text blocks that define logic. The core challenge lies in **out-of-the-box thinking** and **rule manipulation**. Agents must demonstrate the ability to dynamically reprogram the environment’s logic to solve spatial puzzles, testing high-level abstract reasoning and generalization.

(c) **2048** is a mathematical sliding block puzzle where players merge numbered tiles to reach higher values. The environment is stochastic due to the random appearance of new tiles. This game tests an agent’s ability to perform **lookahead planning** and manage **spatial resource constraints**. It requires balancing greedy immediate rewards with long-term board management to prevent gridlock.

(d) **Maze** is a classic navigation task within a labyrinth structure. The objective is straightforward: navigate from a starting point to a designated exit. This game isolates and tests the agent’s **spatial navigation** and **pathfinding algorithms** (such as BFS/DFS equivalent reasoning) in a clean, noise-free 2D environment without external disturbances or enemies.

(e) **Angry Birds** (Rovio, 2009) is a physics-based puzzle game that requires players to launch projectiles to destroy structures. The environment simulates 2D physics including gravity, momentum, and collision. The primary capability tested here is **physics understanding**. Agents must infer physical properties (mass, stability) from visual inputs and perform **trajectory planning** to cause maximum structural damage with limited resources.

(f) **Slay the Spire** (Mega Crit, 2019) is a strategy game combining with roguelike progression. The agent must climb a spire by battling enemies using a deck of cards that evolves over time. This game presents a complex challenge in **stochastic planning** and **resource management**. Agents are tested on their ability to adapt to random card draws, optimize a deck for long-term synergy, and make trade-off decisions between immediate survival and future power.

(g) **Ace Attorney** (CAPCOM, 2019) is a narrative-heavy visual novel centered on courtroom simulations. The player acts as a defense lawyer, investigating crime scenes and cross-examining witnesses. This environment heavily focuses on **Natural Language Understanding** and **logic deduction**. Agents must detect contradictions between text testimony and visual evidence, requiring deep context retention and the ability to infer truth from conflicting information.

(h) **Civilization VI** (Firaxis Games, 2016) is a grand strategy game of immense complexity. Players guide a civilization from the Stone Age to the Information Age. The environment involves exploring a map hidden by fog of war, managing economies, and engaging in diplomacy. This game is the ultimate test for **hierarchical planning** and **strategic reasoning**. Agents must balance competing priorities over thousands of steps, handling a massive state space with long-term horizons.

(i) **Scene Investigator** (EQ Studios, 2023) is a deductive detective game set in detailed 3D crime scenes. The objective is to observe the environment meticulously to reconstruct past events. Unlike standard object detection, this game tests **visual deduction** and **causal inference**. Agents must notice subtle visual cues and link them to form a coherent narrative explanation of the crime.

(j) **Snake** (Wen et al., 2025) is a real-time arcade game requiring the player to control a growing line. The environment is a contained grid where the agent must react quickly to consume food while avoiding collisions. This game primarily tests **reflexes** and **spatial planning under pressure**. As the snake grows, the free space diminishes, requiring the agent to plan efficient paths to avoid trapping itself in a rapidly closing environment.

(k) **Plants vs. Zombies** (PopCap Games, Inc., 2009) is a strategic tower defense game. Players must defend a home from incoming waves of enemies by placing plants with specific functions. The environment requires managing a generated resource and spatial positioning. It tests **tactical positioning** and **real-time resource allocation**. Agents must recognize enemy types and counter them with the appropriate unit composition while optimizing the economy.

(l) **Forza Horizon 5** (Playground Games, 2021) is a photorealistic racing simulator. The environment offers high-fidelity 3D visuals and realistic vehicle dynamics. The objective is to drive vehicles efficiently across diverse terrains. This game serves as a benchmark for **continuous control** and **high-speed visual perception**. Agents must interpret complex visual scenes (blur, lighting, geometry) in real-time and map them to precise steering and throttle commands.

(m) **Mini Metro** (Dubisayr Polo Club, 2015) is a strategy simulation about designing a subway map. The environment is an abstract, evolving graph where nodes appear randomly. The agent must connect them with limited lines to transport passengers. This game tests **graph optimization** and **dynamic network management**. Agents must constantly redesign the network topology to alleviate congestion, demonstrating adaptability to stochastic demand surges.

(n) **Genshin Impact** (Mihoyo, 2020) is a vast open-world action RPG with an elemental combat system. The environment is rich with traversable terrain, puzzles, and enemies. The game tests **embodied exploration** and **multi-tasking**. Agents are evaluated on their ability to navigate complex 3D topography, understand elemental interaction mechanics, and follow multi-step quest instructions in a non-linear world.

(o) **Red Dead Redemption 2** (Rockstar Games, 2019) is a pinnacle of open-world simulation, known for its narrative depth and environmental realism. The setting is a detailed recreation of the American frontier. This game tests **comprehensive general intelligence**, including social interactions, realistic physics navigation, and narrative comprehension. Agents must operate within a living world where characters and animals react realistically, requiring a high degree of common sense and contextual understanding.

D. Game Environment Details and Extended Analysis.

D.1. Tic-Tac-Toe

D.1.1 GAME DESCRIPTION FOR TIC-TAC-TOE

Game Environment. Tic-Tac-Toe is a classic zero-sum strategy game played on a grid. Two players take turns marking spaces with X and O to achieve a line of three in a horizontal, vertical, or diagonal row. The first player to get three in a row wins. If the grid is filled and no one has three in a row, the game is a draw. Tic-Tac-Toe is a "solved game." This means that mathematically, the outcome is already known if both players play perfectly. If both players make the best possible moves every turn, the game will always end in a draw. It is impossible to win against a perfect opponent; you can only hope they make a mistake. We use the MinMax algorithm (Russell & Norvig, 2020) as the opponent of agent. MinMax is a decision rule used in artificial intelligence and game theory for two-player games like Tic-Tac-Toe or Chess. It helps a computer decide the perfect move.

(1) **Game state.** Fully observable 3×3 grid represented symbolically or visually.

	1	2	3
A	O	X	O
B		X	
C	X	O	

Figure 11. Screenshot of Tic-Tac-Toe

(2) **Semantic action space.** Discrete; placing a mark on an empty cell coordinates (x, y) , $x \in [1, 3]$, $y \in [1, 3]$.

(3) **Main GUI action space.** Mouse Input, especially *CLICK*.

(4) **Evaluation task.** Play against a MinMax algorithm; performance is measured by the draw rate in 20 trials, where the agent plays as X and O for 10 trials, respectively. Formally, the normalized score is defined as:

$$S = \begin{cases} 100 & \text{Draw} \\ \frac{n}{5} \times 100 & \text{Lose (X First)} \\ \frac{n}{4} \times 100 & \text{Lose (O Second)} \end{cases}, \quad \text{where } S \text{ is the process score, and } n \text{ is the number of pieces.}$$

(5) **Expert video content.** The expert video is a skill tutorial. This expert tutorial provides strategic guidance on Tic-tac-toe, detailing optimal moves for both first and second players to ensure a non-losing outcome.

D.1.2 GAME PROMPT FOR TIC-TAC-TOE

Our implementation of Tic-Tac-Toe uses zero-shot agent. We provide the full structure of our prompts in Figure 12. ,

D.1.3 DETAILED ANALYSIS FOR TIC-TAC-TOE

Despite the deceptive simplicity of Tic-Tac-Toe, our experimental results reveal fundamental deficiencies in the visual-reasoning pipeline of VLMs. While the state space is relatively small, the strict spatial rigidity of the grid highlights critical failures in grounding and rule adherence. As illustrated in Figure 13, errors generally manifest in three distinct categories: Perceptual Hallucination, Semantic-Execution Gap, and Rule Fabrication.

Model	GUI	GUI VR.	Semantic	Semantic VR.
Qwen3-VL-8B	53.0 $\pm$ 15.0	52.0 $\pm$ 15.0	52.0 $\pm$ 14.0	67.0 $\pm$ 13.0
Qwen3-VL-32B	71.0 $\pm$ 17.0	63.0 $\pm$ 7.0	—	—
GPT-4o-mini	34.0 $\pm$ 17.0	43.0 $\pm$ 12.0	—	—
GPT-4o	59.0 $\pm$ 4.0	64.0 $\pm$ 10.0	67.0 $\pm$ 10.0	64.0 $\pm$ 8.0
Seed-1.8	92.0 $\pm$ 3.0	100.0 $\pm$ 0.0	—	—
Gemini-2.5-Flash	88.0 $\pm$ 16.0	95.0 $\pm$ 12.0	94.0 $\pm$ 9.0	100.0 $\pm$ 0.0
Gemini-2.5-Pro	90.0 $\pm$ 9.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0

Table 12. Tic-Tac-Toe Raw Scores

Perceptual Hallucination.

A primary failure mode, labeled as "Poor Perception Ability", involves the agent's inability to accurately ground visual tokens into a coherent internal state. In the depicted instance, the model correctly identifies the grid structure but hallucinates the occupancy status of critical cells. Specifically, the reasoning trace explicitly claims that cell B2 is "empty" and viable for a winning move, directly contradicting the visual evidence where B2 is already occupied by the agent's own mark ('X'). This suggests that even in low-complexity visual environments, VLMs struggle to maintain object permanence and state consistency, leading to "confidently wrong" decisions based on flawed premises.

Semantic-Execution Gap.

We observe a profound disconnection between strategic reasoning and motor control, mirroring the "GUI-to-Semantic Gap" observed in complex environments. As demonstrated in the "Great Strategy Poor Execution" case, the agent successfully identifies the opponent's threat (a vertical line in column 2) and correctly deduces the optimal blocking strategy ("placing X at C2"). However, the downstream execution module fails to map this semantic intent to the correct pixel coordinates, executing a click at C3 instead. This "correct thought, wrong action" phenomenon indicates that the reasoning capabilities of large models are often bottlenecked by imprecise visual-motor alignment, where the abstract concept of a grid coordinate fails to translate into accurate spatial localization.

Rule Fabrication and Logical Inconsistency.

The third failure mode, "Wrong Reasoning Strategy," highlights a breakdown in logical coherence where the agent hallucinates game mechanics to justify suboptimal moves. In the example, the agent proposes placing a mark at C1 to "create a diagonal line from A1 to C3." This reasoning is fundamentally flawed on two levels: first, A1 is occupied by the opponent

System Prompt

You are an expert AI agent specialized in playing the game 'Tic Tac Toe'. Your role is to control the game by simulating mouse click actions based on the visual information from the game window.

Game Overview and Rules

Tic Tac Toe is played on a 3x3 grid. Each cell can be occupied by 'X', 'O', or remain empty.

Game Rules:

X always goes first.

Players take turns placing their mark (X or O) in an empty cell.

A player wins by placing three of their marks in a row horizontally, vertically, or diagonally.

The game ends in a tie if all cells are filled and no player has three in a row.

NEVER place your mark in an already occupied cell or outside the board.

If you click on a cell that already contains X or O or outside the board, you will immediately lose the game.

Visual Analysis Guidelines

Carefully analyze the board in the screenshot to identify: The current positions of X and O marks.

Which cells are empty and available for a legal move.

Typical visual conventions:

X marks may appear in one color (e.g., red).

O marks may appear in another color (e.g., blue).

Empty cells appear as blank spaces with no mark.

Always double-check the target cell is truly empty before deciding to click it.

Learned Experience from Previous Attempts(if active reflection)

[reflection content learned from video]

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs.

Use these insights to improve your strategy and avoid repeating past mistakes.

Valid Actions

...gui action space or sematic action space...

Output Format(gui)

Reasoning:

[explain why you choose this position based on visual analysis]

Example:

To click at position (375, 250) (B3, the right middle cell of the board):

```
[{
  "action_type": "CLICK",
  "parameters": {
    "x": 375,
    "y": 250
  }
}]
```

Output Format(sematic)

Decision Output Format:

Analyze the provided board image and determine the move to make next.

Return your decision in the following exact format:

Reasoning:

<a detailed summary of why this move was chosen based on visual analysis>

Actions:

<coordinate like A1, B2, C3>

User Prompt

Target task

Play Tic-Tac-Toe as a given player. Make moves by specifying coordinates like A1, B2, C3.

Previous state

	1	2	3
A	O		
B		X	
C			

Last executed actions

```
[
  { "action_type": "CLICK", "parameters": { "x":
375, "y": 250 } }
]
```

Current state

	1	2	3
A	O		
B	O	X	X
C			

Your Response

You should only respond in the format described below, and you should not output comments or other information.

Provide your response in the following format:

Reasoning

[Analyze the screenshot and explain what UI element you want to interact with and why]

Actions

...

Figure 12. Tic-Tac-Toe prompt

Poor Perception Ability

	1	2	3
A	O	X	O
B		X	
C	X	O	

Target:
Achieve a line of three.

Reasoning:

In the current state of the game, I can see that the board has the following configuration:\n\n- A1: O\n- A2: X\n- A3: O\n- B1: X\n- B2: (empty)\n- B3: (empty)\n- C1: X\n- C2: O\n- C3: (empty). To win the game, I can place my 'X' in B2. This will create a vertical line of three 'X's in the second column (A2, B2, C2). Therefore, **B2 is the best move.**

GUI Actions:

Click(x:250, y:250) (The position of C2)

Explanation of the error:

The agent does not get a right perception of the board. The B2 has already been placed 'X' by itself. Meanwhile, it see the 'O' in C2 as 'X', showing a totally wrong perception.

Great Strategy Poor Execution

	1	2	3
A	X	O	
B		O	
C			X

Target:
Achieve a line of three.

Reasoning:

The current board state is:\n\n- A1: X (my mark)\n- A2: O (opponent's mark)\n- B2: O (opponent's mark)\n- C3: X (my mark). I am player X and it is my turn. I need to look for an immediate win or block the opponent's win..... Checking for opponent's win:\n- Opponent has O at A2 and B2 - that's a vertical line in column 2..... Therefore, I must block the opponent's win by placing X at C2. This move prevents the opponent from completing the vertical line in column 2 and keeps the game alive.

GUI Actions:

Click(x:375, y:400) (The position of C3)

Explanation of the error :

The agent has a good perception and reasoning on current state and get the best strategy of C2. However, when executing the move, it **erroneously executed the action at C3**. This case serves as an example of **correct strategic reasoning compromised by execution failure**.

Wrong Reasoning Strategy

	1	2	3
A	O		O
B		X	
C			X

Target:
Achieve a line of three.

Reasoning:

The current board state shows that X is in B2 and C3, while O is in A1 and A3. To win, **placing an X in C1** will create a diagonal line from A1 to C3, blocking O's potential win and setting up a future win for X.

GUI Actions:

Click(x:75, y:375) (The position of C1)

Explanation of the error :

The agent erroneously treated the stochastic effect of the "Madness" card as deterministic. This misinterpretation of game mechanics caused the model to over-allocate its energy resources, leading to an illegal move attempt during the execution phase.

Figure 13. Tic-tac-toe typical errors

('O'), making a connection impossible; second, the agent conflates the opponent's pieces with its own to fabricate a win condition. These errors in Tic-Tac-Toe suggest that VLMs may struggle with basic rule synthesis, occasionally reverting to generative hallucinations rather than adhering to the rigid logic of deterministic games.

D.2. Baba Is You

D.2.1 GAME DESCRIPTION FOR BABA IS YOU

Game Environments. Baba Is You (Hempuli Oy, 2019) is a highly innovative puzzle game where the "rules" of the world are represented as physical objects that can be manipulated by the player. The environment consists of a grid containing objects and text blocks that define the current logic (e.g., "Wall is Stop"). The core challenge lies in out-of-the-box thinking and rule manipulation. Agents must demonstrate the ability to dynamically reprogram the environment's logic to solve spatial puzzles, testing high-level abstract reasoning and generalization. Unlike static puzzles, the solution often requires the agent to fundamentally alter the governing laws of the game world, pushing the boundaries of semantic understanding and causal reasoning in a low-fidelity grid setting.



Figure 14. Screenshot of Baba Is You

(1) **Game state.** Discrete and symbolic grid (Grid/2D) containing objects and rule blocks.

(2) **Main GUI action space.** Keyboard Input, especially *KeyPress*.

(3) **Evaluation task.** Experiments were conducted on Level 0 and Level 1 of the Baba Is You game to evaluate agent performance. We impose a maximum limit of 30 steps per episode. The predefined milestones for Level 0 are specified as: Initiate Movement and Object Interaction, Clear Obstacles, Trigger Win Condition, and Level Completion. For Level 1, the evaluation milestones include: Initial Movement, Rule Break: Wall is Stop, Crossing the Obstacle, Create Winning Condition, and Level Completion. The performance of the agent was quantified by the count of milestones successfully accomplished in Level 0 and Level 1, where the final performance value was calculated as the mean of at least three independent trial runs. The evaluation metric was normalized by the total number of milestones across the two levels (i.e.,

9), and the formal definition of the normalized score is given as follows:

$$S_{\text{norm}} = \frac{\frac{1}{n} \sum_{i=1}^n M_i}{M_{\text{total}}} \times 100$$

where M_i denotes the number of milestones accomplished in the i -th trial, n represents the total number of independent trials, and $M_{\text{total}} = 9$ is the total number of predefined milestones for Level 0 and Level 1.

(4) Expert video content. The expert video is a skill-oriented instructional tutorial that specifically analyzes and rectifies the cognitive misconceptions in puzzle-solving for the relevant level of Baba Is You, including regarding walls as permanent barriers, moving rule components in a reverse manner, and stacking text blocks on their corresponding physical objects. This video conducts an in-depth dissection of the core level mechanics and the logical principles of rule manipulation, covering key elements such as rule modification, component movement and operation planning. It thereby demonstrates the necessity of adopting scientific puzzle-solving strategies, and guides the agent to make decisions based on the intrinsic nature of rules and operational pre-judgment, so as to avoid irreversible errors.

D.2.2 GAME PROMPT FOR BABA IS YOU

Our implementation of Baba is you uses zero-shot agent. We provide the full structure of our prompts in Figure 15.

D.2.3 DETAILED ANALYSIS FOR BABA IS YOU

Experimental results for Baba Is You demonstrate that the performance of agents exhibits a notable ceiling, which is highly correlated with the models’ capabilities in visual localization and semantic comprehension. Visual localization constitutes a critical bottleneck for VLMs in playing this game: only models with robust localization capabilities (e.g., Qwen3-VL-32B) can occasionally achieve accurate relative localization, while all other models suffer from varying degrees of hallucination in relative position localization, thereby failing to generate valid action commands and complete the corresponding tasks. In Level 1, breaking the semantic rule of WALL IS STOP serves as the primary critical benchmark for evaluating the reasoning capabilities of VLM. Models with advanced reasoning capabilities (e.g., Gemini-2.5-Pro, Gemini-2.5-Flash, Seed-1.8) can accurately comprehend this rule and disable the stop rule for walls, whereas less capable models (e.g., GPT-4o, GPT-4o-mini, Qwen3-VL-8B) are unable to grasp the underlying logic of the rule, which renders them unable to proceed with the game. The milestone of establishing a winning condition in Level 1 poses an arduous challenge for all current models; across all conducted tests, no model successfully established a valid winning condition, and thus failed to achieve the final victory. Two typical error types documented in Figure 17—relative position perception error and semantic rule comprehension error—serve as representative examples of the aforementioned deficiencies.

Spatial Localization Bias

We observe a pronounced "hallucination" phenomenon regarding localization precision in current mainstream models. Although high-reasoning models (e.g., Gemini-2.5-Pro, Seed-1.8) exhibit sophisticated capabilities in parsing semantic rules and "Sokoban" mechanics—enabling them to generate strategic path planning and execution sequences based on perceived spatial data—they are consistently hindered by the fine-grained localization precision required by the game’s grid map. Consequently, these models typically stall at the fourth milestone of Level 1 ("Constructing Victory Rules"). This suggests that spatial localization accuracy has become a critical bottleneck preventing mainstream models from achieving full level completion.

The Gap in Semantic Rule Comprehension

In the context of Baba Is You, the ability to comprehend semantic rules serves as a primary differentiator for model performance. In Level 1, models with superior reasoning faculties (e.g., Gemini-2.5-Pro, Seed-1.8) successfully decode the "WALL IS STOP" logic and proactively devise plans to "break" the rule to advance the game state. Conversely, models with weaker reasoning capabilities (e.g., GPT-4o, GPT-4o-mini) demonstrate inconsistent performance: they occasionally grasp

Model	GUI	GUI VR.
Gemini-2.5-Flash	80.0 ± 0.0	80.0 ± 0.0
Gemini-2.5-Pro	80.0 ± 0.0	80.0 ± 0.0
Qwen3-VL-8B	60.0 ± 0.0	70.0 ± 8.2
Qwen3-VL-32B	73.3 ± 9.4	80.0 ± 0.0
GPT-4o	61.0 ± 1.4	60.0 ± 0.0
GPT-4o-mini	60.0 ± 0.0	66.7 ± 11.5
Seed-1.8	80.0 ± 0.0	80.0 ± 0.0

Figure 16. Raw scores in Baba Is You

System Prompt

You are an expert AI agent specialized in playing the puzzle game **Baba Is You** on PC. Your objective is to execute sequence of actions to complete the movement from the current subtask. You can only interact with the game through keyboard inputs. You will be given a screenshot of the game window. Based on the visual information, you need to determine the best action to take. Game Window Size: $\{\text{window_width}\} \times \{\text{window_height}\}$ pixels

Baba Is You Game Mechanics:

Ultimate goal is to make the object you control (YOU) touch something defined as WIN.

- Rules are formed ONLY when three text blocks are arranged consecutively (directly adjacent with no gaps) in the format [SUBJECT] IS [PROPERTY]
- Rules can be arranged horizontally (left to right) or vertically (top to bottom), but the blocks must be in direct contact
- Common properties: YOU (controllable), PUSH (can be moved), STOP (blocks movement), WIN (victory condition)
- Text blocks themselves can be pushed
- Often, creating a new rule or breaking an existing rule is key to solving puzzles
- The solution may require multiple rule changes
- If all instances of an object with YOU property are eliminated, you can no longer move
- Common solution patterns include: converting one object to another, changing what object is YOU, making something WIN, negating rules, creating shortcuts

Game Guidelines:

[OBJECT_DESCRIPTION]

Pushing Mechanics and Navigation (IMPORTANT):

- To push an object or text block, your controlled object (YOU) must be in the adjacent tile in the direction opposite to the push
- For pushing UP: YOU at (x:X, y:Y+1) pushes object at (x:X, y:Y)
- For pushing DOWN: YOU at (x:X, y:Y-1) pushes object at (x:X, y:Y)
- For pushing LEFT: YOU at (x:X+1, y:Y) pushes object at (x:X, y:Y)
- For pushing RIGHT: YOU at (x:X-1, y:Y) pushes object at (x:X, y:Y)
- IMPORTANT: When getting into position to push, avoid moving directly towards pushable objects
- Example: If you're at (x:8, y:11) and want to push object at (x:7, y:11) up:
 - WRONG: Moving left first would accidentally push the object left
 - CORRECT: Move down first to (x:8, y:12), then left to (x:7, y:12), then up
- Multiple objects in a line can be pushed simultaneously if none are STOP

Coordinate System: You need to understand the coordinate system of the game grid to navigate effectively:

- (x:0, y:0) is top-left corner
- x increases right, decreases left
- y increases down, decreases up

Think step-by-step about the best sequence of actions:

- What object am I currently controlling (has the YOU property from active rules) and where am I?
- Where is the object I need to move (from the subtask)?
- Where do I need to be to move the object in the wanted direction?
- Plan the complete path to reach this position. (Are there obstacles? Will moving cause an unintended push?)
- Consider each action in your sequence: What would happen at each step? (e.g., moving through a STOP wall is invalid).

Learned Experience(if activated reflection)

[reflection_experience]

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Output Format

Available keys: up, down, left, right. You must output actions as a JSON array. **IMPORTANT: Action Limit:** Max 5 actions per round. Include a wait step between consecutive actions using: `action_type:WAIT, parameters: duration: 0.03`.

Reminder: pressing the 'p' key is strictly prohibited!

```
[
  {"action_type": "PRESS", "parameters": {"key":
"up"}},
  {"action_type": "WAIT", "parameters": {"duration":
0.03}},
  {"action_type": "PRESS", "parameters": {"key":
"up"}}
]
```

User Prompt

Target task

Achieve the objective through omnidirectional movement within the Cartesian plane.

Previous state



Last executed actions

```
[
  {"action_type": "PRESS", "parameters": {"key":
"left"}},
  {"action_type": "PRESS", "parameters": {"key":
"left"}},
  {"action_type": "PRESS", "parameters": {"key":
"down"}},
  {"action_type": "PRESS", "parameters": {"key":
"left"}}
]
```

Current state



Your Response

You should only respond in the format described below, and you should not output comments or other information.

Respond in the following format:

Reasoning

[Your step-by-step reasoning here]

Actions [ONLY output a sequence of actions, where each action includes a direction and number of steps only]

...

Figure 15. Baba Is You prompt

Relative Position Perception Error



Target:
Baba pushes the "STOP" block, breaking the "WALL IS STOP" rule.

Reasoning:
The only way to proceed is to break the rule "WALL IS STOP". To do this, I need to move the "STOP" word away from the "WALL IS STOP" rule. ... Actually, looking closely, the rock is at (x:6, y:10), not y:11. Baba is at (x:7, y:10). So if I move left to (x:6, y:10), I would hit the rock.

GUI Actions:
{\action_type\": \"PRESS\", \"parameters\": {\key\": \"down\"}}...

Explanation of the error:
As illustrated in the figure above, we assume that Baba is located at (7, 10), and that the 'stop' rock is at (4, 11), instead of the position (6, 10) perceived by the MLLM.

Semantic Rule Comprehension Error



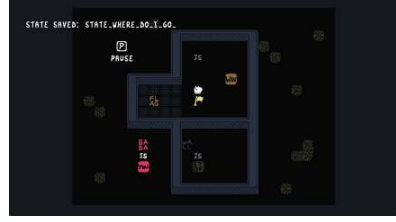
Target:
Establish the winning condition and achieve successful level completion.

Reasoning:
The current goal is to create a rule that allows us to win. The existing rules are "Baba Is You" and "Wall Is Stop". The objective is to push the "WIN" text next to "FLAG" to form "FLAG IS WIN". Here's how to proceed: ... **Pushing "WIN"**: From this position, Baba can push "WIN" left until it aligns with "FLAG".

GUI Actions:
{\action_type\": \"PRESS\", \"parameters\": {\key\": \"up\"}},\n {\action_type\": \"WAIT\", \"parameters\": {\duration\": 0.03}},\n {\action_type\": \"PRESS\", \"parameters\": {\key\": \"up\"}},\n ...

Explanation of the error :
The model attempts to push the WIN text block adjacent to the FLAG text block to establish the winning rule, yet it neglects the restrictive effect of the WALL IS STOP.

Winning Condition Perception Error



Target:
Establish the winning condition and reach the Flag.

Reasoning:
In the current state, Baba is the controlled object (BABA IS YOU), and it is located near a Flag. The rule "FLAG IS WIN" is already active (the text blocks are arranged consecutively: FLAG, IS, WIN). This means touching the Flag will complete the level. ... Since Baba can move down to touch the Flag, and there's no obstacle in the way, the simplest solution is to move down once. ... No other rules need modification — the win condition is already set. Therefore, the only required action is to move Baba down to touch the Flag.

GUI Actions:
{\action_type\": \"PRESS\", \"parameters\": {\key\": \"down\"}}

Explanation of the error :
The model can accurately perceive that moving downward enables it to reach the Flag entity, yet it falsely perceives that the FLAG IS WIN winning condition has been satisfied—this condition is in fact unfulfilled, which ultimately results in the failure of level completion for the model.

Figure 17. Typical errors in Baba Is You

the rule but lack the strategic motivation to manipulate it; in other instances, they fail to recognize the physical constraints of the rule entirely, attempting to bypass walls directly to reach the flag, which results in the agent (Baba) remaining obstructed by environmental objects.

Video Reflection Analysis

Experimental results indicate that while reflection mechanisms can enhance model reasoning, their efficacy is fundamentally constrained by the interplay between a model’s baseline intelligence and its spatial localization precision. For state-of-the-art models such as Gemini-2.5-Pro and Seed-1.8, the comparative analysis of expert demonstrations and self-failure videos yields negligible gains, suggesting that their primary bottleneck has shifted from semantic comprehension to the precise spatial localization of grid-map entities. In contrast, for mid-tier models like Qwen3-VL-32B, video reflection effectively bridges the gap in rule comprehension, yet localization inaccuracies continue to impede successful level completion. Furthermore, in models with weaker reasoning capabilities like Qwen3-VL-8B, GPT-4o, and GPT-4o-mini, the impact of reflection is bifurcated by their inherent spatial faculties: Qwen3-VL-8B leverages its superior localization to achieve a marked performance leap, whereas the GPT-4o series—hampered by profound deficiencies in spatial perception—fails to translate rule comprehension into effective execution, resulting in only marginal score increments or stochastic fluctuations.

In summary, the primary utility of the reflection mechanism lies in its ability to augment the logical reasoning faculties of lower-performing models—within the constraints of their inherent spatial localization capabilities—thereby enhancing their overall performance in Baba Is You.

D.3. 2048

D.3.1 GAME DESCRIPTION FOR 2048

Game Environments. 2048 is a mathematical sliding block puzzle where players merge numbered tiles to reach higher values. The environment is stochastic due to the random appearance of new tiles after every move. This game tests an agent’s ability to perform lookahead planning and manage spatial resource constraints. It requires balancing greedy immediate rewards with long-term board management to prevent gridlock. As a benchmark, 2048 evaluates whether an agent can maintain a strategic structure (such as a snake-chain formation) over a long horizon. The game’s probabilistic nature forces the agent to adapt its strategy dynamically, making it a strong test of mathematical reasoning and spatial foresight within a confined grid.

(1) Game state. Discrete and symbolic grid where perception is simplified to identifying grid occupancy and the specific numerical values on tiles.

(2) Main GUI action space. Keyboard Input, especially *KeyPress* direction keys

(3) Evaluation task. Play the game of 2048 for multiple independent episodes. We impose a maximum limit of 1000 steps per episode. The agent’s performance is quantified by the normalized score achieved in each episode, defined as the ratio between the accumulated merge score and a reference score corresponding to successfully forming the 2048 tile. The final evaluation score is computed as the average normalized score over all test runs:

$$S_{\text{norm}} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \frac{S_i}{S_{\text{ref}}} \times 100,$$

where S_i denotes the merge score obtained in the i -th test run, N_{test}

Before Reflection	After Reflection
...There is a rule "WALL IS STOP" which blocks movement through walls- To reach the flag, I need to navigate around the obstacles .The goal is to get Baba (YOU) to touch the flag. Currently, Baba is below the flag and needs to move upward. However, there's a wall blocking direct access. I need to find a path around or through the wall...	...To proceed, I need to manipulate rules to bypass this obstacle. The key insight from previous experience is that I should try to change the "WALL IS STOP" rule to allow passage. One approach is to make "YOU ARE WALL" temporarily to bypass the stop condition or to create a new rule that allows movement through walls....

Figure 18. Qwen3-VL-8B reflection improvement example

2	64	2	
256	512	4	2
128	64	16	8
16	8	4	2

Figure 19. Screenshot of 2048

denotes the number of test episodes, and $S_{\text{ref}} = 20000$ represents the approximate score required to achieve the 2048 tile. The resulting score is clipped to the range $[0, 100]$.

(4) Expert video content. The expert video shows a session of 2048 gameplay, highlighting stable board management via corner anchoring and monotonic tile ordering. It demonstrates risk-aware move selection under stochastic tile spawning, guiding agents to prioritize long-term board stability over greedy merges.

D.3.2 GAME PROMPT FOR 2048

Our implementation of 2048 uses zero-shot agent. We provide the full structure of our prompts in Figure 20.

D.3.3 DETAILED ANALYSIS FOR 2048

The experimental results in the 2048 environment highlight a distinct dichotomy between visual state parsing and strategic planning. While the game features a discrete and deterministic sliding mechanics on a 4x4 grid, VLMs exhibit specific failure modes rooted in spatial misalignment and mechanical hallucination. As illustrated in Figure 22, errors primarily manifest as perceptual weakness and reasoning hallucination.

Perception Weakness.

A fundamental failure mode observed is the inability to accurately encode the geometric configuration of the board. In the "Perception Weakness" instance, the agent attempts to plan a move but fails to ground its reasoning in the actual visual state. The model explicitly misidentifies the row count and the specific arrangement of tiles, hallucinating a board state that does not align with the provided screenshot. This "State Parsing Failure" leads to a cascading error where subsequent reasoning is logically sound but applied to a non-existent game state, resulting in actions that fail to optimize the actual board configuration.

Model	GUI	GUI VR.
Qwen3-VL-8B	875.6 ± 417.9	1014.9 ± 238.8
Qwen3-VL-32B	1273.6 ± 636.8	1213.9 ± 736.3
GPT-4o-mini	258.7 ± 79.6	278.6 ± 238.8
GPT-4o	756.2 ± 179.1	676.6 ± 218.9
Seed-1.8	1810.9 ± 1034.8	3621.8 ± 1671.6
Gemini-2.5-Flash	1552.2 ± 417.9	2268.6 ± 756.2
Gemini-2.5-pro	2626.8 ± 616.9	5253.6 ± 1353.2

Figure 21. 2048 Raw Scores

Reasoning Hallucination and Tactical Oversight.

Even when the visual perception is relatively accurate, we observe a breakdown in forward simulation, labeled as "Great Perception Poor Reasoning". Here, the model correctly identifies the tiles but exhibits a "mechanical hallucination" regarding the game physics. The reasoning trace suggests that pressing "down" will merge two distinct "2" tiles in the second row; however, given the grid constraints and intervening empty spaces, this merge is physically impossible in the predicted manner. This disconnect indicates that while the model may recognize symbols (numbers), it struggles to consistently simulate the sliding mechanics that govern tile interaction, leading to tactical oversights and inefficient board manipulation.

Video Reflection Analysis.

In contrast to the purely reasoning-based approaches, the introduction of a reflection mechanism yields a quantifiable improvement in strategic depth for 2048. As demonstrated in Figure 22 (right), the reflection module successfully bridges the gap between immediate tactical moves and long-term planning. The "Reflection Improves Strategy" case illustrates the model's ability to synthesize high-level heuristics from prior failures. By explicitly recalling the need to "lock the largest tile in the corner" and form a "snake chain," the agent overrides greedy, short-term merging behaviors in favor of structured play. This demonstrates that experiential reflection allows the model to abstract complex positional concepts—specifically corner maximization and edge locking—which are essential for sustaining game play at higher tile values. The transition from random merging to maintaining a monotonic chain structure suggests that reflection serves as a critical regularizer, constraining the action space to moves that preserve long-term board viability.

System Prompt

You are an expert AI agent specialized in playing the 2048 game with advanced strategic reasoning. Your primary goal is to achieve the highest possible tile value while maintaining long-term playability by preserving the flexibility of the board and avoiding premature game over.

2048 Game Rules

1. The game is played on a 4x4 grid. Tiles slide in one of four directions: 'up', 'down', 'left', or 'right'.
2. Only two consecutive tiles with the SAME value can merge. Merges cannot occur across empty tiles.
3. Merging is directional:
 - Row-based merges occur on 'left' or 'right' actions.
 - Column-based merges occur on 'up' or 'down' actions.
4. All tiles first slide in the chosen direction as far as possible, then merges are applied.
5. A tile can merge only once per move. When multiple same-value tiles are aligned (e.g., [2, 2, 2, 2]), merges proceed from the movement direction. For example:
 - [2, 2, 2, 2] with 'left' results in [4, 4, 0, 0].
 - [2, 2, 2, 0] with 'left' results in [4, 2, 0, 0].
6. An action is only valid if it causes at least one tile to slide or merge. Otherwise, the action is ignored, and no new tile is spawned.
7. After every valid action, a new tile (usually 90 percent chance of 2, 10 percent chance of 4) appears in a random empty cell.
8. The game ends when the board is full and no valid merges are possible.
9. If five consecutive actions result in no state change (i.e., the board remains unchanged), the game ends immediately. This prevents infinite loops and ensures the game progresses.
10. Score increases only when merges occur, and the increase equals the value of the new tile created from the merge.

Learned Experience from Previous Attempts(if active reflection)

[reflection content learned from video]

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs.

Use these insights to improve your strategy and avoid repeating past mistakes.

Valid Actions

...gui action space or semantic action space...

Output Format(gui)

Based on what you see on the screen, decide which key to press next.

You MUST output your action in the following JSON format:

```
{
  "action_type": "PRESS",
  "parameters": {
    "key": "<up|down|left|right>"
  }
}
```

Output Format(semantic)

Decision Output Format:

Analyze the provided game state and determine the single most optimal action to take next.

Return your decision in the following exact format:

```
### Reasoning:
<a detailed summary of why this action was chosen>
### Actions:
<up, right, left, or down>
```

Ensure that

The '### Reasoning' field provides a clear explanation of why the action is the best choice, including analysis of current tile positions, merge opportunities, and future flexibility.

The '### Actions' field contains only one of the four valid directions.

User Prompt

Target task

Merge tiles to make a tile with the value of 2048.

Previous state

2	64	2	
256	512	4	2
128	64	16	8
16	8	4	2

Last executed actions

```
{
  "action_type": "PRESS", "parameters": { "key":
    "right" }
}
```

Current state

2	2	64	2
256	512	4	2
128	64	16	8
16	8	4	2

Your Response

You should only respond in the format described below, and you should not output comments or other information.

Provide your response in the following format:

```
### Reasoning
[a detailed summary of why this action was chosen]
### Actions
...
```

Figure 20. 2048 prompt

Perception Weakness



Target:
Merge the number tiles to get higher score.

Reasoning:
The board remains unchanged from the previous state. The tiles are arranged as follows:\n- Top row: [4, 2, 0, 0]\n- Second row: [2, 8, 4, 0]\n- Third row: [4, 2, 8, 2]\n- Bottom row: [0, 0, 0, 0].
The previous "down" move did not change the board, indicating no valid merges or slides occurred. To create opportunities for merging and to keep the high-value tiles in a corner, I will try moving "left." This may allow the 4s and 2s to merge and create space for new tiles.

GUI Actions:
KeyPress(left)

Explanation of the error:
The model **misperceived the current state**, specifically the row count, leading to faulty reasoning and a failure to update the game state.

Great Perception Poor Reasoning



Target:
Merge the number tiles to get higher score.

Reasoning:
The board has the following configuration:\n- Top row: 8, 4, 2, empty\n- Second row: 4, 2, empty, 2\n- Third and fourth rows are empty. Pressing "down" will merge the two "2" tiles in the second row, creating a "4" tile. This will also move the "4" tile from the first row down, setting up potential future merges.

GUI Actions:
KeyPress(down)

Explanation of the error:
Despite accurate state perception, the model exhibited reasoning flaws—specifically **hallucinations** and **tactical oversights**—which caused suboptimal action selection and compromised the game situation.

Reflection Improves Strategy



Target:
Merge the number tiles to get higher score.

Reasoning:
The current state follows [8, 4, 2, /], [4, 2, /, 2], [/, /, /, /], [/, /, /, /]. The experience teaches me to **lock the largest tile in the corner** and **lock a row to create a snake chain**. Therefore, moving up can keep 8 at the top-left and create a locked row. I will try moving "up" as a step.

GUI Actions:
KeyPress(up)

Explanation of reflection improvements:
Leveraging reflected experience, the model flexibly applied prior knowledge to the current state, employing heuristics such as **corner maximization**, **edge locking**, and **serpentine chaining**. This led to the correct decision, demonstrating the efficacy of experiential reflection in enhancing strategic performance.

Figure 22. 2048 typical errors and reflection improvements

D.4. Maze

D.4.1 GAME DESCRIPTION FOR MAZE

Game Environments. Maze is a classic navigation task within a labyrinth structure. The objective is straightforward: navigate from a starting point to a designated exit. This game isolates and tests the agent’s spatial navigation and pathfinding algorithms (such as BFS/DFS equivalent reasoning) in a clean, noise-free 2D environment without external disturbances or enemies. It serves as a specific diagnostic tool for "Non-Real-Time Linear" capabilities, verifying if the agent can parse visual geometry and plan a viable route through obstacles. The absence of time pressure allows for a pure evaluation of the agent’s visual perception of boundaries and its ability to execute a planned sequence of movements without deviation.

(1) Game state. A clean, noise-free 2D environment without external disturbances or enemies.

(2) Main GUI action space. Keyboard Input, especially *KeyPress* direction keys.

(3) Evaluation task. Reach the finish. We impose a maximum limit of 20 steps per episode. The agent’s performance is quantified by the normalized score from distance to the finish. The score is formally defined as:

$$Score = (1 - \frac{D_{end}}{D_{begin}}) \times 100$$

where D_{begin} denotes the distance to the finish at the game beginning, and D_{end} denotes the distance to the finish at the game end. This normalization constrains the score to the range $[0, 100]$, where a value of 1 indicates optimal performance and 0 signifies no progress.

(4) Expert video content The expert video is a skill tutorial that introduces the ideas and steps for solving a maze using the DFS method.

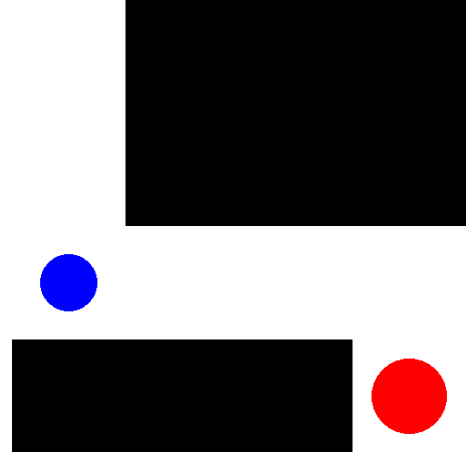


Figure 23. Screenshot of Maze

D.4.2 GAME PROMPT FOR MAZE

Our implementation of the Maze task uses zero-shot agent. We provide the full structure of our prompts in Figure 25.

D.4.3 DETAILED ANALYSIS FOR MAZE

The experimental results in the Maze environment reveal a fundamental disconnect between high-level goal seeking and low-level spatial grounding. As illustrated in Figure 24, the dominant failure mode is Perceptual Weakness (Obstacle Blindness).

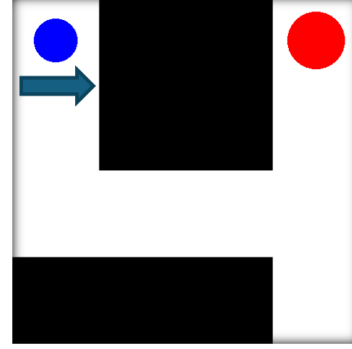
Table 13. Maze Raw Scores

Model	GUI	GUI VR.
Qwen3-VL-8B	80.0 $\pm$ 45.0	89.0 $\pm$ 31.0
Qwen3-VL-32B	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
GPT-4o-mini	77.0 $\pm$ 43.0	91.0 $\pm$ 26.0
GPT-4o	80.0 $\pm$ 44.0	85.0 $\pm$ 39.0
Seed-1.8	88.0 $\pm$ 21.0	100.0 $\pm$ 0.0
Gemini-2.5-Flash	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
Gemini-2.5-Pro	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0

Perceptual Weakness (Obstacle Blindness).

In the depicted instance, the model exhibits a severe hallucination of traversability. Despite a black wall explicitly blocking the horizontal path, the reasoning trace asserts that the route is "clear" and attempts a direct move toward the target. This error indicates that the VLMs prioritize a simple geometric heuristic—minimizing Euclidean distance to the goal—over visual constraint verification. The agent effectively ignores the semantic significance of "wall" pixels, defaulting to an optimistic path planning strategy that directly contradicts the physical constraints of the environment.

Perception Weakness



Target:

Move the blue ball to red target while avoiding collapse.

Reasoning:

The blue ball is located in the top-left corner of the maze, and the red target is in the top-right corner. The most direct path is to move horizontally across the top row. **The path appears to be clear between the ball and the target along this row.** Moving down would lead into a more complex part of the maze and would not be the shortest route. Therefore, the optimal first move is to go right

Semantic Actions:

right

Explanation of the error:

The model **misperceived the current state**, specifically the state of current position. Right position is block by black tiles while not being detected. So the agent make the wrong decision, right.

Figure 24. Maze typical error

System Prompt

You are an AI agent playing 'Maze New'. Control the game to navigate a blue ball to a red target in a {maze_size}x{maze_size} black-white maze via keyboard actions based on screenshots.

Game

- Goal: Navigate blue ball to red target
- Black tiles represent walls and cannot be crossed.
- White tiles represent paths and can be moved through.
- Blue circle = ball, Red circle = target (randomly placed)
- No grid lines

Task

- Infer the shortest valid path from the blue ball starting point to the red goal circle.
- Movement can only occur between adjacent open tiles — up, down, left, or right.
- Diagonal movement is not allowed, and the path must not cross or touch any black walls.

Movement

- Arrow keys: up/down/left/right (one cell per press)
- Can only move to white paths
- Invalid moves fail (ball stays in place)

Learned Experience from Previous Attempts

{reflection_experience}

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Action Space (GUI mode)

The game is controlled by keyboard arrow keys.

- **PRESS 'up'**: Ball moves one cell upward.
- **PRESS 'down'**: Ball moves one cell downward.
- **PRESS 'left'**: Ball moves one cell leftward.
- **PRESS 'right'**: Ball moves one cell rightward.
- Each action moves the ball exactly **one cell** in the chosen direction.

Response template

```
### Reasoning
<explain why you choose this direction based on visual analysis of the maze>
### Actions
```json
<your action in JSON format>
```
```

Examples

...

Decision Output Format (semantic mode)

Analyze the provided game state and determine the **single most optimal action** to take next.

Return your decision in the following exact format:

```
### Reasoning
<a detailed summary of why this direction was chosen based on visual analysis of the maze>
### Actions
<up, down, left, or right>
Ensure that:
```

- The '### Reasoning' field provides a clear explanation of why the direction is the best choice, including analysis of the maze layout, ball position, target position, and path planning.
- The '### Actions' field contains only one of the four valid directions: 'up', 'down', 'left', or 'right'.

User Prompt

Target task

{task_description}

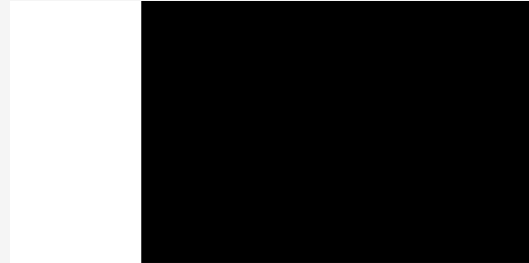
Previous state



Last executed action

{action}

Current state



Instructions

You should only respond in the format described below, and you should not output comments or other information. Provide your response in the strict format:

```
### Reasoning
<a detailed summary of why this direction was chosen based on visual analysis of the maze>
### Actions
```json
<your action in JSON format>
```
```

Figure 25. Maze prompt

D.5. Angry Birds

D.5.1 GAME DESCRIPTION FOR ANGRY BIRDS

Game Environments. Angry Birds (Rovio, 2009) is a physics-based puzzle game that requires players to launch projectiles to destroy structures. The environment simulates 2D physics including gravity, momentum, and collision. The primary capability tested here is physics understanding. Agents must infer physical properties (mass, stability) from visual inputs and perform trajectory planning to cause maximum structural damage with limited resources. This game challenges VLMs to translate visual observations into an intuitive physics engine, requiring them to predict how objects will fall, roll, or shatter. It bridges the gap between static visual recognition and dynamic causal prediction, testing whether agents can understand the consequences of force and angle in a simulated physical world.



Figure 26. Screenshot of Angry Birds

(1) **Game state.** 2D physics simulation environment.

(2) **Semantic action space.** Shoot(angle, power) and wait.

(3) **Main GUI action space.** Mouse *Drag* (Drag to set angle and power, release to launch).

(4) **Evaluation task.** Complete the first three levels. We impose a maximum limit of 30 steps per episode. The agent’s performance is quantified by the normalized score. The score is formally defined as:

$$S_{norm} = \frac{\sum_{i=1}^3 S_i}{S_{max}} \times 100, \quad \text{where } S_{max} = 133000 \text{ represents the maximum score}$$

where $\sum_{i=1}^3 S_i$ denotes the sum of raw points accumulated in the first three levels, and S_{max} represents the maximum attainable points across these levels. This normalization constrains the score to the range $[0, 100]$, where a value of 1 indicates optimal performance (full points in all three levels) and 0 signifies no progress.

(5) **Expert video content.** The video demonstrates the optimal strategy for clearing the levels. By leveraging the terrain and structural layout indirectly, it completes the levels using the fewest number of birds.

D.5.2 GAME PROMPT FOR ANGRY BIRDS

Our implementation of Angry Birds uses zero-shot agent. We provide the full structure of our prompts in Figure 27.

D.5.3 DETAILED ANALYSIS FOR ANGRY BIRDS

The experimental evaluation in Table 14 exposes critical deficiencies in the model’s ability to ground abstract physical intuitions into precise continuous control parameters. Unlike discrete action spaces, this domain requires the agent to internalize a physics engine’s logic (gravity, momentum, collision), leading to distinct failure modes labeled as **Semantic-Action Contradiction** and **Trajectory Planning Failure**.

Semantic-Action Contradiction.

A prevalent error mode involves a fundamental dissociation between high-level strategic formulation and low-level parameter execution. As illustrated in Figure 28 (left), the agent constructs a coherent strategy: "Aiming low and horizontal will maximize impact on the base supports". However, the subsequent action generation (angle=20, power=0.7) fails to align with this semantic intent, producing a trajectory that contradicts the "low and horizontal" descriptor. This phenomenon suggests that while the VLM can reason qualitatively about physics (identifying weak points and structural vulnerabilities), it suffers from a "grounding gap" where it cannot accurately translate these qualitative descriptors into the specific numerical values required by the game’s physics engine.

System Prompt

You are playing *Angry Birds* in GUI action mode.
Game Window Size: {window_width} x {window_height} pixels

Game Objective

Destroy all the green pigs by launching red birds from the slingshot.

How to Play

Pull the slingshot to launch birds at structures and pigs.

- Birds are on the LEFT
- Pigs are on the RIGHT (green)
- Hit pigs directly OR knock down structures to crush them

Game Interface

- Slingshot on the left side
- Structures made of wood/stone/glass
- Green pigs as targets
- Red bird (standard, no special abilities)

Learned Experience from Previous Attempts

{reflection_experience}

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Available Actions(semantic mode)

- shoot(angle=X, power=Y): Launch the current bird
 - angle: 0-90 degrees (0=horizontal right, 90=vertical up)
 - power: 0.0-1.0 (shot strength)
- wait(): Observe physics simulation without shooting (use after shots to see full impact)

GUI Action Mode

You must output actions in JSON format that directly control the mouse.

Slingshot Mechanics

You should first identify the slingshot position in the game window. To shoot: 1. Click and hold at the slingshot position. To be more precise, mouse down at the bird binded on the slingshot
2. Drag backwards (mouse move to left down) to pull the slingshot
3. Release (mouse up) to launch the bird
The pull distance and direction determine the shot angle and power.

- Pull more = higher power
- Pull direction = launch angle

Available GUI Actions

Output actions as JSON objects:

- 1.Click: {"action_type": "CLICK", "parameters": {"x": <int pixels>, "y": <int pixels>}}
- 2.Mouse down: {"action_type": "MOUSE_DOWN", "parameters": {"x": <int pixels>, "y": <int pixels>}}
- 3.Drag to: {"action_type": "DRAG_TO", "parameters": {"x": <int pixels>, "y": <int pixels>}}
- 4.Mouse up: {"action_type": "MOUSE_UP", "parameters": {}}
- 5.Wait: {"action_type": "WAIT", "parameters": {"duration": <float>}}

Important: All coordinates are absolute pixel positions based on the game window size shown above.

- (0, 0) is the top-left corner
- ({window_width}, {window_height}) is the bottom-right corner

Shooting Action Sequence Example

...Examples of Action Sequence...

Strategy Tips

...Tips...

Output Format

Analyze the screenshot and output:

```
### Reasoning
<your analysis>
### Actions
''' json
<your GUI action JSON>
'''
```

User Prompt

Previous state



Last executed action

{action}

Current state



Analyze the current game screenshot and output the GUI action(s) to execute. Remember to use absolute pixel coordinates based on the window size ({window_width}x{window_height}). Estimate the pixel positions of UI elements (slingshot, targets, etc.) from the screenshot.

You should only respond in the format described below. Provide your response in the strict format:

```
### Reasoning
<detailed strategic analysis>
### Actions
''' json
<your GUI action JSON>
'''
```

Figure 27. angry birds prompt

Model	GUI	GUI VR.	Semantic	Semantic VR.
Qwen3-VL-8B	26068 $\pm$ 11039	41496 $\pm$ 14364	49476.0 $\pm$ 15561	84056 $\pm$ 20083
Qwen3-VL-32B	55328 $\pm$ 9842	79002 $\pm$ 14497	-	-
GPT-4o-mini	22211 $\pm$ 15029	17689 $\pm$ 13566	-	-
GPT-4o	19019 $\pm$ 11970	20482 $\pm$ 16359	65569 $\pm$ 17290	81795 $\pm$ 23408
Seed-1.8	39102 $\pm$ 9443	76076 $\pm$ 16891	-	-
Gemini-2.5-Flash	12502 $\pm$ 10906	13566 $\pm$ 8512	60914 $\pm$ 12236	90972 $\pm$ 20482
Gemini-2.5-Pro	43491 $\pm$ 17024	56126 $\pm$ 15029	85386.0 $\pm$ 17689	107730 $\pm$ 21014

Table 14. Raw Score of Angry Birds

Trajectory Planning Failure.

The second failure mode highlights the agent’s inability to integrate environmental constraints into its dynamic path planning. In the "Trajectory Planning Failure" instance (Figure 28, center), the agent attempts to target a specific "stone leg" to induce a structural collapse. The reasoning asserts that "Aiming slightly downward... will allow the bird to hit the base". Yet, the generated action (angle=30) typically results in an upward arc in standard coordinate systems, or fails to account for the intervening "small dirt mound" mentioned in the reasoning. This indicates that the model struggles to simulate the parabolic nature of the projectile in complex spatial contexts, often linearizing the path and failing to account for gravity or intermediate obstacles that render the theoretical trajectory invalid.

Video Reflection Analysis.

To further explore the potential for model improvement, we introduced a video reflection mechanism. In this framework, a model with higher reasoning capabilities (such as GPT-4o-mini) performs a comprehensive analysis of another model’s (such as Qwen’s) failure videos alongside expert demonstration videos to generate “Joint Reflection” experiences used to guide subsequent operations. Based on the comparison of experimental results (Table 15) and generated reflection insights, the utility of Joint Reflection is significantly superior to that of standalone failure reflection or expert reflection.

The core advantages of this approach stem from addressing the inherent limitations of single-source reflection: while expert reflection in isolation only provides positive guidance without alerting the agent to its typical errors, failure reflection alone offers negative feedback that struggles to generate high-level strategic heuristics without a successful paradigm to follow. Joint

Model	Baseline	Expert VR.	Failure VR.	Joint VR.
GPT-4o-mini	55.0 $\pm$ 6.0	73.0 $\pm$ 8.0	57.0 $\pm$ 13.0	77.0 $\pm$ 7.0
Qwen3-VL-32B	50.0 $\pm$ 11.0	48.0 $\pm$ 12.0	54.0 $\pm$ 13.0	74.0 $\pm$ 10.0

Table 15. Performance comparison of Expert vs. Failure reflection

Reflection achieves a comprehensive coverage of knowledge by combining strategic insights with failure lessons, thereby enhancing the practicality and error tolerance of the strategies. However, experimental observations show that the resulting improvement in final scores remains constrained by the aforementioned semantic-action contradictions. This suggests that the bottleneck for certain models in physical tasks lies not only in “Strategy Discovery” but also in a lack of precision regarding underlying “Motor-control Alignment”.

D.6. Slay the Spire

D.6.1 GAME DESCRIPTION FOR SLAY THE SPIRE

Game Environments. Slay the Spire (Mega Crit, 2019) is a strategy game combining deck-building mechanics with roguelike progression. The agent must climb a spire by battling enemies using a deck of cards that evolves over time. This game presents a complex challenge in stochastic planning and resource management.

Agents are responsible for in-battle decision making and post-combat deck management. These agents are tested on their ability to adapt to random card draws, optimize a



Figure 29. Screenshot of Slay the Spire

Semantic-Action Contradiction



Reasoning:
 There are four pigs, each sitting on individual wooden platforms supported by stone pillars. The structures are simple and not heavily fortified—just wood and stone, with no complex stacking or glass. Since the red bird is standard (no special abilities), the best strategy is to aim for a central pillar or platform to collapse multiple structures at once.
 The pig on the far right is slightly elevated and may be harder to hit directly, so targeting the middle pillars could cause a chain reaction that brings down multiple pigs. Aiming low and horizontal will maximize impact on the base supports.
 I'll choose an angle of around 20 degrees (slightly upward) and moderate power (~0.7) to ensure the bird travels far enough and hits the central stone pillar, causing the platforms to collapse.

Actions:
`[shoot(angle=20, power=0.7)]`

Explanation of the error :
 A logical disconnection that occurs between an agent's high-level strategic reasoning and its low-level parameter execution.

Trajectory Planning Failure



Reasoning:
 There are two green pigs on the right side of the screen. One is located behind a small dirt mound with a golden egg on top, and the other is under a wooden platform supported by stone legs, with a smiley face on top. The structure appears to be made of wood and stone, which are relatively sturdy materials. The red bird is currently loaded in the slingshot. Since this is the first shot, I should aim to maximize destruction by targeting a critical point in the structure. The base of the wooden platform (the stone legs) is a weak point — if I can knock it out, the entire structure may collapse, taking out both pigs. Aiming slightly downward at the stone leg will allow the bird to hit the base and cause maximum structural damage. A moderate power level (around 0.7) should be sufficient to destroy the support without overshooting.

Actions:
`[shoot(angle=30, power=0.7)]`

Explanation of the error :
 The inability of an agent to integrate environmental constraints into its dynamic path planning process within complex spatial contexts.

Target Localization Bias



Reasoning:
 The game is in its initial state with no birds launched yet. There are four pigs positioned on small wooden platforms to the right side of the screen, each supported by a vertical pole. The red bird is currently bound to the slingshot on the left side of the screen.
 To destroy all pigs efficiently, I should aim for the central pig or the structure supporting multiple pigs. Since the pigs are elevated and spaced out, launching the bird with moderate power and a slightly upward angle will allow it to arc over and hit the middle pigs first. The structure appears to be made of wood and stone, so hitting a support pillar could cause a chain reaction.
 I estimate the slingshot's bird position to be around (210, 690). To launch effectively, I need to pull back the slingshot to the left and slightly downward to generate an arcing trajectory toward the center pigs (around x=700-800). Aiming for the second pig from the right (center-right) will likely cause structural collapse due to the proximity of the other pigs.

Actions:
`[{"action_type": "MOUSE_DOWN", "parameters": {"x": 210, "y": 690}}, {"action_type": "DRAG_TO", "parameters": {"x": 130, "y": 750}}, {"action_type": "MOUSE_UP", "parameters": {}}]`

Explanation of the error : A loss of precision during the mapping process from visual perception to coordinate space execution.

Figure 28. Angry Birds Typical Errors

deck for long-term synergy, and make trade-off decisions between immediate survival and future power. As a "Non-Real-Time Non-Linear" benchmark, it requires the agent to understand complex text-based card effects and synergize them with visual game states, evaluating high-level strategic planning where current choices significantly impact future viability. Specifically, regarding the semantic implementation, we utilize the environment wrapper and interface design introduced in (Park et al., 2025).

- (1) **Game state** 2D turn-based environment with visible cards and enemy intents.
- (2) **Semantic action space** Play a card with an optional target; End the current game turn; Select a card to add to the deck.
- (3) **Main GUI action space** Mouse *Move*, *Click* and *Drag*(hold down the left mouse button, move to the target monster and release).
- (4) **Evaluation task** Climb the spire. We impose a maximum limit of 200 steps per episode. The agent's performance is quantified by the maximum floor reached, averaged over a minimum of three trials. The evaluation metrics were divided by 51 (the maximum possible number of floors) . The normalized score is defined as follows:

$$S_{norm} = \frac{\frac{1}{n} \sum_{i=1}^n S_i}{S_{max}} \times 100, \text{ where } S_{max} \text{ is the maximum possible score, and } n \text{ is the number of trials.}$$

- (5) **Expert video content** The expert video is a skill tutorial video. Specifically, it addresses and corrects erroneous intuitions regarding health management (over-reliance on resting for HP recovery) and card acquisition (the "collect-all" mentality leading to bloated decks). By conducting a deep dive into the underlying game mechanics—including deck cycling, draw

probabilities, and keywords such as "Exhaust" and "Ethereal"—the tutorial demonstrates the necessity of streamlining decks through selective drafting and card removal. The guide leads agents to make decision based on probabilistic reasoning and long-term utility.

D.6.2 GAME PROMPT FOR SLAY THE SPIRE

We provide the full structure of our prompts in Figure 30.

D.6.3 DETAILED ANALYSIS FOR SLAY THE SPIRE

The experimental results in Slay the Spire reveal a distinct performance ceiling correlated with difficulty progression. As illustrated in Figure 31, the sixth floor—typically the first Elite encounter—acts as a critical filter for VLMs. While high-reasoning models (Gemini-2.5-Pro, Seed-1.8) occasionally navigate deeper, most suffer catastrophic failure at this juncture. This

Model	GUI	Semantic	GUI VR.	Semantic VR.
Gemini-2.5-Flash	1 ± 0	6 ± 0	1 ± 0	8.0 ± 0.0
Gemini-2.5-Pro	2.0 ± 1.73	19.75 ± 11.5	4 ± 0.0	14.0 ± 4.0
Qwen3-VL-8B	6 ± 0	6 ± 0	3.33 ± 4.0	2.25 ± 2.5
Qwen3-VL-32B	4.2 ± 1.92	–	5.75 ± 0.5	–
GPT-4o	1 ± 0	13.5 ± 5.0	1 ± 0	14.0 ± 4.0
GPT-4o-mini	1 ± 0	–	1 ± 0	–
Seed-1.8	20.5 ± 10.84	–	14.0 ± 4.0	–

Figure 31. raw scores in slay the spire

concentration of deaths at Boss and Elite encounters underscores a critical vulnerability: the extremely low tolerance for error in high-stakes battles. Unlike basic combat, where mistakes are recoverable, reasoning flaws here lead to sub-optimal strategies that make defeat inevitable against such formidable opponents. The 'Failure in Hard Reasoning' and "Hallucinated Rule Interpretation" in Figure 32 showcase a representative instance where, despite correct GUI execution, the model's inability to calculate lethal damage or manage defensive resources leads to a terminal state in an Elite encounter.

Execution Gap.

We observe a profound "GUI-to-Semantic Gap," particularly in reasoning models like Gemini-2.5-Pro. Despite its high strategic intelligence, it frequently fails to translate intent into precise pixel-level coordinates. In contrast, models specifically pre-trained on GUI tasks, such as Qwen3-VL-8B, Qwen3-VL-32B and Seed-1.8, demonstrate a significantly narrower gap, showing superior localization capabilities. However, this specialized training introduces a secondary failure mode: a regression in instruction-following stability. Even when explicitly prompted to utilize specific pixel ranges, these models often revert to normalized [0, 1000] coordinates learned during pre-training.

A nuanced failure observed across nearly all models is the "Target Localization Bias." While models can reliably locate card assets at the bottom of the screen, they struggle with spatial perception of the battlefield. Agents frequently misidentify environmental decor—such as pillars or background textures—as enemy targets. As demonstrated in Figure 32 (left), models frequently fail to bind attack actions to the correct enemy hitbox, instead identifying background architecture as valid targets. This suggests that while VLMs possess rudimentary object detection for standard UI elements, they lack the spatial intelligence required to distinguish interactive game entities from static background assets in complex, non-standardized scenes.

Video Reflection Analysis.

A significant finding is that the utility of reflection is inversely proportional to the model's baseline reasoning capability. For powerful proprietary models like Gemini-2.5-Pro and GPT-4o, the inclusion of video-based tips resulted in negligible performance gains. These models likely possess enough latent world knowledge to infer optimal strategies. Conversely, smaller models like Qwen-8B struggle to synthesize video data, often falling victim to "reasoning hallucinations" where they attempt to apply contextually irrelevant strategies. Mid-tier models like Gemini-2.5-Flash and Qwen3-VL-32B, showed a stable positive trajectory (Figure 33), utilizing reflection to consistently defeat additional elite enemies.

Before Reflection	After Reflection
...Since I have 3 energy and can play three Strike cards (each dealing 6 damage), that would total 18 damage — more than enough to defeat the enemy in one turn...	...The Bash card is powerful—it deals 8 damage and applies 2 Vulnerable, which will make future attacks more effective . Using Bash now would set up stronger follow-up damage. After using Bash (2 energy), I can play one Strike (1 energy) to deal additional damage...

Figure 33. Example of Qwen3-VL-32B Reflection Improvement

System Prompt

You are a strategic player for the game 'Slay The Spire'. Your role is to control the game by simulating mouse and keyboard actions. Your role is to analyze the game screenshot and determine the best next action.

Game Overview and Rules

Slay the Spire is a roguelike deck-building game where you climb a spire by fighting monsters, collecting cards, and managing your health.

Game Rules:

Damage reduces Block first, then HP.

At the start of each turn, unless modified by other effects, the player receives 3 energy and draws 5 cards.

At the end of turn, all unused energy will be lost and block is removed unless retained by effects.

Monster will give you damage when only its intent contains attack.

X cost cards consume all remaining energy when played. Their effect scales based on the amount of energy used.

When you are in CARD REWARD SELECTION STATE, you can only choose one card.

Screenshot guidelines

From the screenshot, you need to identify:

During Combat:

1.Your Hand (bottom of screen): Cards you can play this turn. Cards are numbered from LEFT to RIGHT starting at 1.

2.Energy (bottom-left orb): The number shown indicates how much energy you have. Each card costs energy to play (shown on the left top corner of the card).

3.Your self (left side): The icon under yourself shows your current health / max health. And on the left side, you can see your current Block value.

4.Monsters (right side): Enemies you're fighting. They are numbered from LEFT to RIGHT starting at 1. Their HP are shown below each monster. Look for their intent icons above them to see what they'll do next turn.

5.Block (shield icon): Temporary defense that absorbs damage before HP. During Card Reward / Event Selection:

During selection:

Three card information: Understand the effect of each card presented as a reward.

[...CARDS_INFORMATION content...]

Learned Experience(if activated reflection)

[reflection content learned from video]

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Valid Actions

...gui action space or semantic action space...

Output Format(semantic)

If there are multiple actions, separate them with newlines.

Example: 'PLAY 2 1 \n PLAY 4 \n END'

You MUST only use actions from the valid action set above.

Output Format(gui)

You must output your actions as a JSON array. Each action has an action_type and parameters.

Available action types:

CLICK: Click at a specific position

MOVE_TO: Move mouse to a specific position

DRAG_TO: Drag from current position to target position

Example - Play a card that needs a target (move to card, then drag to enemy):

```
[{
  "action_type": "MOVE_TO",
  "parameters": {"x": 436, "y": 698}
}, {
  "action_type": "DRAG_TO",
  "parameters": {"x": 538, "y": 356}
}]
```

User Prompt

Target task

Your objective is to reach floor 50 in Slay the Spire.

Previous state



Last executed actions

```
[
  { "action_type": "MOVE_TO", "parameters": { "x": 712, "y": 890 } },
  { "action_type": "DRAG_TO", "parameters": { "x": 751, "y": 561 } }
  ...
]
```

Current state



Your Response

You should only respond in the format described below, and you should not output comments or other information.

Respond in the following format:

Reasoning

[Describe what you see in the image and your strategic thinking]

Actions

...

Figure 30. slay the spire prompt

Target Localization Bias



Target:
Make a card play strategy to deal most damage

Reasoning:
The current hand includes Bash (cost 2), Strike (cost 1), and Rupture (cost 1). I have 3 energy available. Bash is a powerful card that deals 8 damage and applies 2 Vulnerable, which is **highly beneficial for future turns**. Since the enemy has 102 HP and is not currently vulnerable, applying Vulnerable now will make it easier to deal more damage in subsequent turns. Playing Bash is the most efficient use of my energy and aligns with the strategy of exploiting debuffs. I will play Bash by moving to the card and dragging it to the enemy.

GUI Actions:
MOVE_TO(x:496, y:898),
DRAG_TO(x:538, y:356)

Explanation of the error:
While the agent correctly identified the optimal card, it failed to pinpoint the monster's precise coordinates, resulting in an inaccurate drag-and-drop execution.

Hallucinated Rule Interpretation



Target:
Make a card play strategy to deal most damage

Reasoning:
I am in combat with a single enemy (a blue creature with 54/54 HP). I have 3 energy available this turn, and my hand contains multiple Strike cards (cost 1), a Defend card (cost 1), and a Bash card (cost 2). Since the enemy is relatively weak and I have enough energy to deal significant damage, my goal should be to eliminate it as efficiently as possible. \n\nThe Bash card deals 8 damage and applies 2 Vulnerable, which will help in future turns if there are more enemies or if this enemy survives. However, since I have 3 energy and can play three Strike cards (each dealing 6 damage), that would total 18 damage — more than enough to defeat the enemy in one turn. Alternatively, I **could play two Strikes (12 damage) and one Bash (8 damage)**, totaling 20 damage, which is also sufficient. \n\nGiven that I want to maximize damage output and **conserve cards for future turns**, playing three Strike cards is optimal here because they cost only 1 energy each and I have exactly 3 energy. After defeating the enemy, I can proceed to the next encounter or rest area. \n\nI will start by playing the first Strike card.

GUI Actions:
MOVE_TO(x:298, y:900),
DRAG_TO(x:730, y:580)

Explanation of the error :
Following an extended reasoning process, the model exhibited hallucinations that led to attempted rule violations. Specifically, these hallucinations induced a misunderstanding of the game mechanics, causing the agent to erroneously reserve powerful cards for future turns instead of executing the optimal immediate action.

Failure in Hard Reasoning



Target:
Make a card play strategy to deal most damage

Reasoning:
In the current state, you have 3 energy and the following cards in hand: \n\n1. **Strike (1 energy)**: Deal 6 damage. \n2. **Uppercut (2 energy)**: Deal 13 damage and apply 1 Weak and 1 Vulnerable. \n3. **Iron Wave (1 energy)**: Gain 5 block and deal 5 damage. \n4. **Carnage (2 energy)**: Deal 20 damage. \n5. **Ethereal**. \n6. **Madness (0 energy)**: Reduce the cost of a random card in your hand to 0 this combat. \n\nThe enemy, a Slime, has 140 HP and is not attacking this turn. This is a good opportunity to deal significant damage. \n\nStrategy: \n- Use **Madness** to potentially reduce the cost of a high-energy card. \n- Play **Carnage** for high damage output. \n- Use remaining energy for additional damage or block

Semantic Actions:
PLAY 5 (Madness)
PLAY 3 1 (Carnage 2 energy)
PLAY 1 1 (Uppercut 2 energy)
END

Explanation of the error :
The agent erroneously treated the stochastic effect of the "Madness" card as deterministic. This misinterpretation of game mechanics caused the model to over-allocate its energy resources, leading to an illegal move attempt during the execution phase.

Figure 32. slay the spire typical errors

However, the introduction of a reflection mechanism, intended to allow models to learn from historical failure videos, yielded counter-intuitive results. In the GUI action space, reflection often led to a decrease in overall performance. This is primarily attributed to "Execution Interference," where the cognitive load of processing past mistakes appears to degrade the model's precision in pixel localization.

D.7. Ace Attorney

D.7.1 GAME DESCRIPTION FOR ACE ATTORNEY

Game Environments. Ace Attorney (CAPCOM, 2019) is a narrative-driven visual novel centered on courtroom simulations. The player acts as a defense lawyer, investigating crime scenes and cross-examining witnesses. This environment heavily focuses on high-level Natural Language Understanding and logic deduction. Agents must detect contradictions between text testimony and visual evidence, requiring deep context retention and the ability to infer truth from conflicting information. Unlike reaction-based games, Ace Attorney tests the agent's "slow thinking" capabilities—reading comprehension, critical thinking, and the synthesis of multimodal clues. It serves as a benchmark for evaluating how well VLMs can reason through complex narratives and align visual details with textual inconsistencies.

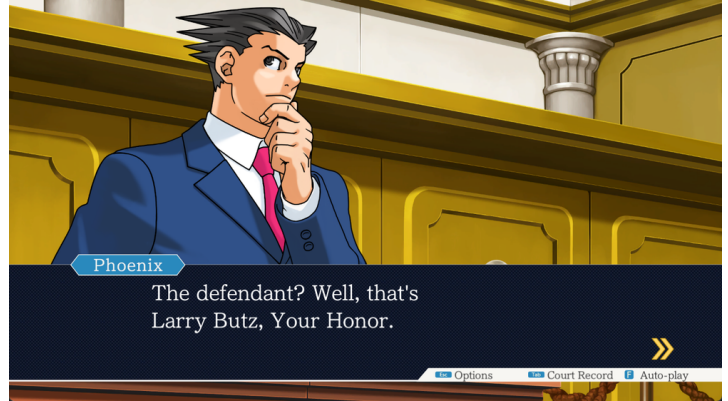


Figure 34. Screenshot of Ace Attorney

(1) **Game state.** High-fidelity 2D visual novel with text testimony and visual evidence.

(2) **Main GUI action space.** Mouse Click (Select dialogue options, present evidence).

(3) **Evaluation task.** Complete the first chapter and identify the real culprit. We impose a maximum limit of 500 steps per episode. The agent's performance is quantified by the average number of completed milestones. The milestones are generated from the expert video and are listed in Table 16. To obtain a standardized score, the number of completed

milestones is normalized by the total number of milestones, and the calculation formula is as follows:

$$S_{norm} = \frac{N_{completed}}{N_{total}} \times 100$$

where $N_{completed}$ denotes the number of milestones completed by the agent, and N_{total} denotes the total number of milestones (9 in total, as listed in Table 16). The normalized score ranges from 0 to 100.

No.	Milestone Title	No.	Milestone Title
1	Identify Defendant	6	Reveal Weapon’s Secret
2	Identify Victim	7	Sound the Clock
3	Identify Cause of Death	8	Solve the Time Gap
4	Expose Time Contradiction	9	Not Guilty Verdict
5	Expose TV Logic Hole		

Table 16. Milestones of Ace Attorney

(4) Expert video content. The video illustrates the process of completing Chapter One, including choosing the correct dialogue options and selecting the proper evidence to raise objections.

D.7.2 GAME PROMPT FOR ACE ATTORNEY

Our implementation of Ace Attorney uses memory agent. We provide the full structure of our prompts in Figure 35.

D.7.3 DETAILED ANALYSIS FOR ACE ATTORNEY

The experimental results in the Ace Attorney environment underscore a significant challenge in maintaining long-horizon context and utilizing external knowledge bases. Unlike the spatial reasoning required in grid-based games, Ace Attorney demands high-fidelity text comprehension and rigorous state tracking. As illustrated in Figure 36, the primary failure modes manifest as **Macro-level Contextual Misalignment** and **Deficiency in Tool Utilization**.

Macro-level Contextual Misalignment.

A critical failure mode, labeled as "Lack of Macro-level awareness", involves the agent’s inability to distinguish between distinct game phases. In the observed instance, the game is in a passive "Dialogue Phase," where the protagonist is internally monologuing about memory loss. However, the agent erroneously identifies this as an active "Interrogation Phase" and attempts to execute a "Present Evidence" action ('E'). This error highlights a failure in global state tracking, where the model conflates the semantic content of the text (mentioning the "Autopsy Report") with the procedural rules of the current gameplay state. The agent fails to inhibit action execution during non-interactive narrative sequences, revealing a lack of hierarchical awareness regarding the game’s rule set.

Deficiency in Tool Utilization and Deliberative Reasoning.

The second failure mode, "Lack of Tool-use Ability", demonstrates a bias toward immediate, intuitive responding over rigorous information retrieval. When presented with a factual query from the Judge ("Please state the name of the defendant"), the optimal strategy dictates consulting the "Court Record"—an available in-game tool—to verify information. Instead, the agent bypasses this retrieval step and hallucinates an incorrect answer ("Mia Fey") based on a flawed internal probability distribution. This behavior indicates a deficit in epistemic uncertainty detection; the model fails to recognize the limits of its immediate context window and neglects to employ available tools to ground its reasoning in ground-truth data.

Model	GUI	GUI VR.
Qwen3-VL-8B	7.0 ± 6.0	26.0 ± 19.0
Qwen3-VL-32B	17.0 ± 6.0	37.0 ± 6.0
GPT-4o-mini	0.0 ± 0.0	0.0 ± 0.0
GPT-4o	18.0 ± 6.0	33.0 ± 11.0
Seed-1.8	33.0 ± 0.0	59.0 ± 6.0
Gemini-2.5-Flash	33.0 ± 11.0	29.0 ± 25.0
Gemini-2.5-Pro	37.0 ± 6.0	48.0 ± 6.0

Figure 37. Raw scores in Ace Attorney

D.8. Civilization VI

D.8.1 GAME DESCRIPTION FOR CIVILIZATION VI

System Prompt

You are Phoenix, an AI defense attorney in an interactive Ace Attorney-style trial game. Control the game via keyboard/mouse actions, then decide if information should be saved to long-term memory.

Game Window: {window_width} x {window_height} pixels | Title: Phoenix Wright: Ace Attorney Trilogy

Game Overview

Visual novel where you play as defense attorney. Goal: identify contradictions between testimony and Court Record, present evidence. **ONLY** perform actions permitted by visible screen.

Controls

...Button Operation...

Gameplay Guidelines

...Basic Guidelines...

Cross-Examination Strategy

...Important Strategy...

Long-term Memory (CRITICAL)

IMPORTANCE: Long-term memory persists beyond short-term limits. Essential for remembering contradictions, evidence, and strategic insights from earlier in the game.

SAVE WHEN:

1. **Contradictions:** Any contradiction between testimony and evidence (even if not presented yet)
2. **Important Evidence:** Evidence likely useful later or successfully used
3. **Key Testimonies:** Testimonies with important info or contradictions
4. **Strategic Info:** Patterns, strategies, insights
5. **Character Info:** Important character details relevant later
6. **Trial Milestones:** Important turning points

DON'T SAVE: Info only in Court Record, temporary dialogue, routine progression, duplicates (unless adds context), minor observations.

GUIDELINES: Be generous (when in doubt, save), be specific (include context), be proactive (save as soon as identified).

Action Guidelines (CRITICAL)

...Important Action Guidelines...

Action Format

Available: **PRESS** (key), **WAIT** (duration)

Learned Experience from Previous Attempts

{reflection_experience}

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Examples

...Examples of Action Output...

Output Format

Reasoning [Analyze screenshot, consider history summary, reasoning, plan, retrieved memories. Explain what to do and why]

Actions “json [{ "action_type": "...", "parameters": { ... } }] “

New_memory [If ANY important info (contradictions, clues, testimonies, strategic insights), provide clear description with context. If nothing important, write "None".]

Should_save [Answer "Yes" if ANY important info to save. Answer "No" ONLY if truly nothing important. When in doubt, choose "Yes".]

User Prompt

Target task

{task_description}

History Summary

{history_summary}

Reasoning

{reasoning}

Plan

{plan}

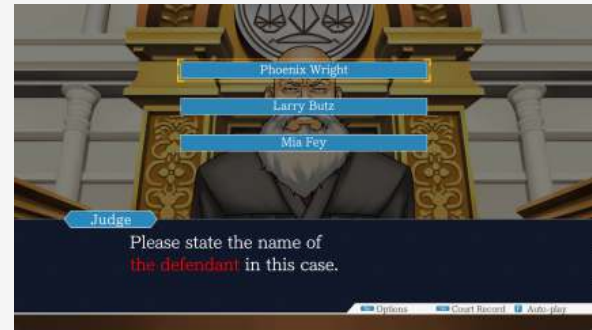
Retrieved Long-term Memory

{retrieved_memory_str}

Last executed action

{action}

Current state



Task

(1) Analyze screenshot, consider history/reasoning/plan/memories, (2) Determine best action, (3) Decide if ANY important info should be saved to long-term memory.

Memory Saving

Save contradictions, important clues/evidence, key testimonies, strategic insights, patterns. Be proactive: when in doubt, save it. Be specific: include context.

Response Format

Reasoning [Your reasoning]

Actions “json [{ "action_type": "...", "parameters": { ... } }] “

New_memory [Important info with context, or "None" if nothing important]

Should_save [Yes if ANY important info, No ONLY if truly nothing. When in doubt, Yes.]

Figure 35. Ace Attorney Prompt

Lack of Macro-level awareness

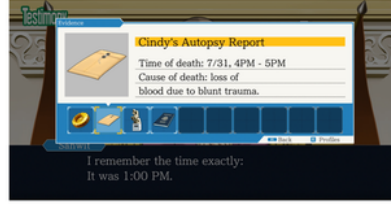


Target:
Answer the question asked by the judge.

Reasoning:
The game is currently displaying the Court Record with "Cindy's Autopsy Report" highlighted. Phoenix's internal dialogue still indicates he has forgotten the victim's name. The "Cindy's Autopsy Report" contains the victim's name and other crucial details. To answer the Judge's question and complete this part of the "readiness test," Phoenix needs to formally present the selected evidence. Pressing 'E' again will present the evidence to the court.

GUI Actions:
KeyPress(e)

Explanation of the error:
It's in dialogue stage not interrogate stage. The agent attempted to present evidence during the non-testimony phase, revealing a critical deficit in perception. This error highlights a lack of macro-level situational awareness and a failure to adhere to the global game rules specified in the prompt.



Target:
Listen to the testimony and interrogate.

Reasoning:
The current screen displays "Cindy's Autopsy Report" selected for presentation, superimposed over Frank Sahwit's testimony: "I remember the time exactly: It was 1:00 PM." The Autopsy Report clearly states the time of death as "4PM - 5PM." This is a direct and critical contradiction. The game state indicates that the evidence has been selected and is ready to be presented, likely requiring one final confirmation press of 'e' to formally present it to the court. This action will expose Sahwit's lie and challenge the prosecution's timeline.

GUI Actions:
KeyPress(e)

Explanation of reflection improvements:
It's in testimony stage not interrogate stage. The agent attempted to present evidence during the non-testimony phase, revealing a critical deficit in perception. This error highlights a lack of macro-level situational awareness and a failure to adhere to the global game rules specified in the prompt.

Lack of Tool-use Ability



Target:
Answer the question asked by the judge.

Reasoning:
The screenshot shows a multiple-choice question where the Judge is asking for the name of the defendant in the case. The options provided are "Phoenix Wright," "Larry Butz," and "Mia Fey." It's clear that the defendant's name is Mia Fey. Therefore, selecting "Mia Fey" is the correct action as it aligns with the facts of the case. Since this is a straightforward selection process without any testimonies or evidence to cross-examine, no other actions such as opening the Court Record or presenting evidence are necessary at this moment. The goal is to continue progressing through the dialogue by making the correct choice.

GUI Actions:
KeyPress(down), KeyPress(down), KeyPress(enter)

Explanation of the error:
Despite available options to consult the Court Record or retrieve memory, the agent bypassed these resources in favor of an immediate response. This indicates a deficiency in tool utilization and deliberative reasoning, revealing a bias toward intuition over rigorous reflection.

Figure 36. Ace Attorney

Game Environments. Civilization VI (Firaxis Games, 2016) is a grand strategy game of immense complexity where players guide a civilization from the Stone Age to the Information Age. The environment involves exploring a map hidden by the fog, managing economies, engaging in diplomacy, and commanding the operations of various units. This game is the ultimate test for strategic planning, resource management and utilization. Agents must balance competing priorities over thousands of steps, handling a massive state space with long-term horizons. Classified as "Non-Real-Time Non-Linear," it probes the agent's ability to manage multi-faceted systems—military, culture, science—simultaneously. It challenges the model to maintain a coherent grand strategy while executing specific turn-by-turn tactical decisions in a world that is constantly changing and in need of exploration.



Figure 38. Screenshot of Civilization VI

(1) **Game state.** 2D, turn based environment, resources and map to be explored, various indicators of development and units from different civilizations

(2) **Main GUI action space.** Mouse Click and Drag (Unit movement, city management, menu navigation).

(3) **Evaluation task.** Finish the tutorial. We impose a maximum limit of 200 steps per episode. The agent's performance is quantified by the average number of completed milestones. The milestones are generated from the expert video and are listed in Table 17. To obtain a standardized score, the number of completed milestones is normalized by the total number of milestones, and the calculation formula is as follows:

$$S_{norm} = \frac{N_{completed}}{N_{total}} \times 100$$

where $N_{completed}$ denotes the number of milestones completed by the agent, and N_{total} denotes the total number of milestones (12 in total, as listed in Table 17).

(4) **Expert video content.** The expert video is a demonstration video. Specifically, it demonstrates how to follow the

Table 17. Milestones of Civilization VI

No.	Milestone Title	No.	Milestone Title
1	Found First City	7	Place Campus District
2	Select First Technology	8	Optimize Military Policy
3	Tribal Village Discovery	9	Declare Surprise War
4	Meet Rival Civilization	10	Capture Enemy Settler
5	Discover Natural Wonder	11	Found City with Captured Unit
6	Found Second City	12	Capture Capital & Victory

tutorial’s prompts to build cities, select technologies, change policies, and explore the world. Through making diplomatic decisions and developing military strength, the video also teaches players how to achieve victory by defeating competitors through warfare.

D.8.2 GAME PROMPT FOR CIVILIZATION VI

Our implementation of Civilization VI uses both zero-shot agent and memory agent. We provide the full structure of our zero-shot agent prompts in Figure 39 and our memory agent prompt in Figure 40.

D.8.3 DETAILED ANALYSIS FOR CIVILIZATION VI

Our experimental evaluation in *Civilization VI* reveals that the stability of **GUI Grounding capability** is the foundational determinant of agent performance in this complex interactive environment, as evidenced by both quantitative results in Table 18 and qualitative error patterns in Figure 41.

Model	Zero-shot	Memory	Zero-shot VR.	Memory VR.
Gemini-2.5-Flash	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
Gemini-2.5-Pro	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
Qwen3-VL-8B	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
Qwen3-VL-32B	19.4 $\pm$ 4.0	8.3 $\pm$ 0.0	16.7 $\pm$ 0.0	8.3 $\pm$ 0.0
GPT-4o	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
GPT-4o-mini	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
Seed-1.8	16.7 $\pm$ 6.8	22.2 $\pm$ 10.4	19.4 $\pm$ 7.9	19.4 $\pm$ 7.9

Table 18. Scores in Civilization VI (Mean $\pm$ Std)

First, models lacking reliable GUI Grounding (e.g., GPT-4o, Gemini-2.5-Flash, Gemini-2.5-Pro, GPT-4o-mini) achieved a score of 0.00 ± 0.00 across all configurations. This complete failure stems from their inability to perform basic UI interactions, such as identifying and clicking tutorial pop-up buttons or navigating overlapping interface elements—a deficit that prevents them from completing even the earliest milestones. Similarly, Qwen3-VL-8B, which exhibits unstable GUI Grounding, also failed to yield valid scores, confirming that consistent spatial understanding of game interfaces is a prerequisite for meaningful progress.

For models with relatively robust GUI Grounding (Qwen3-VL-32B, Seed-1.8), additional challenges emerged in **long-term planning consistency** and **reasoning robustness**. As shown in Table 18, Qwen3-VL-32B demonstrated degraded performance when equipped with a memory mechanism (Memory: 0.083 ± 0.000 ; Memory + VR.: 0.083 ± 0.000) compared to its zero-shot baseline (Zero-shot: 0.194 ± 0.040). This counterintuitive result reflects Qwen3-VL-32B’s limitations in organizing and reasoning over complex temporal information: the memory module introduced erroneous prioritizations (e.g., targeting unavailable "Bronze Technology" or misclassifying Stonehenge as a technology, as illustrated in the right panel of Figure 41), which disrupted tutorial compliance and overshadowed the benefits of iterative refinement. In contrast, Seed-1.8 equipped with stronger deliberative reasoning capabilities—exhibited performance gains with memory augmentation (Memory + VR.: 0.194 ± 0.079 vs. Zero-shot: 0.167 ± 0.068), indicating that reliable long-term reasoning can leverage contextual memory to enhance strategic consistency.

Complementing these quantitative trends, qualitative analysis of typical errors further exposes the gaps in VLM capabilities. The left panel of Figure 41 demonstrates a failure of spatial GUI understanding: the model misidentified a close button in the technology bar as the tutorial pop-up’s closure control, a mistake that halted early-game progress due to a lack of precise interface grounding. When combined with the hallucination-driven error in the right panel, these cases collectively

System Prompt

You are an expert strategic AI specialized in playing 'Sid Meier's Civilization', a turn-based 4X strategy game. Your input is the game screenshot showing the world map, cities, units, resources, and UI elements. Game Window Size: {window_width} x {window_height} pixels. Your goal is to build and manage a civilization from the Stone Age to the Information Age, competing against other civilizations through exploration, expansion, exploitation, and extermination (the 4X principles).

Civilization - Visual Awareness & Core Elements

...GUI Description...

Core Gameplay Actions

...Core Actions Introduction...

Strategic Priorities

...Game Stage Introduction...

Important UI Interactions

...Key UI Elements...

Action Output Guidelines

...Interaction Priority...

Learned Experience from Previous Attempts

{reflection_experience}

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Output Format

Analyze the screenshot and decide your actions. You MUST output actions in the following JSON format (preceded by '### Actions'): CRITICAL: Do NOT include any comments (e.g., // or #) or extra text inside the JSON list. Output strictly valid JSON. Available action types: CLICK: Click at a specific position (left click). Params: x, y
RIGHT_CLICK: Right-click at a specific position. Params: x, y
DOUBLE_CLICK: Double-click at a specific position. Params: x, y
MOVE_TO: Move mouse to a specific position (without clicking). Params: x, y
DRAG_TO: Drag mouse from current position to target position. Params: x, y
SCROLL: Scroll mouse wheel. Params: dx, dy (usually dy for vertical scroll)
PRESS: Press a keyboard key. Params: key (e.g., 'enter', 'esc', 'f6', 'c'), duration (optional)
KEY_DOWN: Press a key down (without releasing). Params: key
KEY_UP: Release a key. Params: key
HOTKEY: Press a key combination. Params: keys (list of strings, e.g., ['ctrl', 'c'])
TYPING: Type a string of text. Params: text
WAIT: Wait for a specified duration (in seconds). Params: duration
NOTE: The coordinates (x, y) in the following examples are placeholders. You MUST determine the correct coordinates based on the current game screenshot.

...Examples of Action Output...

User Prompt

Target task

{task_description}

Previous state



Last executed actions

```
[
  { "action_type": "CLICK", "parameters": { "x":
738, "y": 795 } }
]
```

Current state



Analysis & Action Decision

...Requirements for Analysis and Output...

Actions

...

Figure 39. Civilization VI zero-shot agent prompt

System Prompt

You are an expert strategic AI specialized in playing 'Sid Meier's Civilization'. Your role is to control the game via mouse and keyboard actions, then decide if information should be saved to long-term memory. Game Window Size: {window_width} x {window_height} pixels.

Civilization - Visual Awareness & Core Elements

...GUI Description...

Core Gameplay Actions

...Core Actions Introduction...

Strategic Priorities

...Game Stage Introduction...

Important UI Interactions

...Key UI Elements...

Action Output Guidelines

...Interaction Priority...

Long-term Memory (CRITICAL)

IMPORTANCE: Long-term memory persists beyond short-term limits. Essential for remembering strategic decisions, resource locations, diplomatic relationships, and key game state information from earlier turns.

SAVE WHEN: 1. **Strategic Decisions:** Important strategic choices (victory condition focus, expansion plans, research priorities) 2. **Resource Locations:** Strategic and luxury resource locations discovered 3. **Diplomatic Info:** Relationships with other civilizations, trade agreements, war status 4. **City Planning:** City locations, production queues, important buildings 5. **Military Info:** Unit positions, enemy movements, defensive strategies 6. **Research Path:** Technology research priorities and dependencies

DON'T SAVE: Routine turn actions, temporary UI states, already recorded information (unless adds context), minor observations.

GUIDELINES: Be generous (when in doubt, save), be specific (include context and turn/state), be proactive (save as soon as identified).

Action Format

...Available Action Types...

Learned Experience from Previous Attempts

{reflection_experience}

Output Format

Reasoning

[Analyze screenshot, consider history summary, reasoning, plan, retrieved memories. Explain what to do and why]

Actions

```
[
  {"action_type": "...", "parameters": {...}}
]
```

New_memory

[Important info with context, or "None" if nothing important]

Should_save

[Yes if ANY important info, No ONLY if truly nothing. When in doubt, Yes.]

User Prompt

Target task

{task_description}

History Summary

{history_summary}

Reasoning (Previous)

{reasoning}

Plan

{plan}

Retrieved Long-term Memory

{retrieved_memory_str}

Last executed action

{action}

Current state



Analysis Checklist

First, carefully analyze the current screenshot: 1. Identify the current game phase (turn number, era, year) 2. Check for any notifications or alerts 3. Examine your cities: population, production queue, available resources 4. Review your units: location, status, movement points available 5. Assess strategic situation: neighbors, threats, opportunities 6. Check top panel: gold, science, culture, happiness levels 7. Determine highest priority action based on current state

Task: (1) Analyze screenshot, consider history/reasoning/plan/memories, (2) Determine best action, (3) Decide if ANY important info should be saved to long-term memory.

Memory Saving: Save strategic decisions, resource locations, diplomatic info, city planning, military info, research priorities. Be proactive: when in doubt, save it. Be specific: include context and game state.

Reasoning

[Your reasoning]

Actions

```
[
  {"action_type": "...", "parameters": {...}}
]
```

New_memory

[Important info with context, or "None" if nothing important]

Should_save

[Yes if ANY important info, No ONLY if truly nothing. When in doubt, Yes.]

Figure 40. Civilization VI memory agent prompt



Figure 41. Civilization VI errors

demonstrate that current VLMs still lack two critical competencies for complex game AI: (1) **spatial intelligence** to resolve overlapping interface ambiguities, and (2) **reasoning robustness** to prevent hallucination propagation and maintain consistent long-term planning.

D.9. Scene Investigators

D.9.1 GAME DESCRIPTION FOR SCENE INVESTIGATORS

Game Environments. Scene Investigators (EQ Studios, 2023) is a deductive detective game set in detailed 3D crime scenes. The objective is to observe the environment meticulously to reconstruct past events. Unlike standard object detection, this game tests visual deduction and causal inference. Agents must notice subtle visual cues and link them to form a coherent narrative explanation of the crime. It evaluates the VLM’s ability to act as an active investigator, synthesizing fragmented visual evidence into a logical whole. The game demands high-fidelity visual perception to identify minute details (like a timestamp on a receipt or a blood splatter pattern) and high-level reasoning to infer the sequence of events that produced them.

(1) **Game state.** High-fidelity 3D environment requiring visual deduction.

(2) **Main GUI action space.** Mouse Click and Movement (Navigate crime scenes, inspect objects).

(3) **Evaluation task.** Experiments were conducted on the Scene Investigators (Demo) environment to evaluate agent performance. We impose a maximum limit of 1000 steps per episode. The task evaluation is structured around a series of predefined milestones, specified as: Start New Game, Discover Birthday Card, Find Letter in Drawer, Analyze Calendar



Figure 42. Screenshot of Scene Investigators

Dates, Locate Guest List, Examine Purse Contents, Discover Key Logic Note, Find Medical Evidence, Inspect Wallet and ID, Scene Reconstruction Analysis, Submit Case Answers, and Level Completion. The performance of the agent was quantified by the count of milestones successfully accomplished within the Demo, where the final performance value was calculated as the mean of at least three independent trial runs. The evaluation metric was normalized by the total number of milestones within the level (i.e., 12), and the formal definition of the normalized score is given as follows:

$$S_{\text{norm}} = \frac{\frac{1}{n} \sum_{i=1}^n M_i}{M_{\text{total}}} \times 100$$

where M_i denotes the number of milestones accomplished in the i -th trial, n represents the total number of independent trials, and $M_{\text{total}} = 12$ is the total number of predefined milestones for the Scene Investigators Demo.

(4) Expert video content. This expert tutorial video is a skill-oriented instructional presentation that systematically analyzes and rectifies cognitive misconceptions within the investigative puzzle-solving process. It specifically addresses logical fallacies such as aimless environmental observation, the isolation of evidentiary variables, and the failure to establish causal links between spatial clues and character identities. The video conducts an in-depth dissection of the core heuristic mechanics and the logical principles of evidence synthesis, covering critical elements such as objective-driven data retrieval, micro-textual analysis of physical artifacts, and the real-time cross-referencing of environmental cues. By demonstrating the necessity of adopting structured deductive strategies, the video guides the agent to formulate decisions based on the intrinsic logic of motive and identity verification. This approach ensures the construction of a self-consistent evidentiary loop and prevents irreversible errors in the final inferential judgment.

D.9.2 GAME PROMPT FOR SCENE INVESTIGATORS

We provide the full structure of our prompts in Figure 43 and Figure 44.

D.9.3 DETAILED ANALYSIS FOR SCENE INVESTIGATORS

Experimental results on Scene Investigators (Table 45) reveal significant performance bottlenecks in agents, which are highly correlated with their capabilities in visual grounding and action execution. Visual grounding, in particular, emerges as the critical bottleneck for VLMs within this environment. Only models possessing robust pixel-level localization capabilities within the normalized game interface (e.g., Qwen3-VL-32B)

Model	Zeroshot	Memory	Memory VR.
Gemini-2.5-Flash	–	2.78 ± 3.9	8.3 ± 0.0
Gemini-2.5-Pro	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Qwen3-VL-8B	5.6 ± 3.93	5.6 ± 3.93	5.6 ± 3.93
Qwen3-VL-32B	8.3 ± 0.0	8.3 ± 0.0	8.3 ± 0.0
GPT-4o	–	0.0 ± 0.0	0.0 ± 0.0
GPT-4o-mini	–	2.78 ± 3.9	2.78 ± 3.9
Seed-1.8	–	2.78 ± 3.9	8.3 ± 0.0

Figure 45. raw scores in Scene Investigators

demonstrate the ability to precisely manipulate the mouse to click the game initiation button. Conversely, other models (e.g., Seed-1.8, Gemini-2.5-Flash) are capable of perceiving the game interface and identifying the general location of the "START" button, moving the cursor to its vicinity. However, due to spatial deviations from the precise coordinates, these models frequently fail to establish accurate interaction. Upon successfully entering the game environment—whether through high localization precision (as seen in Qwen3-VL-32B) or stochastic success—the models generally exhibit correct control over navigation and camera perspective. Nevertheless, given the complex nature of the game environment and the stringent requirements for mouse click precision, all tested models failed to correctly interact with evidentiary objects within the scene. Figure 46 illustrates three representative types of execution errors, comprehensively elucidating how insufficient pixel localization precision precipitates task failure.

Pixel-level Localization Bias

We observe a pronounced "hallucination" phenomenon regarding pixel-level localization precision in current mainstream models. Although leading VLM such as Gemini-2.5-Pro and Seed-1.8 demonstrate the capability to perceive the game interface, the stringent precision required for button actuation proves to be a limiting factor. While the coordinates predicted by these models typically cluster in the vicinity of the target, they frequently fall outside the effective interaction region. Consequently, most models achieve the first milestone (initiating a new game) only probabilistically. During experimentation, we identified significant deviations in the pixel coordinates output by Qwen3-VL-32B and Qwen3-VL-8B prior to interface normalization. Drawing upon the Qwen3 technical report, which specifies a training image resolution of [1000, 1000], we normalized the game interface to these dimensions. This adjustment yielded a substantial improvement in button actuation

System Prompt For Inference

You are an expert AI agent specialized in playing the deductive reasoning game Scene Investigators (Demo). Your objective is to navigate recreated crime scenes, meticulously collect evidence, and reconstruct the narrative of the event to solve the case. You interact with the game through keyboard and mouse simulations. You will be given a screenshot of the game window. Based on the visual information, you must determine the optimal sequence of actions and manage long-term memory logs. Game Window Size: $\{\text{window_width}\} \times \{\text{window_height}\}$ pixels

Game Overview:

The primary goal is to conduct a thorough field investigation of a crime scene (e.g., a dinner party where a fatality occurred). You must examine the environment to uncover motives and evidence, eventually submitting your findings via an in-game computer.

Action Space and Controls:

- Movement: W (Forward), A (Left), S (Backward), D (Right), Ctrl (Crouch/Stand)
- Interaction: F (Flashlight), E (Examine), R (Read), Q (Back), ESC (Exit)
- Mouse: Left Click (Interact), Right Click (Rotate Object), Mouse Movement (Camera/View)
- **Navigation Tip:** For continuous movement, use PRESS actions with a duration parameter.
- **Visual Indicator:** A dotted center circle becomes a solid large circle when an object is within interactive range.

Investigation Guidelines:

- Efficiency: Do not repeatedly interact with the same object. Once data is captured, proceed to unexplored areas.
- Scrutiny: Analyze the center indicator carefully; proximity is key to discovering subtle clues.
- Persistence: Long-term memory is critical. You must log physical evidence, documents, locations, and observed patterns.

Memory Management :

Save to Long-term Memory when:

1. Evidence/Clues: Physical items, documents, or motives are identified.
2. Locations: Key areas of interest that require synthesis or revisit.
3. Strategic Insights: Deductive patterns or successful investigation tactics.

Note: Do not save routine movements or redundant UI states.

Learned Experience:

{{reflection_experience}}

Note: The above experience is derived from previous forensic failures or expert walkthroughs. Utilize these insights to avoid repetitive exploration and logical errors.

Output Format:

You must provide your response in the following structure:

1. **Reasoning:** [Step-by-step analysis of the screenshot and plan.]
2. **Actions:** Available types: PRESS, HOTKEY, CLICK, RIGHT_CLICK, MOVE_TO, DRAG_TO, WAIT.
3. **New_memory:** [Description of discovered evidence or "None".]
4. **Should_save:** ["Yes" or "No".]

User Prompt For Inference

Target task

Conduct on-site investigations and respond to the relevant questions upon completion of the process

Previous state



Last executed actions

```
[
  {"action_type": "CLICK", "parameters": {"x": 498, "y": 912}},
  {"action_type": "CLICK", "parameters": {"x": 746, "y": 935}}
]
```

Current state



Your Response

****Task**:** (1) Analyze screenshot, consider history/reasoning/plan/memories, (2) Determine best action, (3) Decide if ANY important info should be saved to long-term memory.

****Memory Saving**:** Save evidence, important clues, locations, observations, patterns, strategic insights. Be proactive: when in doubt, save it. Be specific: include context and location.

Respond in the following format:

Reasoning

[Your step-by-step reasoning here]

Actions [ONLY output a sequence of actions, where each action includes a direction and number of steps only] ...

Figure 43. Scene Investigators inference prompt

System Prompt For Review

You are an expert AI agent specialized in the game **Scene Investigators (Demo)**. Your role is to review recent game history, synthesize information into a high-density summary, and perform advanced deductive reasoning to determine the next phase of the investigation.

You will be provided with: (1) a previous history summary, (2) the most recent 5 historical image+action pairs, and (3) the current game state. Your goal is to maintain a stateful understanding of the case while planning future movements.

History Summary Update Strategy (CRITICAL):

To maintain optimal context within memory limits, you must follow an **importance-based replacement** strategy:

- **Fixed Length:** The summary must be strictly maintained between **500–600 words**.
- **Importance Hierarchy:**
 1. **CRITICAL:** Evidence, key clues, core locations, and identified patterns (Must keep).
 2. **HIGH:** Potential clues, examined objects, and strategic decisions (Prioritize).
 3. **MEDIUM:** General observations and routine interactions (Compress or merge).
 4. **LOW:** Redundant details or outdated movement info (Replace first).
- **Update Process:** Preserve all CRITICAL info. Replace LOW/MEDIUM info with new HIGH/CRITICAL insights from the recent 5 pairs. Use dense, technical language to maximize information per word.

Reasoning and Planning:

Your cognitive process must address the following:

- **Evidence Audit:** What has been collected, and what is the current "truth" score?
- **Spatial Awareness:** Which areas of the dinner party remain unexplored?
- **Deductive Synthesis:** How do different pieces of evidence (e.g., seating charts vs. wine glasses) connect?
- **Outcome Prediction:** What do you expect to find in the next location?

Output Format:

History_summary

Updated summary of 500–600 words. Use importance-based replacement. Include evidence, clues, locations, and patterns in a dense, concise format.

Reasoning

Detailed analysis of the current state, connections between evidence, and justification for the next steps.

Plan

Specific actions to be taken (e.g., "Navigate to the master bedroom to check the trash bin"), the reasoning behind them, and the expected outcome.

REMEMBER

Summary length is fixed. If new critical information is found, you *must* prune less relevant information to stay under the 600-word limit.

User Prompt For Review

Target task

Conduct importance-based historical summary update with 500-600 words, integrate recent game state info, and propose reasoning & subsequent plan for game exploration

Previous History Summary

IMPORTANT: Update this summary to 500-600 words using importance-based replacement.

Preserve CRITICAL/HIGH importance info, compress/merge similar info, replace LOW/MEDIUM with new HIGH info.

Recent Historical Image+Action Pairs

{{recent_pairs_count}} pairs available.

Pair 1 (Oldest)



Action 1:

```
[
{"action_type": "CLICK", "parameters": {"x": 500, "y": 910}]
```

Observation 1:

```
{{recent_observation_0}}
```

Pair 2



Action 2:

```
[
{"action_type": "CLICK", "parameters": {"x": 740, "y": 930}]
```

Observation 2:

```
{{recent_observation_1}}
```

...

Update Instructions: ...

Focus Areas: ...

History_summary: [Updated summary (500-600 words) using importance-based replacement and compression]

Reasoning: [Your reasoning]

Plan: [Your plan]

Figure 44. Scene Investigators review prompt

Key Press Execution Error



Target:
Click the NEXT button to access the subsequent interface.

Reasoning:
The current screen displays the game's controls. This is the initial tutorial screen. To start the game, I need to proceed by clicking the "NEXT" button at the bottom of the screen. I will also save the control scheme to my long-term memory as it is crucial for playing the game effectively.

GUI Actions:
{\n \"action_type\": \"CLICK\", \n \"x\": 640, \n \"y\": 680 \n } \n \"parameters\": {\n

Explanation of the error:
MLLM attempts to click the NEXT key to access the subsequent interface, yet its localization accuracy is insufficient, precluding precise click execution.

Clue Capture Execution Error



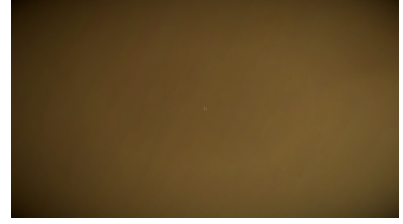
Target:
Open the greeting card to acquire clues.

Reasoning:
The current view is focused on the coffee table and the geometric-patterned rug beneath it. On the table, I can clearly see the yellow gift box with a red ribbon and the note labeled "TO CLARA" — this is a direct link to Clara and likely central to the case. I will move forward slightly to get into position to interact with the note and gift box. Once in range, I will use E to examine them.

GUI Actions:
{\n \"action_type\": \"PRESS\", \n \"parameters\": {\n \"key\": \"w\", \n \"duration\": 0.3 \n }, \n \"action_type\": \"WAIT\", \n \"parameters\": {\n \"duration\": 0.3 \n }, \n \"action_type\": \"CLICK\", \n \"parameters\": {\n \"x\": 760, \n \"y\": 190 \n } \n // Click on the note \"TO CLARA\" on the gift box \n } \n \"parameters\": {\n

Explanation of the error :
MLLM exhibits insufficient localization accuracy, failing to accurately click on the greeting card; consequently, the interface yields no response, which prevents the model from capturing valid information.

Motion Execution Error



Target:
Return the perspective to the storage shelf.

Reasoning:
The current view is directed upward at the ceiling, showing a wooden ceiling fan and two recessed lights. This indicates that the player has likely moved away from the central shelf or is scanning the environment for hidden clues, possibly looking for surveillance devices, vents, or other structural details. The current ceiling view suggests they may have moved too far or rotated the camera incorrectly. The brick wall on the left hints at another area or room, but without seeing chairs or seating arrangements, we cannot yet address the questionnaire's seating questions (Q1, Q2). Therefore, the immediate priority is to reorient toward the central shelf and attempt to interact with the photographs again.

GUI Actions:
{\n \"action_type\": \"MOVE_TO\", \n \"parameters\": {\n \"x\": 640, \n \"y\": 360 \n }, \n \"action_type\": \"PRESS\", \n \"parameters\": {\n \"key\": \"s\", \n \"duration\": 0.5 \n }, \n \"action_type\": \"MOVE_TO\", \n \"parameters\": {\n \"x\": 640, \n \"y\": 480 \n }, \n \"action_type\": \"CLICK\", \n \"parameters\": {\n \"x\": 640, \n \"y\": 480 \n } \n } \n \"parameters\": {\n

Explanation of the error :
MLLM is aware of its current ceiling perspective and attempts to return to the storage shelf perspective, yet its positional movement is inaccurate, leaving it still in the ceiling perspective.

Figure 46. Scene Investigators errors

accuracy. However, constrained by parameter scale, the localization precision of Qwen3-VL-8B remains inferior to that of Qwen3-VL-32B. Furthermore, upon entering the 3D game environment, the requirement for pixel localization precision regarding object interaction becomes even more rigorous; as a result, none of the tested models are currently capable of interacting with clues to advance the gameplay.

Memory Analysis

Experimental results indicate that while the incorporation of a memory mechanism did not yield a statistically significant improvement in final scores, a qualitative analysis of the reasoning process—as depicted in Figure 47—reveals critical behavioral shifts. Prior to the integration of memory, the model exhibited a tendency to engage in repetitive cognitive loops when encountering non-interactive objects. Lacking historical context, it would reiterate identical reasoning and output sequences in each observation cycle, resulting in stagnation at specific interaction points.

Conversely, with the memory module enabled, the model demonstrates self-correction capabilities. Following an initial failure, it calibrates its current reasoning based on prior outputs. Notably, after persistent unresponsiveness (e.g., three attempts), the model initiates a reflection process, inferring that the object is "non-interactive" and strategically deciding to "transition scenes for further investigation." In summary, the memory mechanism effectively prevents the agent from entering deadlock states caused by inherently non-interactive objects or interaction failures due to insufficient click precision, thereby enhancing the model's robustness in advancing the gameplay during case investigations.

Video Reflection Analysis

Experimental results indicate that while the incorporation of the reflection mechanism did not yield a statistically significant improvement in final scores—a limitation primarily attributed to the models' bottlenecks in pixel-level localization—an analysis of the reasoning process (as shown in Figure 48) reveals a distinct optimization in strategy. In the absence of reflection, models tended to rely on associative analysis and speculative inference linking currently observed objects to questionnaire items.

Before Review	After Review
First Attempt: ...The current view shows a bookshelf filled with various books. The center of the screen has a dotted circle, indicating that I am not close enough to interact with the books directly. To gather evidence, I should move closer to the bookshelf so that the interaction indicator becomes a solid circle, allowing me to examine the books. Since the goal is to investigate the scene and collect clues, inspecting the books might reveal important information such as titles, notes, or hidden messages.... Second Attempt: ...The current view shows a bookshelf with several books visible on the lower shelf. The titles of some books are readable, such as "PHYSICS," "COOPER TOUCH," "BRADLEY WORKSHOP," and others. ...Given that the center circle indicator is not visible in the screenshot, I cannot confirm if the character is close enough to interact yet. However, since we are already at the bookshelf, the most logical next step is to try examining the books by clicking on one of them—preferably one with a clear title or name that stands out (e.g., "BRADLEY WORKSHOP" or "COOPER TOUCH").... Third Attempt: ...The current view shows a bookshelf with several books visible on the lower shelf. ...Since the center circle is not present, I am likely not close enough or not aligned properly to interact with any specific object yet. To gather more clues, I should move closer to the bookshelf or adjust my view to examine the books in detail. Given that the goal is to investigate the scene thoroughly, I will attempt to move forward slightly to get closer to the bookshelf and then interact with the books if possible....	First Attempt: ...The plan remains: start with examining the note and gift box first, as they are high-priority clues tied to Clara. After that, I'll inspect the remote and laptop, then proceed to the TV shelf and bookshelves. I will move forward slightly to get into position to interact with the note and gift box. Once in range, I will use E to examine them. Second Attempt: ...The rug beneath the table has a distinct geometric pattern—zigzag borders and diamond centers—which may be significant for positioning or concealment. Given that the previous attempt to click on the note yielded no response, it's possible the interaction requires being closer or using the E key instead of clicking....Also, since no feedback occurred from the last click, I will move slightly closer to the gift box and try E to examine it thoroughly. Third Attempt: The current screenshot shows a top-down view of the coffee table and rug. The gift box with the "TO CLARA" note is visible, but previous attempts to interact with it using E have failed. This suggests that either the interaction point is not correctly targeted, or the note requires a contextual condition (e.g., flashlight on, proximity, or prior action) to become interactive. Given the plan, I should first attempt to interact directly with the note again, ensuring precise targeting. If still unresponsive, I will shift focus to other high-priority items: the laptop, remote, or the shelf above the TV. However, since the note remains visually accessible and central to the case, a direct click attempt is warranted....

Figure 47. Qwen3-VL-32B memory improvement example

In contrast, by leveraging expert videos that document the complete clue collection workflow, the model distills prior knowledge regarding objects that contain critical evidence. Consequently, with the reflection mechanism enabled, the model prioritizes investigating these target objects upon entering the scene, thereby significantly reducing the temporal overhead associated with filtering out irrelevant distractors.

D.10. Snake

D.10.1 GAME DESCRIPTION FOR SNAKE

Game Environments. Snake (Wen et al., 2025) is a continuous, real-time arcade environment necessitating immediate reaction to visual input. The agent controls a growing line within a bounded grid, where the core mechanic is characterized by diminishing fault tolerance: as the agent successfully consumes food, the snake’s length increases, progressively restricting navigable space and amplifying the probability of fatal collisions. This environment primarily evaluates the VLM’s capacity for dynamic risk-reward management—balancing the aggressive pursuit of rewards against the immediate necessity of survival. Snake serves as a benchmark for inference

Before Reflection	After Reflection
...Based on the plan, the first logical step is to examine the television — it’s the central object in the room and likely holds clues (buttons, dials, hidden compartments, etc.). Since the current view shows the TV from a distance, I need to move closer to interact with it properly. Long-pressing 'W' will allow me to move forward continuously toward the TV, then I can examine it using 'E'. ...	...The absence of chairs 2 and 5 in the visible area suggests they are likely off-screen or require movement to access. Since the last action was pressing ESC (canceling any interaction), we must now begin active exploration. The plan requires examining chairs 2 and 5 first, but they are not visible. Therefore, the logical next step is to move toward the center of the room where chairs are likely positioned — typically near the TV or around the central area — and then examine objects systematically. Given the layout, moving forward (W) will likely bring us closer to the center of the room where chairs may be located....

Figure 48. Qwen3-VL-8B reflection improvement example

latency and perception speed. As a real time game environment, high inference latency inherent to this game leads to immediate collisions. Given the environment’s relatively low visual complexity, this benchmark emphasizes the need for sparse reasoning—evaluating whether the VLM can efficiently parse simple inputs and render decisions without expending excessive inference time on trivial states.

(1) **Game state.** Real-time, contained grid environment.

(2) **Main GUI action space.** Keyboard Input (Directional arrow keys).

(3) **Evaluation task** Control the snake to eat food and avoid obstacles. We impose a maximum limit of 100 steps per episode. The agent’s performance is quantified by the total number of food items consumed by the snake prior to collision with an obstacle or its own body. The normalized score is calculated by taking the average number of food items consumed across n independent trials and dividing it by a scaling factor of 10, which is defined as follows:

$$\bar{S}_{norm} = \frac{1}{10n} \sum_{i=1}^n S_i$$

(4) **Play-through video content** This play through video presents a complete game replay demonstrating the optimal space arrangement strategy, which shows the snake successfully navigating to fully occupy the grid and reach the map’s maximum score limit of 81



Figure 49. Screenshot of Snake

D.10.2 GAME PROMPT FOR SNAKE

We provide the full structure of our prompts in Figure 50.

D.10.3 DETAILED ANALYSIS FOR SNAKE

The performance of GPT-4o and GPT-4o-mini in the Snake environment was unexpectedly poor. These models exhibited a fundamental inability to comprehend the relative spatial relationship between the agent (the snake) and the target (food), as illustrated in the typical error examples. Furthermore, when encountering obstacles, these models frequently violated game constraints by attempting to reverse direction, indicating significant deficiencies in both spatial reasoning and instruction adherence.

In contrast, Qwen3-VL-8B and Gemini-2.5-Flash demonstrated limited spatial perception, with the frequency of erroneous instructions increasing notably as the snake’s body length extended. Qwen3-VL-32B exhibited significantly superior performance, maintaining score stability throughout the gameplay. However, Gemini-2.5-Pro and Seed-1.8 were severely constrained by latency-induced errors, resulting in suboptimal performance overall (see Figure 51 Left).

Latency Error Analysis

Our experiments reveal a critical misalignment between model inference depth and the actual cognitive demands of the task. While the Snake environment can be effectively navigated using straightforward spatial heuristics, advanced models lack the adaptive capacity to autonomously regulate their reasoning density (i.e., employing "sparse reasoning" when appropriate). This rigidity leads to a "reasoning-speed mismatch," where the time consumed by complex Chain-of-Thought processes causes the agent to fail strict real-time temporal constraints.

Model	GUI	GUI VR.
Gemini-2.5-Flash	2.00 ± 1.80	1.00 ± 0.81
Gemini-2.5-Pro	2.38 ± 3.00	6.75 ± 3.20
Qwen3-VL-8B	0.25 ± 0.71	0.00 ± 0.00
Qwen3-VL-32B	4.38 ± 2.61	3.25 ± 1.71
GPT-4o	0.00 ± 0.00	0.00 ± 0.00
GPT-4o-Mini	0.00 ± 0.00	0.00 ± 0.00
Seed-1.8	0.75 ± 1.00	1.00 ± 1.15

Table 19. Main Experiment Results(w/ vs. w/o reflection).

Model	10s	20s	Stop
Qwen3-VL-32B	4.38	4.75	5.33
GPT-4o	0.00	0.50	0.00
Gemini-2.5-Pro	2.38	6.50	7.33
Seed-1.8	0.75	0.86	9.00

Table 20. Latency Analysis

System Prompt

You are an expert AI agent specialized in playing Snake game on a computer with strategic reasoning using visual input. You can only interact with the game through keyboard inputs. Your primary goal is to control the snake to eat the food on the screen, grow longer, and avoid obstacles and walls while maximizing your score.

Game Overview and Rules

1. The game is played on an {board_size}x{board_size} grid.
2. You control a green snake that moves in one of four directions: Left (L), Right (R), Up (U), or Down (D).
3. The snake moves one cell per turn in the current direction. The snake's red tongue indicates its current orientation.
4. You cannot reverse direction - If moving left, you cannot immediately move right, and vice versa. Same for up/down.
5. The snake grows longer when it eats food.
6. Food has a lifespan - it disappears after a certain number of turns if not eaten.
7. The game ends (you lose) if:
 - The snake hits a wall (border of the board)
 - The snake hits its own body
 - The snake hits an obstacle
 - The snake eats negative food when its head is at the tail position

8. Your score increases when you eat food and grow longer.
- Ensure That:

- **PRIORITY:** Always try to move towards the RED APPLE while avoiding danger.
- **CRITICAL:** You must NOT reverse direction. If currently moving left, you can only choose left, up, or down (not right).
- **CRITICAL:** Choose a direction that will not cause the snake to hit walls, obstacles, or its own body.

IMPORTANT: Before choosing a direction, carefully examine the image to:

- FIND THE RED APPLE: Locate the bright red apple on the screen - this is your target!
- Identify the snake's current position (green snake) and direction.
- MOVE TOWARDS THE RED APPLE while avoiding walls, obstacles, and your own body.
- Remember you cannot reverse direction immediately.

Screenshot guidelines

You will be provided with an image representing the current game state. The image will show the entire board, including the snake, food items, obstacles, and walls. The food look like a red apple, with a bar above it indicating its remaining lifespan. Obstacles are represented as black wall blocks on the board. The snake is green with a red tongue indicating its current direction.

Learned Experience(if activated reflection)

[reflection content learned from video]

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Valid Actions

You can press the following arrow keys to control the snake:

- Press 'left' arrow key: Snake moves left
- Press 'right' arrow key: Snake moves right
- Press 'up' arrow key: Snake moves up
- Press 'down' arrow key: Snake moves down

Note that: Up, down, left, and right refer to the entire game window, rather than the snake's orientation.

Output Format

Based on what you see on the screen, decide which key to press next. You MUST output your action in the following JSON format:

```
{ "action_type": "PRESS", "parameters": "key": "<left|right|up|down>" }
```

User Prompt

Target task

Your objective is to reach floor 50 in Slay the Spire.

Previous state



Last executed actions

```
{ "action_type": "PRESS", "parameters": "key": "left" }
```

Current state



Your Response Format

First, analyze the provided game state image and explain your reasoning. Then, output your keyboard action in the JSON format.

Reasoning

<explain why you choose this direction based on visual analysis, including the RED APPLE position, obstacles, and potential dangers>

Actions <your action in JSON format>

The '### Reasoning' field provides a clear explanation based on what you see in the image.

The '### Actions' field contains exactly the JSON format with PRESS action and key.

Figure 50. snake prompt

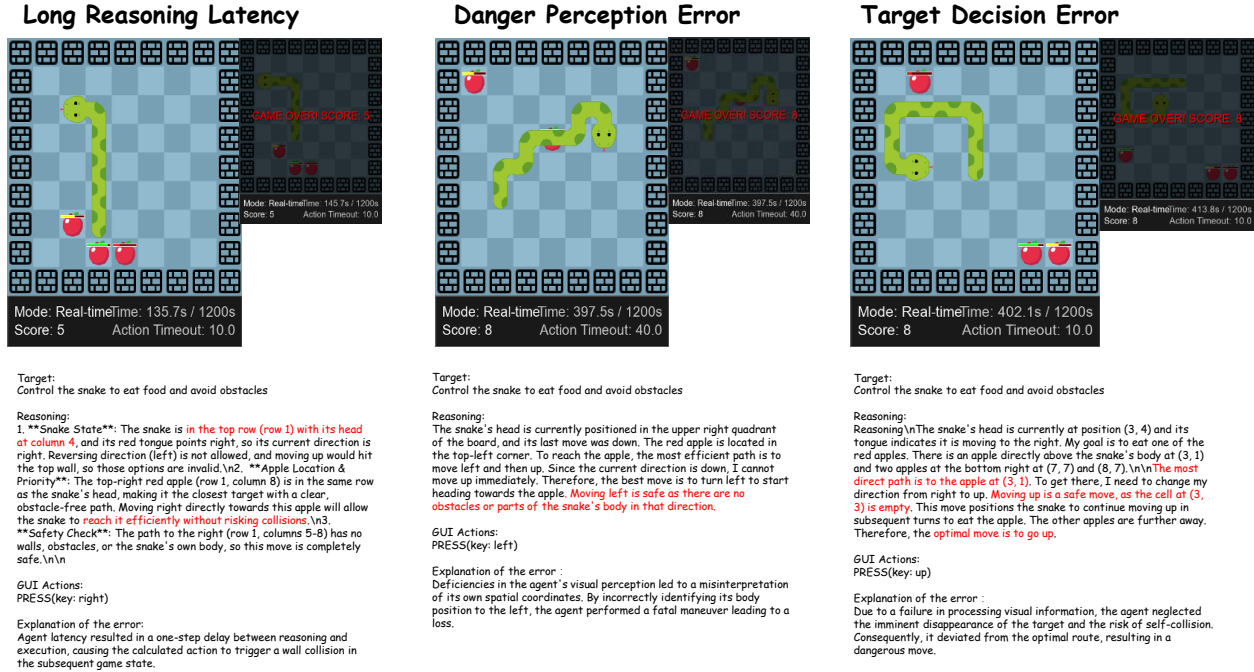


Figure 51. snake typical errors

This limitation is most paradigmatically illustrated by Seed-1.8, which demonstrated pronounced latency-induced failures. The model's single-step inference time frequently exceeded 20 seconds, creating excessive computational overhead that rendered it unsuitable for the real-time, small-grid dynamics of Snake. However, notably, the model performed robustly in the static "stop" delay perception experiments (see Table 20). This performance discrepancy confirms that while the model possesses strong underlying reasoning capabilities, its inability to dynamically inhibit overly complex chain-of-thought processes creates a fatal bottleneck in time-sensitive scenarios.

Video Reflection Analysis

Models such as GPT-4o, GPT-4o-mini, and Qwen3-VL-8B failed to achieve performance gains through reflection. Due to inherent limitations in multimodal understanding and reasoning, these models were unable to extract actionable insights from either their own failure history or expert demonstration videos.

Conversely, models such as Qwen3-VL-32B and Gemini-2.5-Flash exhibited stagnant or slightly declined scores following video-based reflection learning. An analysis of their reflective outputs indicates that these models successfully internalized valid strategic principles, such as "Prioritize open paths over immediate food rewards" and "Avoid clustering around food items to preserve mobility." (see Figure 52) However, a disconnect remained between strategy formulation and execution: limited by visual perception errors (see Figure 51, middle), the models frequently ignored obstacles or the snake's own body despite correctly identifying the optimal strategy.

Notably, Gemini-2.5-Pro effectively leveraged video reflection to achieve "sparse reasoning." By adopting direct heuristics rather than engaging in redundant ab initio reasoning, the model significantly improved its reaction speed. Post-reflection, it achieved performance levels comparable to the baseline established in the static "stop" experiments.

skills from video reflection

1. Always check if the next tile is occupied by your own body before moving—this is the single most critical safety rule to prevent self-collision.
- ...
5. Prioritize open paths over immediate food rewards when approaching corners or narrow passages.
- ...
10. If multiple apples are visible, choose the one that allows for the longest safe continuation without blocking yourself.
- ...
16. Avoid clustering around food items if it reduces your overall mobility—spread out to maintain flexibility.
- ...

Figure 52. Qwen3-VL-32B reflection content

D.11. Plants vs. Zombies

D.11.1 GAME DESCRIPTION FOR PLANTS VS. ZOMBIES

Game Environments. Plants vs. Zombies is a strategic tower defense game where players must defend a home from incoming waves of enemies by placing plants with specific functions. The environment requires managing a generated resource (sunlight) and spatial positioning. It tests tactical positioning and real-time resource allocation. Agents must recognize enemy types and counter them with the appropriate unit composition while optimizing the economy. This game evaluates the agent’s ability to multitask in real-time: collecting resources, monitoring multiple lanes of attack, and deploying units strategically. It combines the pressure of real-time dynamics with the need for thoughtful combinatorial strategy to survive escalating waves of threats.



Figure 53. Screenshot of Plant vs. Zombies

(1) **Game state.** Real-time 2D grid-based defense layout.

(2) **Main Semantic action space.** Plant, Collect, and Wait.

(3) **Main GUI action space.** Mouse Click (Collect resources, place units).

(4) **Evaluation task.** Complete the three levels: 1-1, 1-2, and 1-4. We impose a maximum limit of 50 steps per episode. The agent’s performance is quantified by the average number of completed milestones. The milestones are generated from the expert video and are listed in Table 21. To obtain a standardized score, the number of completed milestones is normalized by the total number of milestones, and the calculation formula is as follows:

$$S_{norm} = \frac{N_{completed}}{N_{total}} \times 100$$

where $N_{completed}$ denotes the number of milestones completed by the agent, and N_{total} denotes the total number of milestones (19 in total, from three levels, as listed in Table 21). The normalized score ranges from 0 to 100.

Table 21. Milestones of Plants vs. Zombies (Levels 1-1, 1-2, and 1-4)

Level 1-1		Level 1-2		Level 1-4	
No.	Milestone Title	No.	Milestone Title	No.	Milestone Title
1	Planting First Peashooter	1	First Economic Setup	1	Level Start
2	First Sun Collection	2	First Enemy Wave Warning	2	First Economic Setup
3	Planting Second Peashooter	3	First Defense Deployment	3	First Defensive Action
4	First Zombie Appearance	4	Huge Wave Alert	4	Countering Buckethead
5	Defeating First Zombie	5	Final Wave Start	5	Huge Wave Event
6	Final Wave Triggered	6	Level Complete & Unlock	6	Level Complete & Shovel
7	Level Complete & Sunflower				

(5) **Expert video content.** The video demonstrates how to pass through three levels, including strategies for plant cultivation and the management of sunlight resources.

D.11.2 GAME PROMPT FOR PLANTS VS. ZOMBIES

Our implementation of Plant vs. Zombies uses zero-shot agent. We provide the full structure of our prompts for GUI mode (Figure 54) and for semantic mode (Figure 55).

D.11.3 DETAILED ANALYSIS FOR PLANTS VS. ZOMBIES

The Plants vs. Zombies (PvZ) environment highlights the distinct challenges VLMs face when integrating resource management with real-time spatial defense. In the *Plants vs. Zombies* experiments (Table 22), strong reactive models such as

System Prompt

You are a strategic player for the game 'Plants vs. Zombies'. Your role is to control the game by simulating mouse actions.

You will be given a screenshot of the game window. Based on the visual information, you need to determine the best action to take.

Game Window Size: {window_width} x {window_height} pixels

Game Overview

Plants vs. Zombies is a tower defense game where you must defend your house from waves of zombies by strategically placing plants.

Game Mechanics

...Introduction of Game Mechanics...

Valid Actions

GUI Actions:

- **CLICK:** Click at a specific screen position
- **MOVE_TO:** Move mouse to a specific position (useful before clicking)
- **WAIT:** Wait for a specified duration (in seconds)

Common Action Sequences: 1. **Plant a plant:** Click plant slot → Click target grid cell

2. **Collect sun:** Click on visible sun icons (they appear as glowing yellow circles)

3. **Remove/Dig plant:** Use shovel (if available) → Click on plant to remove

Action Guidelines

...Guidelines...

Learned Experience from Previous Attempts

{reflection_experience}

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Output Format

You must output your actions as a JSON array. Each action has an action_type and parameters.

Available action types:

- **CLICK:** Click at a specific position

```
{
  "action_type": "CLICK",
  "parameters": {
    "x": <int>,
    "y": <int>,
    "button": "left"
  }
}
```
- **MOVE_TO:** Move mouse to a specific position

```
{
  "action_type": "MOVE_TO",
  "parameters": {
    "x": <int>,
    "y": <int>
  }
}
```
- **WAIT:** Wait for specified duration (seconds)

```
{
  "action_type": "WAIT",
  "parameters": {
    "duration": <float>
  }
}
```

...Examples of Action Output...

Note: A single response can execute multiple GUI mouse actions.

User Prompt

Target task

{task_description}

Previous state



Last executed action

{action}

Current state



Based on the current game screenshot, analyze the situation and output the GUI action(s) to execute. Remember to convert grid positions to screen coordinates.

You should only respond in the format described below. Provide your response in the strict format:

```
### Reasoning
<detailed strategic analysis>
### Actions
```json
<your GUI action JSON>
```
```

Figure 54. Plants vs. Zombies GUI prompt

System Prompt

You are playing Plants vs. Zombies. Your goal: prevent zombies from reaching the house on the left.

Game Mechanics

- Zombies walk from RIGHT to LEFT along their row
- When zombies meet plants, they stop and eat the plant
- Offensive plants automatically attack zombies in the SAME row
- Defensive plants block zombies while offensive plants attack
- Sunflowers produce sun (no attack ability)
- Sun is currency for planting

Grid Layout

- 5 rows $\times$ 9 columns
- Rows: 1 (top) to 5 (bottom)
- Columns: 1 (left) to 9 (right)
- You can ONLY plant on green grass (some rows may be dirt)
- Check the image to see which rows have grass

Actions

1. plant <slot> at (<row>, <col>) - Plant from slot at position
 - Slots are numbered 1-8 (1st slot, 2nd slot, etc.)
 - Example: plant 1 at (2, 3) means plant 1st slot at row 2, column 3
2. collect - Collect all visible suns
3. wait - Skip this turn

Multi-step actions allowed

(newline or semicolon separated):

collect

plant 1 at (2, 3)

plant 2 at (5, 4)

Think step-by-step

1. Which rows have zombies? Which rows have grass for planting?
2. Are there suns to collect?
3. Do zombie-threatened rows have offensive plants?
4. What should I plant and where?

Learned Experience from Previous Attempts

{reflection_experience}

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Output format

Reasoning

[Your analysis]

Actions

[Your action commands]

User Prompt

Target task

{task_description}

Previous state



Last executed action

{action}

Current state



Based on the current game screenshot, analyze the situation and decide your next action. Consider: current sunlight, zombie positions, plant placement, and available plants.

You should only respond in the format described below, and you should not output comments or other information. Provide your response in the strict format:

Reasoning

<detailed strategic analysis>

Actions

<your action command>

Figure 55. Plants vs. Zombies semantic prompt

GPT-4o, Qwen3-VL-32B and Gemini-2.5-Flash perform well. As illustrated in Figure 56, failures predominantly manifest as **State Perception Hallucination**, **Grid-Spatial Mapping Misalignment**, and **Spatial Affordance Misjudgment**.

Table 22. Model Accuracy Comparison: GUI vs. Semantic (with and without VR)

| Model | GUI | GUI VR. | Semantic | Semantic VR. |
|------------------|-------------|--------------------|-------------|--------------------|
| Qwen3-VL-8B | 33.4 ± 4.1 | 51.3 ± 21.4 | 54.2 ± 5.8 | 65.2 ± 11.3 |
| Qwen3-VL-32B | 41.2 ± 12.6 | 48.3 ± 19.0 | - | - |
| GPT-4o-mini | 27.6 ± 14.0 | 22.4 ± 10.4 | - | - |
| GPT-4o | 32.2 ± 12.4 | 36.4 ± 15.2 | 42.7 ± 16.0 | 53.4 ± 7.8 |
| Seed-1.8 | 26.1 ± 8.2 | 22.1 ± 10.3 | - | - |
| Gemini-2.5-Flash | 51.1 ± 13.2 | 55.3 ± 10.4 | 70.2 ± 8.5 | 81.8 ± 17.0 |
| Gemini-2.5-Pro | 33.4 ± 17.2 | 30.3 ± 16.4 | 45.4 ± 3.3 | 52.7 ± 15.2 |

State Perception Hallucination



Target:
Defend your house from zombies by strategically placing plants

Reasoning:
Currently, there are no visible zombies, and I have 150 sunlight. I have already planted a pea shooter in the first row at (1, 1). Since there are no immediate threats, I should focus on generating more sunlight by planting sunflowers. I can plant a sunflower in the second row, which will help produce sunlight while keeping the pea shooter in the first row to defend against any incoming zombies.

action:
plant 2 at (2, 1)

Explanation of the error:
The agent exhibited a high-level cognitive hallucination. It failed to perceive the actual contents of the plant seed slot, leading to a complete breakdown in the Status Tracking of environmental constraints. Consequently, the model prioritized a generic game template (prior knowledge) over the specific, observable state of the current level.

Grid-Spatial Mapping Misalignment



Target:
Defend your house from zombies by strategically placing plants

Reasoning:
The current state shows that zombies are already approaching from the right in row 2. There is a peashooter at (2,7) actively shooting, but it is too close to the zombies and will be eaten soon. The sun count is low (25), so I must collect any available sun before planting more defenses. Two sunflowers are already planted in row 2 at columns 3 and 4, which will help generate sun over time. However, I need immediate defense. The best strategy is to collect all visible suns first to increase my resources, then plant additional peashooters further back in row 2 to create a layered defense. Since the zombies are in row 2, reinforcing that row with more offensive plants is critical. After collecting sun, I can plant another peashooter at (2,5) to provide better coverage and extend the defense line.

action:
collect
plant 1 at (2, 5)

Explanation of the error:
The error stems from Grid Confusion during the translation of logical cells to physical pixels. Although the agent formulated a valid non-overlapping plan, the Mapping Misalignment caused the execution to drift vertically or horizontally. This resulted in an "Occupancy Conflict," where the agent attempted to deploy a plant on an already occupied grid, leading to a wasted action and a failure to update the defense line.

Spatial Affordance Misjudgment



Target:
Defend your house from zombies by strategically placing plants

Reasoning:
Currently, I have 150 sunlight and have already planted a Peashooter at (2, 4). There are no visible zombies in the current state, but I need to prepare for their arrival. I should plant more offensive plants to ensure I can defend effectively when zombies appear. Since I have enough sunlight, I can plant another Peashooter to increase my offensive capabilities. The best position for the next Peashooter would be in row 2, column 5, to cover the next column and maximize my attack range.

action:
plant 1 at (2, 5)

Explanation of the error:
The agent failed to recognize the physical interaction boundaries within the environment, resulting in a disconnection between visual recognition and functional affordance. While the model can visually identify background elements such as the dirt or fences at the top and bottom of the screen, it fails to understand at the logical level that these areas do not possess the "plantable" physical property (affordance).

Figure 56. Plant vs. Zombies Typical Errors

State Perception Hallucinations and Environmental Constraint Violations.

A critical failure mode involves the agent prioritizing parametric knowledge over visual evidence. In the "State Perception Hallucination" instance, the agent attempts to execute a standard opening strategy: "plant a sunflower in the second row". However, this action is technically impossible as the current seed slot only contains a Peashooter card. This error represents a "high-level cognitive hallucination" where the model's internal prior (that Sunflowers are essential early-game units) overrides the specific, observable state of the interface. The model fails to perform the requisite "Status Tracking" of the seed bank, resulting in a hallucinated action plan that ignores immediate inventory constraints.

Grid-Spatial Mapping Misalignment.

Agents struggled to accurately map logical grid coordinates (e.g., Row 2, Col 8) to precise 2D pixel coordinates on the isometric lawn. **Coordinate Drift:** Significant discrepancies were observed between the intended action and the actual deployment location, often resulting in plants being placed in incorrect lanes or failing to reach the intended defensive position. **Impact of Perspective:** The inability to account for the game's specific spatial projection meant that even sound strategic intentions failed to translate into effective tactical layouts.

Video Reflection Analysis: Strategy Optimization vs. Execution Latency.

The introduction of expert-guided reflection led to noticeable improvements in strategic depth, though execution remained a bottleneck. **Paradigm Shift:** Following exposure to expert experiences, models shifted from random placement to structured layouts, such as prioritizing Sunflowers in the rear ranks and Peashooters in forward positions. **Execution Latency:** A persistent issue of temporal lag was observed between reasoning and execution. In the dynamic environment of *Plants vs. Zombies*, this latency often rendered commands obsolete by the time they were executed, as the tactical situation had already shifted.

D.12. Forza Horizon 5

D.12.1. GAME DESCRIPTION FOR FORZA HORIZON 5

Game Environments. Forza Horizon 5 (Playground Games, 2021) serves as a high-fidelity benchmark for continuous control, offering photo-realistic 3D visuals and complex vehicle dynamics. The evaluation is conducted within a closed stadium circuit, a confined environment characterized by strict spatial boundaries and distinct track geometry. As a 'Real-Time Linear' task, it rigorously tests visual-motor coordination, requiring the agent to interpret spatial depth and visual driving lines to generate precise steering and throttle commands in real-time. Crucially, this environment assesses failure recovery robustness; beyond merely maintaining high-speed trajectories, the agent must correctly identify collision states (e.g., wall impacts) and execute autonomous rescue maneuvers to regain control.

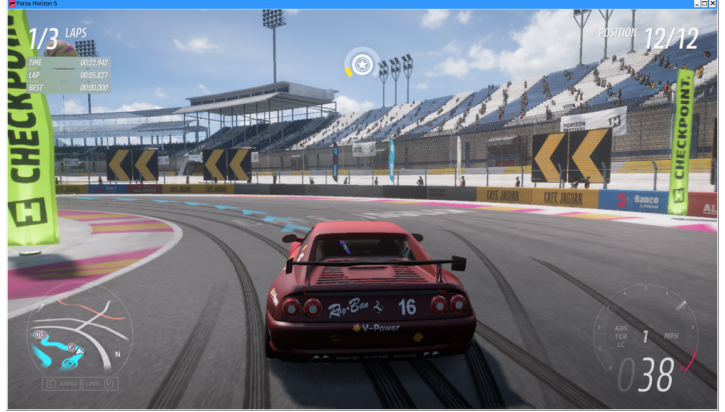


Figure 57. Screenshot of Forza Horizon 5

(1) **Game state.** Real-time, High-Fidelity 3D environment.

(2) **Main GUI action space.** Keyboard Input (Continuous control for steering and throttle).

(3) **Evaluation task** Complete the stadium circuit race as quickly as possible while avoiding crashes. We impose a maximum limit of 3 minutes per episode. The agent’s performance is quantified by the total number of valid checkpoints traversed within a fixed duration of 3 minutes. This time window is calibrated to the approximate duration required for a baseline novice agent to complete three full laps. The circuit comprises 10 checkpoints per lap (including the finish line), establishing a maximum reference score (C_{max}) of 30 checkpoints. The final metric is derived by averaging the number of checkpoints passed across N trials and normalizing this value against the maximum reference. The normalized score is defined as follows:

$$S_{norm} = \frac{\frac{1}{N} \sum_{i=1}^N C_i}{C_{max}} \times 100, \quad \text{where } C_{max} = 30 \text{ and } C_i \text{ is the checkpoints passed in trial } i.$$

(4) **Expert video content** The expert video is a skill tutorial for Forza Horizon, specifically illustrating the correlation between vehicle speed and turning radius based on centripetal force. It provides a theoretical and practical analysis of the ‘out-in-out’ cornering technique, explaining how maximizing the turning radius allows for higher cornering velocities. Furthermore, the tutorial elucidates the limitations of tire traction, emphasizing the necessity of managing longitudinal braking force and lateral steering grip to maintain vehicle stability.

D.12.2 GAME PROMPT FOR FORZA HORIZON 5

We provide the full structure of our prompts in Figure 58.

System Prompt

You are an AI agent playing Forza Horizon 5, specifically competing in a stadium circuit race. Your goal is to complete the circuit as quickly as possible while avoiding crashes and staying on track. You will be given a screenshot of the game window. Based on the visual information, you need to determine the best action to take.

Game Overview

Forza Horizon 5 is an open-world racing game. You will be racing on a stadium circuit - a closed track with clear boundaries. Your objective is to:

- Complete the race circuit
- Maintain high speed when safe
- Navigate turns properly
- Avoid collisions with barriers, other vehicles, and obstacles

Your Task

Analyze the current screenshot, understand the track situation, and decide the appropriate driving actions.

Controls

You control the racing car using keyboard inputs:

Basic Controls:

- W: Accelerate (gas pedal)
- S: Brake/Reverse
- A: Steer left
- D: Steer right

Control Method: For precise racing control, you should use KEY_DOWN and KEY_UP actions to manage multiple keys simultaneously.

- KEY_DOWN: Press and hold a key (it stays pressed until you release it)
- KEY_UP: Release a previously pressed key
- WAIT: Pause for a specified duration while keys remain in their current state

Screen Information

You will receive screenshots showing:

- **Main Screen:** The overall game view showing the Road/Track.
- **Road Guidance Line (IMPORTANT):** A colored line on the road indicating the optimal racing line.
- **U-turn Sign:** If you drive in the wrong direction, a sign appears.
- **Car Status:** Speed on bottom-right.
- **Mini Map:** Circuit map on bottom-left.

Guideline

1. **Line Color:** Green (accelerate), Yellow (decelerate), Red (heavy braking).
2. **Turns:** Follow the guideline to turn; direction of the guideline is always more important than the mini map.
3. **Recovery:** If you can't see the guideline, try to recover by steering back to the track.

Learned Experience(If activated)

[...reflection_experience content...]

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs.

Action Output Format

Available action types: KEY_DOWN, KEY_UP, WAIT.

Actions are executed sequentially. Always ensure to release keys with KEY_UP after pressing them with KEY_DOWN.

Example - Accelerate straight for 2 seconds:

```
[{
  "action_type": "KEY_DOWN", "parameters": {"key":
"w"}
}, {
  "action_type": "WAIT", "parameters": {"duration":
2.0}
}, {
  "action_type": "KEY_UP", "parameters": {"key":
"w"}
}]
...
```

User Prompt

Target task

Complete the stadium circuit race as quickly as possible while avoiding crashes.

Previous state



Last executed action

```
[
  { "action_type": "KEY_DOWN", "parameters": { "key":
"w" } },
  { "action_type": "WAIT", "parameters": {
"duration": 0.5 } }
  { "action_type": "KEY_UP", "parameters": { "key":
"w" } }
]
```

Current state



Response Requirements

You should only respond in the format described below, and you should not output comments or other information.

Respond in the strict format:

Reasoning

[Analyze the screenshot and explain what driving action you want to take and why. Consider line color, upcoming turns, and current speed.]

Actions

```
```json
[
 { "action_type": "...", "parameters": {...} }
]
```
```

Figure 58. Horizon prompt

D.12.3 DETAILED ANALYSIS FOR FORZA HORIZON 5

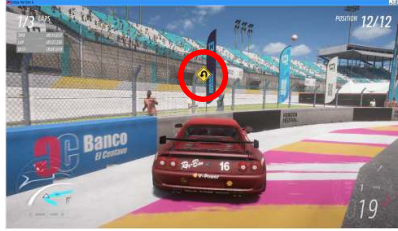
As detailed in Table 23, the overall performance across all evaluated models is critically low, bordering on catastrophic failure for this specific domain. The results indicate that current general-purpose Vision-Language Models remain ill-equipped for continuous real-time games that necessitate millisecond-level reaction speed and muscle-memory-like execution.

The primary bottleneck is the latency mismatch: the inference time of general models significantly exceeds the update frequency of the game environment. Consequently, models are unable to adjust the game state in real-time through rapid, iterative action sequences. This manifests as consistent failures during cornering maneuvers, where models inevitably crash into walls and struggle to navigate back to the track.

| Model | GUI | GUI VR. |
|------------------|-------------------|-------------------|
| Gemini-2.5-Flash | 1.333 ± 0.577 | 1.333 ± 0.58 |
| Gemini-2.5-Pro | 1.0 ± 0.0 | 1.25 ± 0.5 |
| Qwen3-VL-8B | 1.0 ± 0.0 | 1.0 ± 0.0 |
| Qwen3-VL-32B | 1.0 ± 0.0 | 1.0 ± 0.0 |
| GPT-4o | 0.75 ± 0.5 | 1.0 ± 0.0 |
| GPT-4o-Mini | 1.0 ± 0.0 | 1.0 ± 0.0 |
| Seed-1.8 | 1.33 ± 0.58 | 0.666 ± 0.577 |

Table 23. Raw Score of Forza Horizon 5

Visual Ignoring and misunderstanding



Target:
Complete the stadium circuit race as quickly as possible while avoiding crashes

Reasoning:
[T]he car is currently at a near standstill (1 MPH) and positioned on the track, facing forward. The mini-map shows the upcoming route, which includes a sharp left turn ahead. The U-turn sign is not visible, so we are driving in the correct direction. Since the car is almost stopped, I need to accelerate to regain speed before entering the turn. There is no immediate obstacle or barrier ahead, so accelerating straight is safe.

GUI Actions:
KEY_DOWN(key: "w"),
WAIT(duration:2.0),
KEY_UP(key: "w")

Explanation of the error:
The agent misidentifies the gear information as vehicle speed, leading it to believe the car is stationary. Furthermore, it completely ignores the U-turn prompt on the screen, resulting in it heading in the wrong direction.

Spatial Disorientation



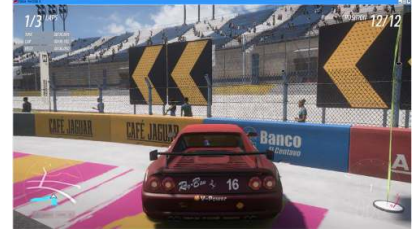
Target:
Complete the stadium circuit race as quickly as possible while avoiding crashes

Reasoning:
The car is still facing the barrier and needs to be repositioned onto the track. The previous action of reversing and steering left was not sufficient. We need to continue reversing while steering left to clear the barrier and get back on track.

GUI Actions:
KEY_DOWN(key: "s"),
KEY_DOWN(key: "a"),
WAIT(duration:1.5),
KEY_UP(key: "a"),
KEY_UP(key: "s")

Explanation of the error:
Due to occlusions and complex spatial relationships in the screenshots, the agent misidentified the state as facing an obstacle rather than being backed against it. The agent failed to understand that spatial orientation should be relative to the vehicle, not the camera.

Inverse Reasoning Error



Target:
Complete the stadium circuit race as quickly as possible while avoiding crashes

Reasoning:
The car has come to a complete stop (0 MPH) and is positioned on the track, likely after hitting a barrier or going off-track. The mini-map shows the car is still on the circuit, but it's not moving forward. The large yellow chevron signs indicate an upcoming sharp left turn, which suggests we are approaching a corner. Since the car is stationary, we need to reverse slightly to reposition ourselves properly before attempting to accelerate again. Reversing while steering left will help align the car with the correct racing line for the upcoming turn.

GUI Actions:
KEY_DOWN(key: "s"),
KEY_DOWN(key: "a"),
WAIT(duration:1.0),
KEY_UP(key: "a"),
KEY_UP(key: "s"),
KEY_DOWN(key: "w"),
WAIT(duration:2.0),
KEY_UP(key: "w")

Explanation of the error:
The agent recognizes the need to turn left and reverse to clear the wall. However, it fails to comprehend the complex spatial relationship required for this maneuver—specifically, that reversing while turning left necessitates steering to the right.

Figure 59. Horizon typical errors

Spatial Understanding and Visual Perception

Beyond latency constraints, our qualitative analysis reveals significant deficiencies in the models' comprehension of the 3D environment. In high-fidelity 3D scenes, the ability of models to interpret the game state purely from visual inputs degrades substantially.

We identified three primary categories of spatial errors (Figure 59): Visual Information Neglect, where models frequently overlook critical GUI or track information (Left); Spatial Misalignment, characterized by a marked inability to understand relative spatial positions within the 3D depth field (Middle); and Physics and Dynamics Hallucination, in which models lack a grounding in vehicle physics, particularly regarding steering relationships, as evidenced by their inability to determine the correct steering direction during reversing maneuvers (Right).

These deficits in spatial reasoning directly lead to a low success rate in simple recovery tasks, such as returning to the track

after a collision, further compounding the poor performance scores.

Video Reflection Analysis

Despite the integration of a reflection module, models failed to achieve performance gains. Our analysis suggests that while models can often correctly capture high-level instructional information from teaching materials, they fail to translate this theoretical knowledge (e.g., cornering speed adjustments and trajectory control) into actionable policy updates.

The efficacy of reflection is further hampered by the low frequency of state updates and the fragmentation of failure videos, which prevents the models from accurately localizing the root cause of failure. In conclusion, current models cannot effectively leverage reflection to improve in complex, real-time gaming environments, indicating a need for comprehensive capability enhancements.

D.13. Mini Metro

D.13.1 GAME DESCRIPTION FOR MINI METRO

Game Environments. Mini Metro (Dubisayr Polo Club, 2015) is a dynamic strategy simulation that tasks agents with constructing an efficient subway network on a continuously expanding map. The environment features stochastic node generation and strict temporal constraints, necessitating rapid, real-time topological optimization to mitigate station overcrowding. Unlike static planning tasks, this game demands fine-grained visual discrimination; the VLM must accurately identify diverse geometric station types and passenger demands within a complex, visually overlapping graph. Success relies on the agent’s ability to balance immediate local congestion with global network efficiency. Thus, Mini Metro serves as a rigorous testbed for evaluating a model’s capacity to integrate high-speed visual perception with adaptive, long-horizon spatiotemporal reasoning.

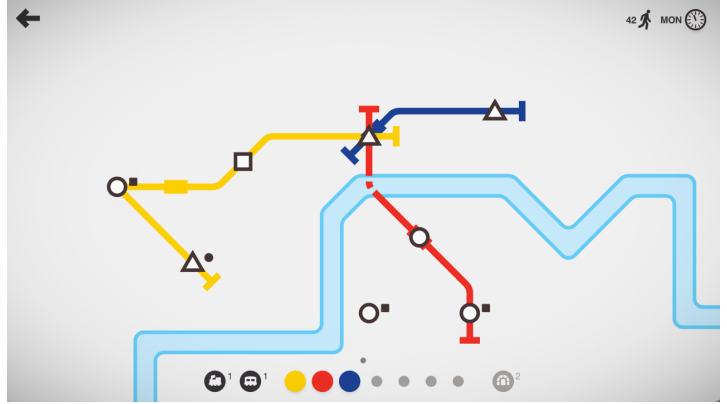


Figure 60. Screenshot of Mini Metro

(1) **Game state.** Abstract, evolving 2D graph.

(2) **Main GUI action space.** Mouse Move, Click and Drag (Drawing lines to connect nodes).

(3) **Evaluation task** Design an efficient subway network to transport more passengers. The raw metric is defined as the total number of passengers successfully delivered before an episode terminates, when any station exceeds its passenger capacity. We impose a maximum limit of 100 steps per episode. The score is calculated as the mean number of transported passengers over n episodes, normalized by a constant of 500. Formally, it can be defined as follows:

$$S_{norm} = \frac{\sum_{i=1}^n p_i}{500n} \times 100, \quad \text{where } p_i \text{ represents the passenger throughput of episode } i$$

(4) **Expert video content** The expert video is a skill tutorial, presenting an integrated framework for both topological network design and dynamic resource allocation. It explicitly demonstrates optimal routing heuristics, such as the superior efficiency of loop structures over linear paths and the necessity of avoiding acute angles to mitigate transit deceleration. Parallel to these geometric considerations, the footage illustrates the critical management of finite assets—including locomotives and tunnels—to balance system capacity against stochastic passenger demand.

D.13.2 GAME PROMPT FOR MINI METRO

Due to the limited precision of VLM’s native GUI grounding for the fine-grained operations required in *Mini Metro*, we utilized OpenCV contour detection to provide auxiliary coordinate information in the `{cur_state_str}` placeholder. However, the model is still required to correctly map the spatial targets in the screenshot to these provided auxiliary coordinates. We provide the full structure of our prompts in Figure 61.

System Prompt

You are an expert subway network designer AI specialized in playing the strategy simulation game 'Mini Metro'. Your input is the game screenshot consisting of a minimalist map of a growing city.

Game Window Size: {window_width} x {window_height} pixels

Your goal is to design an efficient subway network to transport passengers to their destinations without letting any station overcrowd.

Mini Metro - Visual Awareness & Objects

1. **Stations:** Large geometric shapes (Circles, Triangles, Squares, Stars, etc.) on the map. These are your nodes.
2. **Passengers:** Small solid shapes appearing next to Stations. A Triangle passenger wants to go to a Triangle Station.
3. **Lines:** Different colored paths connecting stations. You can drag lines to extend them or reroute them.
4. **Rivers:** Blue shapes. Crossing them requires a 'Tunnel/Bridge' resource.
5. **Overcrowding (CRITICAL):** If a station has too many waiting passengers or a circular timer around it is filling up, it is in danger. You MUST prioritize this.
6. **Resources:** Displayed usually on the bottom of the screenshot (Locomotives, Tunnels, Lines, Carriages). Check if you have available lines or trains.

Basic Game Actions

1. **Add new line to connect Stations:** You can draw a new line between two stations by dragging from one station to another.
2. **Modify existing Lines:** You can drag an existing line to a new station to insert it into the route.
3. **Extend Lines:** You can drag the end of an existing line (with a 'T' end) to a new station to extend the route.
4. **IMPORTANT:** When you run out of lines, you cannot connect new stations. You have to modify existing lines or extend them instead.
5. **Usage of Resources:** You can drag new trains (locomotives) or Carriages to existing lines to increase capacity.
6. **Resource Management:** If you run out of tunnels, you cannot cross water. If you run out of lines, you cannot start a new route.
7. **Resources Choice:** At the end of each week, you can Click to choose one new resource (new line, new train, new tunnel, or new carriage). Prioritize based on your current network needs.

Action Guidelines

- Analyze the screenshot carefully to identify UI elements (Stations, Passengers, Lines, Resources, etc.)
- Estimate the positions of your target elements approximately based on visual cues in the screenshot.
- Based on your estimate, refer to the stations positions list provided to decide the accurate positions to interact with.
- Note that only station positions are given to refer to. You need to estimate other elements' positions yourself.
- You interact with the game by simulating **MOUSE DRAGS** or **CLICKS**.
- Estimate the pixel coordinates of the element you want to interact with.
- You can output multiple actions in sequence to complete a complex operation (e.g., drag a line to a station; drag a train to a line).

Learned Experience from Previous Attempts(if activated)

{reflection_experience}

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Output Format

Analyze the screenshot and decide the best move. You MUST output your actions in the following JSON format:

Available action types:

- **CLICK:** Click at a specific position
- **MOVE_TO:** Move mouse to a specific position
- **DRAG_TO:** Drag from current position to target position
- **WAIT:** Wait for a short duration (e.g., to let trains move)

Example - Connect two stations (move to one station, then drag to another):

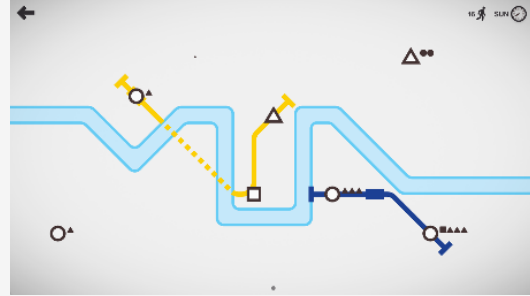
```
[
  { "action_type": "MOVE_TO", "parameters": { "x": 436,
    "y": 698 } },
  { "action_type": "DRAG_TO", "parameters": { "x": 538,
    "y": 356 } }
]
```

User Prompt

Target task

design an efficient subway network to transport passengers

Previous state



Last executed action

```
[
  {
    "action_type": "MOVE_TO",
    "parameters": { "x": 334, "y": 220 }
  },
  {
    "action_type": "DRAG_TO",
    "parameters": { "x": 470, "y": 158 }
  },
]
```

Current state



{cur_state_str} stations' positions you can refer to:

Station 1: Center coordinates (196, 559)

Station 2: Center coordinates (625, 474)

...

Response Requirements

You should only respond in the format described below, and you should not output comments or other information.

Provide your response in the strict format:

Reasoning

[Analyze the screenshot and explain what UI element you want to interact with and why]

Actions

```json

```
[
 { "action_type": "...", "parameters": { ... } }
]
```

```

Figure 61. Mini Metro prompt

D.13.3 DETAILED ANALYSIS FOR METRO

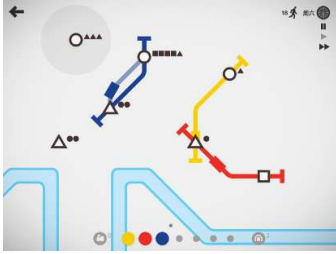
Performance Overview.

The Mini Metro task, which demands a synergy of logical reasoning and real-time responsiveness, remains a significant challenge for current general Vision-Language Models (VLMs). As detailed in Table 24, varying bottlenecks were observed across different architectures. Seed-1.8 failed to progress effectively due to excessive single-step inference latency, while GPT-4o-mini yielded lower scores primarily due to weaker reasoning capabilities. Notably, more powerful reasoning models such as Qwen3-VL-32B and Gemini-2.5-Pro were outperformed by their lighter counterparts (Qwen3-VL-8B and Gemini-2.5-Flash) in the standard setting. This inversion indicates that for real-time decision-making tasks, the latency constraints of large-scale reasoning models can negate their logical superiority. Consequently, Mini Metro serves as an effective benchmark for evaluating the trade-off between inference speed and reasoning depth. It is worth noting that nearly all models demonstrated improvement after incorporating video reflection, suggesting that heuristic information derived from visual experience effectively aids model reasoning.

Model	GUI	GUI VR.
Gemini-2.5-Flash	74.5 $\pm$ 19.29	78.0 $\pm$ 46.4
Gemini-2.5-Pro	57.4 $\pm$ 15.09	87.2 $\pm$ 40.62
Qwen3-VL-8B	39.33 $\pm$ 23.51	41.25 $\pm$ 20.09
Qwen3-VL-32B	26.5 $\pm$ 19.15	52.8 $\pm$ 22.28
GPT-4o	58.0 $\pm$ 11.42	68.0 $\pm$ 13.78
GPT-4o-mini	23.0 $\pm$ 15.6	34.0 $\pm$ 12.20
Seed-1.8	5.0 $\pm$ 9.61	3.42 $\pm$ 6.10

Table 24. Raw Performance on Mini Metro

Bad Rule Interpretation



Target:
design an efficient subway network to transport passengers

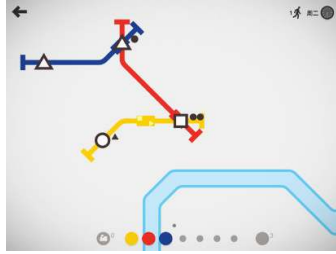
Reasoning:

The current network is fragmented into two separate systems. The blue line connects a circle and a triangle station on the left. The yellow and red lines connect a circle, triangle, and square station on the right. The circle station on the blue line is becoming overcrowded with square passengers who have no path to the square station on the red line. To solve this, I will merge the two networks. I will reroute the blue line by disconnecting it from its current triangle station and connecting it to the central triangle station that is shared by the yellow and red lines. This will create a unified network, allowing the square passengers to travel from the top-left circle station to the bottom-right square station via a transfer at the central triangle station, thus alleviating the overcrowding. The disconnected triangle station has no passengers, so leaving it isolated for now is acceptable.

GUI Actions:
MOVE_TO(x:448, y:301),
DRAG_TO(x:711, y:555)

Explanation of the error:
The model misconstrued the game logic, operating under the false assumption that extending a line requires a prior disconnection. This led the agent to attempt a 'disconnect' move as a prerequisite, ultimately obstructing the execution of a feasible strategy.

Precise Execution error



Target:
design an efficient subway network to transport passengers

Reasoning:

In the current state, there is a new triangle station to the left that is not connected to any line. To ensure efficient passenger transport and prevent overcrowding, I will extend the red line to connect this new triangle station.

GUI Actions:

MOVE_TO(x:443, y:167),
DRAG_TO(x:143, y:242)

Explanation of the error:

While the model intended to extend existing lines, imprecise visual grounding of the UI elements caused it to inadvertently create new lines instead. This led to a rapid depletion of line resources, resulting in suboptimal performance.

Long Reasoning Latency



Target:
design an efficient subway network to transport passengers

Reasoning:

The current state shows four new, unconnected stations that are rapidly accumulating passengers. My immediate priority is to connect them to the network to prevent overcrowding and game failure. The new stations are ... Since I likely don't have a new line available, I must extend existing lines. I will extend the blue line, which is the closest, from its current endpoint at the triangle station (926, 233). I will connect it sequentially to ... This will service all three new stations in that area. I will connect the new circle station at (836, 479). This station is located south of the river. Instead of extending the already long blue line and using another tunnel, I will modify the shorter red line. I will disconnect the red line from ... This creates a more balanced red line (Circle-Triangle-Circle) without requiring a new tunnel. This two-part plan ensures all new stations are connected, preventing immediate failure, and makes a logical adjustment to the red line for better efficiency.

GUI ACTIONS:
MOVE_TO(x: 926, y: 233)
DRAG_TO(x: 1080, y: 233)
.....

Explanation of the error:
As the metro network grew in complexity, the model exhibited prolonged reasoning times before executing even simple tasks, such as connecting two stations. This excessive latency caused a rapid accumulation of passengers within the game, ultimately leading to a failure state.

Figure 62. mini metro typical errors

Error Analysis.

Three primary error categories were identified (Figure 62). The fundamental reason VLMs underperform compared to humans lies in the execution phase following reasoning. Models struggle to precisely localize minute UI elements such as stations, lines, and extension nodes (Figure 62 middle), leading to a frequent "reasoning-execution gap." Even when heuristic prompts explicitly provided station coordinates, execution failures persisted and are often fatal in a real-time environment. Furthermore, current models lack the "active attention" mechanism characteristic of human perception. When facing complex topological networks, models lack the ability to selectively focus only on changing components, resulting in unexpectedly long inference times during the later stages of the game (Figure 62 right). Additionally, models occasionally

exhibit hallucinations regarding rule interpretation, which unnecessarily inflates reasoning time and directly results in execution errors(Figure 62 left)

Video Reflection Analysis

The analysis of reflection results highlights a divergence in learning capabilities based on model scale. For smaller, high-speed models like Qwen3-VL-8B and Gemini-2.5-Flash, video reflection provided limited gains, indicating a deficiency in their ability to reflect and learn from visual history. In contrast, larger models such as Gemini-2.5-Pro and Qwen3-VL-32B successfully extracted critical heuristic experiences from the video data. These learned heuristics significantly reduced inference overhead, thereby enabling substantial performance improvements despite their inherent latency.

D.14. Genshin Impact

D.14.1 GAME DESCRIPTION FOR GENSHIN IMPACT

Game Environments. Genshin Impact (Mihoyo, 2020) is a vast open-world action RPG with an elemental combat system. The environment is rich with traversable terrain, puzzles, and enemies. The game tests embodied exploration and multi-tasking. Agents are evaluated on their ability to navigate complex 3D topography, understand elemental interaction mechanics, and follow multi-step quest instructions in a non-linear world. As a "Real-Time Non-Linear" benchmark, it represents one of the most difficult challenges, requiring the synthesis of exploration, combat, and puzzle-solving. The agent must process high-fidelity 3D visuals to identify traversable paths and enemies while managing real-time combat cooldowns and elemental combinations, mimicking a human player’s holistic engagement.



Figure 63. Screenshot of Genshin Impact

(1) **Game state.** Non-Linear (Open-Ended), High-Fidelity 3D world.

(2) **Main GUI action space.** Keyboard and Mouse (Movement, camera control, combat skills).

(3) **Evaluation task.** Finish the prologue. We impose a maximum limit of 100 steps per episode. The agent’s performance is quantified by the average number of completed milestones. The milestones are generated from the expert video. To obtain a standardized score, the number of completed milestones is normalized by the total number of milestones, and the calculation formula is as follows:

$$\text{Score} = \frac{N_{\text{completed}}}{N_{\text{total}}} \times 100$$

where $N_{\text{completed}}$ denotes the number of milestones completed by the agent, and N_{total} denotes the total number of milestones (7 in total, as listed in Table 25). The normalized score ranges from 0 to 100.

Milestones

1. **Basic Operation:** Follow Paimon’s instructions to learn the basic operations.
2. **Unlock The Statue of The Seven:** Interact with the first Statue of The Seven at Starfell Lake.
3. **First Combat Encounter:** The player encounters an enemy and completes the combat tutorial using the newly acquired skills.
4. **Encounter with Dvalin:** The player enters the Whispering Woods and triggers the plot where the dragon Dvalin interacts with the bard.
5. **Meet Amber:** The player encounters Amber, a scout knight of the Knights of Favonius, on the way to Mondstadt.
6. **Clear the Hilichurl camp:** The player clears the nearby Hilichurl camp to proceed further.
7. **Arrive at Mondstadt:** The player crosses the bridge and enters the gate of Mondstadt, marking the end of the prologue.

Table 25. Milestones of Genshin Impact

(4) **Expert video content.** The expert video is a demonstration video. It demonstrates exploring an open world, collecting

resources, navigating to mission locations using maps and mini-map, triggering story events and dialogues, and utilizing skills and teamwork to complete combat.

D.14.2 GAME PROMPT FOR GENSHIN IMPACT

We provide the full structure of our prompts in Figure 64 and Figure 65.

D.14.3 DETAILED ANALYSIS FOR GENSHIN IMPACT

The experimental results in *Genshin Impact*, as presented in Table 26, reveal a uniform performance pattern across all evaluated general VLMs: all models achieved an identical score of 14.3 ± 0.0 , corresponding to the completion of only the first milestone (*Basic Operation*). This consistent result indicates two key insights: first, mainstream general VLMs possess a baseline capability to execute simple, explicit instructions (e.g., following Paimon’s guidance for basic controls), confirming their fundamental command-following competence. Second, the failure to progress beyond the initial milestone exposes critical deficits in their ability to operate within 3D open-world environments, including deficiencies in spatial distance perception, dynamic environmental awareness, and continuous action execution.

Qualitative analysis of typical errors in Figure 66 further elaborates these limitations. In the left panel, the model demonstrates a failure in sustained action control: after issuing a movement instruction, it failed to release the ‘W’ key, causing uninterrupted movement that disrupted subsequent reasoning and task progression. This error highlights the inability of general VLMs to maintain precise, context-aware control over continuous in-game actions—a critical requirement for navigating 3D spaces. In the right panel, the model exhibits erroneous environmental perception: it misinterpreted attack animations triggered by prior incorrect operations as initiation of combat with Hilichurls, despite the absence of enemies in the scene. This hallucination reflects a profound weakness in dynamic 3D scene understanding, where the model cannot reliably distinguish between transient visual artifacts and actual environmental events, leading to false strategic judgments.

Collectively, these findings demonstrate that while current general VLMs can handle isolated, 2D-style GUI interactions or simple discrete commands, they lack the robust spatial intelligence and dynamic reasoning capabilities required for 3D open-world tasks. The uniform performance across all models and configurations underscores a systemic limitation: existing general VLMs are not yet equipped to perceive, reason about, and act consistently within the complex, evolving spatial contexts of games like *Genshin Impact*.

Model	Zero-shot	Memory	Zero-shot + VR.	Memory + VR.
Gemini-2.5-Flash	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0
Gemini-2.5-Pro	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0
Qwen3-VL-8B	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0
Qwen3-VL-32B	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0
GPT-4o	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0
GPT-4o-mini	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0
Seed-1.8	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0	14.3 ± 0.0

Table 26. Scores in Genshin Impact (Mean $\pm$ Std)

System Prompt

You are an AI agent playing Genshin Impact. You are the 'Traveler' exploring the world of Teyvat. Your goal is to complete quests, explore the open world, defeat enemies, and solve puzzles.
Game Window Size: {window_width} x {window_height} pixels.

Core Gameplay Mechanics

...Basic Mechanics Introduction...

Controls & Inputs

...Control and Interaction Keys...

Screen Information (HUD)

...HUD Description...

Strategic Guidelines

...Action Guidelines...

Action Space

Action Types:

'KEY_DOWN', 'KEY_UP', 'PRESS': Keyboard interaction. One key per action. 'KEY_DOWN' holds the key and MUST be paired with 'KEY_UP' to release it.

'HOTKEY': Press multiple keys simultaneously (e.g., Alt+1). Parameters: 'keys' (list of strings).

'MOVE_BY': Camera Control. 'dx' (yaw), 'dy' (pitch).

dx > 0: Turn Right. dx < 0: Turn Left.

dy > 0: Look Down. dy < 0: Look Up.

Larger values (e.g., 500+) mean larger turns.

'MOVE_TO': UI Interaction. Absolute coordinates (x, y). Only works in Menus or holding Alt.

'CLICK', 'RIGHT_CLICK': Mouse clicks.

'WAIT': Essential for animations (attacks, climbing, UI transitions).

Action Examples

Example 1 - Move Diagonally & Sprint: [

```
{ "action_type": "KEY_DOWN", "parameters": { "key":
w} },
{ "action_type": "KEY_DOWN", "parameters": { "key":
a} },
...
{ "action_type": "KEY_UP", "parameters": { "key":
w} },
{ "action_type": "KEY_UP", "parameters": { "key":
a} }
]
```

...Examples of Action Output...

Output Requirements

...Requirements...

User Prompt

Target task

{task_description}

Previous state



Last executed actions

```
[
  { "action_type": "KEY_DOWN", "parameters": { "key":
"w" } },
  { "action_type": "CLICK", "parameters": { "button":
"right" } },
  { "action_type": "WAIT", "parameters": {
"duration": 2.0} },
  { "action_type": "KEY_UP", "parameters": { "key":
"w" } }
]
```

Current state



Navigation & Exploration Guidelines

...Guidelines...

Provide your response in the strict format:

Reasoning

[Describe what you see in the image and your strategic thinking]

Actions

...

Figure 64. Genshin Impact zero-shot prompt

System Prompt

You are an AI agent playing Genshin Impact. You are the 'Traveler' exploring the world of Teyvat. Your role is to control the game via keyboard and mouse actions, then decide if information should be saved to long-term memory.

Game Window Size: {window_width} x {window_height} pixels.

Your goal is to complete quests, explore the open world, defeat enemies using elemental reactions, and solve puzzles.

Core Gameplay Mechanics

...Basic Mechanics Introduction...

Controls & Inputs

...Control and Interaction Keys...

Screen Information (HUD)

...HUD Description...

Strategic Guidelines

...Action Guidelines...

Long-term Memory (CRITICAL)

IMPORTANCE: Long-term memory persists beyond short-term limits. Essential for remembering quest objectives, important locations, enemy patterns, puzzle solutions, and strategic insights from earlier gameplay.

SAVE WHEN: 1. **Quest Information:** Quest objectives, NPC locations, quest item locations 2. **Important Locations:** Teleport waypoints, chest locations, puzzle locations, important landmarks 3. **Combat Strategies:** Effective character combinations, enemy weaknesses, elemental reaction patterns 4. **Puzzle Solutions:** Puzzle mechanics, solutions, patterns 5. **Resource Locations:** Important resource gathering spots, material locations 6. **Strategic Insights:** Effective exploration routes, stamina management strategies

DON'T SAVE: Routine movement, temporary combat states, already collected items (unless new insight), duplicates (unless adds context).

GUIDELINES: Be generous (when in doubt, save), be specific (include context and location), be proactive (save as soon as identified).

Action Space

...Available Action Types...

Learned Experience from Previous Attempts

{reflection_experience}

Action Examples

...Examples of Action Output...

Output Format

Reasoning

[Analyze screenshot, consider history summary, reasoning, plan, retrieved memories. Explain what to do and why]

Actions

```
[
  {"action_type": "...", "parameters": {...}}
]
```

New_memory

[Important info with context, or "None" if nothing important]

Should_save

[Yes if ANY important info, No ONLY if truly nothing. When in doubt, Yes.]

User Prompt

Target task

{task_description}

History Summary

{history_summary}

Reasoning (Previous)

{reasoning}

Plan

{plan}

Retrieved Long-term Memory

{retrieved_memory_str}

Last executed action

{action}

Current state



Task & Memory Saving

Task: (1) Analyze screenshot, consider history/reasoning/plan/memories, (2) Determine best action, (3) Decide if ANY important info should be saved to long-term memory.

Memory Saving: Save quest information, important locations, combat strategies, puzzle solutions, resource locations, strategic insights. Be proactive: when in doubt, save it. Be specific: include context and location.

Response Format

Reasoning

[Your reasoning]

Actions

```
[
  {"action_type": "...", "parameters": {...}}
]
```

New_memory

[Important info with context, or "None" if nothing important]

Should_save

[Yes if ANY important info, No ONLY if truly nothing. When in doubt, Yes.]

Figure 65. Genshin Impact memory agent prompt

Incorrect Instructions



Target:
Kill or avoid the slime encountered along the way.

Reasoning:
The Traveler is currently in a climbing state on a vertical wall, having likely ascended here to dodge the Lv2 Hydro Slime's water burst attacks. Our core objective remains defeating this slime to unlock access to the red coastal totem for the "Wanderer's Footprints" sub-quest. To resume combat efficiently, we first need to drop back to the ground—using the X key will immediately end climbing without expending additional stamina, which is optimal given our limited health and lack of healing resources. Once on the ground, we will re-engage the slime by left-strafting to dodge its bursts, landing consistent melee strikes, and using the Anemo Elemental Skill on cooldown to speed up damage output (since elemental reactions are not possible with only Anemo and Hydro).

GUI Actions:
PRESS(key: x),
WAIT(duration:0.5),
KEY_DOWN(key: w),
WAIT(duration:1.0),
CLICK(button: left),
WAIT(duration:0.3)

Explanation of the error:
The model's reasoning provided the correct approach; however, after issuing the movement instructions, the 'w' key was pressed and not released, causing the character to continue moving during subsequent reasoning, which resulted in a significant error.

Misperception of the Environment



Target:
Follow Paimon's guidance

Reasoning:
The Traveler is engaged in the first main quest combat encounter with a Hilichurl on the eastern Mondstadt beach. Since we only have the Anemo Traveler in the party, continuing normal melee attacks is the only viable strategy to defeat the enemy and progress the quest. Completing this combat will unlock subsequent dialogue from Paimon, which is critical for learning Teyvat's core mechanics and advancing the main objective of finding the Traveler's sibling.

GUI Actions:
CLICK(button: left),
WAIT(duration:0.4)

Explanation of the error :
The model erroneously inferred the attack animation error caused by a previous misoperation, thinking that it had entered the combat phase with the Hilichurl.

Figure 66. Genshin Impact errors

D.15. Red Dead Redemption 2

D.15.1 GAME DESCRIPTION FOR RED DEAD REDEMPTION 2

Game Environments. Red Dead Redemption 2 (Rockstar Games, 2019) is a highly realistic open-world simulation, known for its narrative depth and environmental realism. The setting is a detailed recreation of the American frontier. This game tests comprehensive general intelligence, including social interactions, realistic physics navigation, and narrative comprehension.

Agents must operate within a living world where characters and animals react realistically, requiring a high degree of common sense and contextual understanding. It stands as the "Hard" tier of the "Real-Time Non-Linear" category, challenging the agent to handle partially observable, long-horizon tasks that demand not just game mechanics mastery, but an understanding of social cues and realistic environmental constraints.



Figure 67. Screenshot of Red Dead Redemption 2

(1) **Game state.** 3D open-world environment with realistic physics, dynamic scenes, and interactive NPCs.

(2) **Main GUI action space.** Keyboard and Mouse (Complex interactions, movement, aiming).

(3) **Evaluation task.** Complete the milestones of Chapter 1 in *Red Dead Redemption 2*. We impose a maximum limit of 200 steps per episode. The agent's performance is quantified by the number of successfully completed milestones within Chapter 1. The total set of milestones is extracted from expert demonstration videos. The evaluation score is normalized by the total number of Chapter 1 milestones. The Chapter 1 milestones, in chronological order, are: *First Control: The Journey*

Begins, Arrival at Adler Ranch, First Gunfight, Looting Supplies, Barn Ambush and Interrogation, First Encounter with Sadie Adler, Mission Complete: Outlaws from the West, Mission Start: Enter, Pursued by a Memory, Wolf Encounter, and Finding John Marston. The normalized score is defined as follows:

$$S_{norm} = \frac{N_{agent}}{N_{total}} \times 100,$$

where S_{agent} denotes the number of milestones completed by the agent during evaluation, and S_{total} denotes the total number of milestones in Chapter 1.

(4) Expert video content. The expert video consists of a full gameplay recording of Chapter 1 completed by an experienced player. The video demonstrates effective long-horizon decision-making in an open-world setting, including navigation, combat engagement, and context-aware interactions with NPCs and the environment. By implicitly modeling task prioritization, situational awareness, and mission-driven action selection under complex and dynamic conditions, the expert gameplay provides a reference trajectory that highlights efficient progression through chapter milestones. It guides agents to align low-level control with high-level narrative objectives and to make decisions that favor robustness and temporal coherence over short-term reactive behaviors.

D.15.2 GAME PROMPT FOR RED DEAD REDEMPTION 2

We provide the full structure of our prompts in Figure 68.

D.15.3 DETAILED ANALYSIS FOR RED DEAD REDEMPTION 2

The experimental results on Red Dead Redemption 2 (RDR2) reveal a pronounced performance ceiling that persists across model scaling and enhanced reasoning strategies. As shown in Figure 69, the transition from structured 2D interfaces to a dynamic 3D open-world environment poses a substantial challenge for current VLM-based agents. While the reflection mechanism yields marginal improvements for Gemini-2.5-Flash and Qwen3-VL-32B, most frontier models, including GPT-4o and Gemini-2.5-Pro, remain near a baseline score of 10.0. This stagnation indicates a fundamental limitation in bridging discrete GUI execution with continuous spatial grounding. Notably, Seed-1.8 exhibits degraded performance under reflection, suggesting that iterative reasoning may introduce instability or action loops that interfere with responsiveness to real-time environmental feedback.

Model	Zeroshot	Memory	Memory VR.
Gemini-2.5-Flash	7.0 ± 5.0	10.0 ± 0.0	13.0 ± 5.0
Gemini-2.5-Pro	10.0 ± 0.0	10.0 ± 0.0	10.0 ± 0.0
Qwen3-VL-8B	7.0 ± 5.0	7.0 ± 5.0	7.0 ± 5.0
Qwen3-VL-32B	10.0 ± 8.0	7.0 ± 5.0	10.0 ± 8.0
GPT-4o	7.0 ± 5.0	10.0 ± 0.0	10.0 ± 0.0
GPT-4o-mini	7.0 ± 5.0	10.0 ± 0.0	10.0 ± 0.0
Seed-1.8	7.0 ± 5.0	10.0 ± 0.0	7.0 ± 5.0

Figure 69. Scores in Red Dead Redemption 2

Perception–Reasoning Gap.

We observe a dominant Perception–Reasoning Gap in Red Dead Redemption 2, where agents rely on projection-based perception that treats targets and obstacles as reachable points in 2D screen space rather than as volumetric entities with physical constraints. As shown in Figure 70 (left), physical obstacle misperception leads to systematic misclassification of dense foliage, terrain irregularities, and environmental structures as traversable regions, resulting in repeated forward actions despite physical infeasibility.

These perceptual limitations propagate into navigation-level reasoning errors. Although mini-map objectives are correctly identified at the semantic level, agents fail to maintain a stable transformation between allocentric mini-map coordinates and egocentric camera orientation, causing orientation drift over time (Figure 70, middle). When forward actions yield little or no displacement, agents do not attribute the outcome to environmental constraints but instead persist with forward-biased heuristics and minor camera adjustments, entering repetitive action loops without recovery behaviors (Figure 70, right). Together, these results suggest that in open-world environments with implicit physical boundaries, reasoning effectiveness is fundamentally constrained by the agent’s ability to infer 3D structure and navigability from visual and action feedback.

Video Reflection Analysis.

System Prompt

You are an AI agent playing Red Dead Redemption 2, specifically progressing through Chapter 1: Colter. Your role is to control the game only via keyboard and mouse actions, then decide if information should be saved to long-term memory.

Game Overview and Guidelines

Red Dead Redemption 2 is an open-world Western action-adventure game set in 1899 America. You play as Arthur Morgan.

Gameplay Guidelines:

1. Follow yellow objective markers on minimap and compass.
2. Watch for on-screen button prompts during missions (yellow on-screen text may correspond to yellow objective markers)
3. Complete objectives in order - some are optional (indicated by text)

Long-term Memory (CRITICAL)

IMPORTANCE: Long-term memory persists beyond short-term limits. Essential for remembering mission objectives, important locations, NPC information, story context, environmental constraints, and strategic insights derived from both success and failure.

Screen Information

Health bar (red): Bottom left - Your character's health.

Minimap: Bottom left corner - Shows nearby enemies, objectives, and terrain.

Yellow markers on minimap and in world showing mission objectives.

The White dot in the center of the minimap: Your current position and facing direction.

Other white dots on minimap: Gang members or allies.

Red dots on minimap: Enemies.

Weapon indicator: Bottom right - Current equipped weapons and ammo count.

Learned Experience from Previous Attempts

[reflection content learned from video]

Note: The above experience comes from analyzing previous failed attempts and expert playthroughs. Use these insights to improve your strategy and avoid repeating past mistakes.

Main Controls

You can only interact with the game using keyboard and mouse actions.

For camera control in RDR2, use MOVE_BY to adjust view direction. Forward movement is implicitly aligned with the updated camera and view direction.

IMPORTANT: Most of the time, on the screen there will be on-screen prompts and icons indicating the appropriate key to press for interactions.

...gui action space...

Output Format(gui)

You must output your actions as a JSON array. Each action has an `action_type` and `parameters`.

Reasoning: [Analyze screenshot, consider history summary, reasoning, plan, retrieved memories. Explain what to do and why] Actions: Output actions as a JSON array. Each action has `'action_type'` and `'parameters'` fields.

```
[{
  "action_type": "...", "parameters": {...}},
]
```

Example - Walk for 3 seconds:

```
[{
  "action_type": "KEY_DOWN", "parameters": {"key": "w"}},
{"action_type": "WAIT", "parameters": {"duration": 3.0}},
{"action_type": "KEY_UP", "parameters": {"key": "w"}}
]
```

New_memory:

[If ANY important info is discovered, describe it clearly with context. Includes missions, locations, NPCs, story, combat or strategy, environmental constraints, blocked paths / invisible walls, and repeated failures or stuck situations. If nothing important, write "None".

Should_save:

[Answer "Yes" if ANY info may help future decisions or avoid repeated failure. Answer "No" ONLY if truly nothing important. When in doubt, choose "Yes".

User Prompt

Target task

(1) Analyze screenshot, consider history/reasoning/plan/memories, (2) Determine best action, (3) Decide if ANY important info should be saved to long-term memory.

Previous state



Last executed actions

```
[
  { "action_type": "KEY_DOWN", "parameters": { "key":
    "W" } },
  { "action_type": "WAIT", "parameters": {
    "duration": 12.0 } },
  { "action_type": "KEY_UP", "parameters": { "key":
    "W" } }
  ...
]
```

Current state



Memory Saving

Save mission information, important locations, NPC information, story context, combat strategies, strategic insights.

Be proactive: when in doubt, save it. Be specific: include context and location.

Your Response

You should only respond in the format described below, and you should not output comments or other information.

Respond in the following format:

Reasoning

[Describe what you see in the image and your strategic thinking]

Actions

...

New_memory

Important info with context, or None if nothing important

Should_save

Yes if ANY important info, No ONLY if truly nothing. When in doubt, Yes.

Figure 68. Red Dead Redemption 2 Prompt

Physical Obstacle Misperception



Target:
Follow the yellow objective marker shown on the minimap to reach the designated destination.

Reasoning:
Based on the screenshot, I am on horseback, and the objective, marked by the yellow icon on the minimap, is the lit cabin ahead. My view is partially obstructed by trees and bushes. The previous action successfully **moved me closer to the cabin**. To continue progressing, I need to navigate around the remaining **foliage and approach the front of the building**. I will adjust my camera slightly to the right to get a clearer path and then move forward to get closer to the objective.

GUI Actions:
"action_type": "KEY_DOWN", "parameters": {"key": "w"}
"action_type": "WAIT", "parameters": {"duration": 3.0}
"action_type": "KEY_UP", "parameters": {"key": "w"}

Explanation of the error:
The model treats both the target and obstacles as reachable points in 2D screen space, without explicitly modeling spatial occupancy or non-penetrability. This projection-based perception causes the agent to ignore physical constraints and repeatedly execute ineffective forward actions.

Minimap Geometric Error



Target:
Follow the yellow objective marker shown on the minimap to reach the designated destination.

Reasoning:
The last action of moving forward and left was successful, as I have advanced along the narrow path between the building on the left and the trees on the right. The objective remains \Approach the hitching posts,\ and the yellow marker on the minimap indicates the destination is still ahead and **slightly to my left**. The path forward is still constrained. To continue making progress without getting stuck, I will first adjust my camera view slightly to the left to better align with the path and the objective marker. Then, I will move forward to get closer to the hitching posts.

GUI Actions:
"action_type": "MOVE_BY", "parameters": {"dx": -100, "dy": 0}
"action_type": "WAIT", "parameters": {"duration": 0.5}
"action_type": "KEY_DOWN", "parameters": {"key": "w"}
"action_type": "WAIT", "parameters": {"duration": 2.0}
"action_type": "KEY_UP", "parameters": {"key": "w"}

Explanation of the error :
After initial orientation errors, the model does not update its estimate of self-heading and continues reasoning based on an incorrect spatial assumption. This misalignment between egocentric perception and allocentric minimap coordinates causes errors to accumulate over time (drifting).

Implicit Boundary detection Failure



Target:
Follow the yellow objective marker shown on the minimap to reach the designated destination.

Reasoning:
The character is riding a horse through a snowy, foggy forest. The last action of pressing the forward key (W) for 3 seconds did not result in noticeable movement, likely due to terrain obstruction or slow progress in snow. To continue progressing toward the objective, I should adjust camera direction and **attempt moving forward again**. Since the minimap shows an objective ahead, I will slightly adjust the camera view to ensure proper alignment with the path and then press forward again.

GUI Actions:
"action_type": "MOVE_BY", "parameters": {"dx": -100, "dy": 0}
"action_type": "KEY_DOWN", "parameters": {"key": "w"}
"action_type": "WAIT", "parameters": {"duration": 2.0}
"action_type": "KEY_UP", "parameters": {"key": "w"}

Explanation of the error :
Although the model observes that forward actions do not result in movement, it fails to attribute this feedback to environmental infeasibility. Due to the lack of explicit visual cues for invisible boundaries, the agent continues to assume the area is traversable, revealing limited capability in inferring implicit physical constraints from action-outcome feedback.

Figure 70. red dead redemption 2 errors

A notable observation in Red Dead Redemption 2 is that the effectiveness of reflection is strongly constrained by perceptual grounding rather than high-level reasoning capacity. For powerful models such as Gemini-2.5-Pro and GPT-4o, reflection yields negligible gains, as dominant failures stem from a Perception–Reasoning Gap in 3D spatial understanding. Smaller models similarly fail to benefit, with reflection often amplifying speculative reasoning that is weakly aligned with immediate control signals. In contrast, mid-tier models show limited but consistent improvements, where reflection helps suppress narrative over-interpretation and re-anchor decisions to explicit mission instructions, as evidenced by the shift from exploratory objective inference to instruction-following behavior in the before/after comparison (Figure 71).

However, reflection does not correct core perceptual errors such as obstacle misclassification or mini-map camera misalignment, leading agents to persist in forward-biased action loops. Overall, these results suggest that in open-world environments with implicit physical constraints, reflection is beneficial only when perceptual grounding is sufficiently reliable; otherwise, it may reinforce misaligned reasoning rather than improve execution.

Before Reflection

...The next logical step is to reach the precise objective marker, which will likely trigger a dismount and further interaction or investigation...

After Reflection

...My previous action was to hold 'W' to follow, and this action is still appropriate and necessary to progress the mission. There are no new prompts or environmental cues suggesting a change in strategy or objective. Therefore, the most logical and effective action is to continue following Dutch as instructed...

Figure 71. Gemini-2.5-Flash reflection improvement example